

Lecture 04

Priority Queues, and Heaps

Priority Queue ADT

- The data in a **priority queue** is a queue of elements with priority.
- The priority can be either max or min:
 - A PQ with max priority always removes the largest element first.
 - A PQ with min priority always removes the smallest element first.
- The priority is fixed during the construction of the PQ.
- This chapter focuses on PQ with max priority only.

Priority Queue ADT

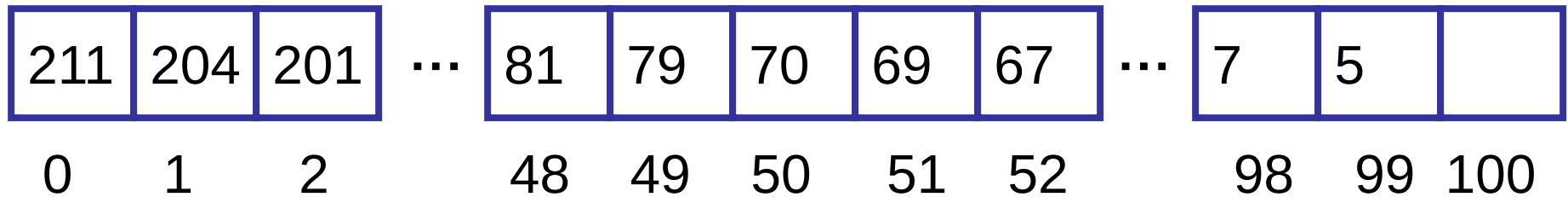
Operations

- ***enqueue***, an operation to add an element to the queue.
- ***dequeue***, an operation to remove the largest element from the queue.
- *isEmpty*, an operation to determine whether or not the queue is empty.
- *clear*, an operation to empty out the queue.

Priority Queue Implementation

- To implement a priority queue, an array sorted in descending order comes to mind
- Dequeueing from a sorted array is easy – just get the value at the current front and increment a front index – this is $\Theta(1)$ time
- However, enqueueing into a sorted array would take some time – the element would have to be inserted into its proper position in the array...

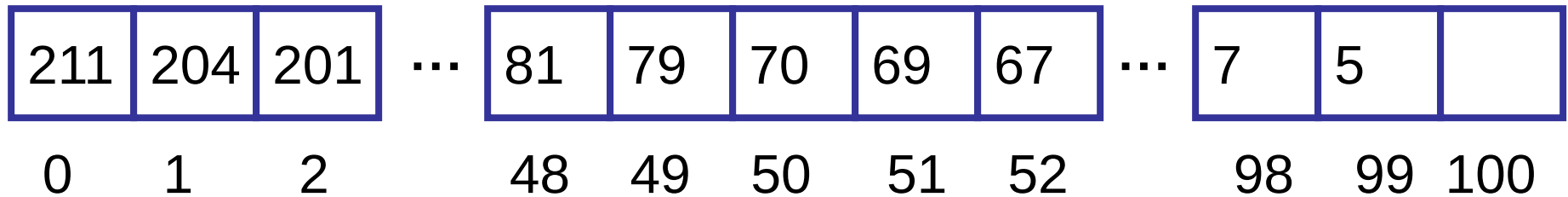
Enqueuing an Element



In this array of elements, each element might be an object, but only the data member considered for maximum value is shown.

Enqueueing an Element (cont.)

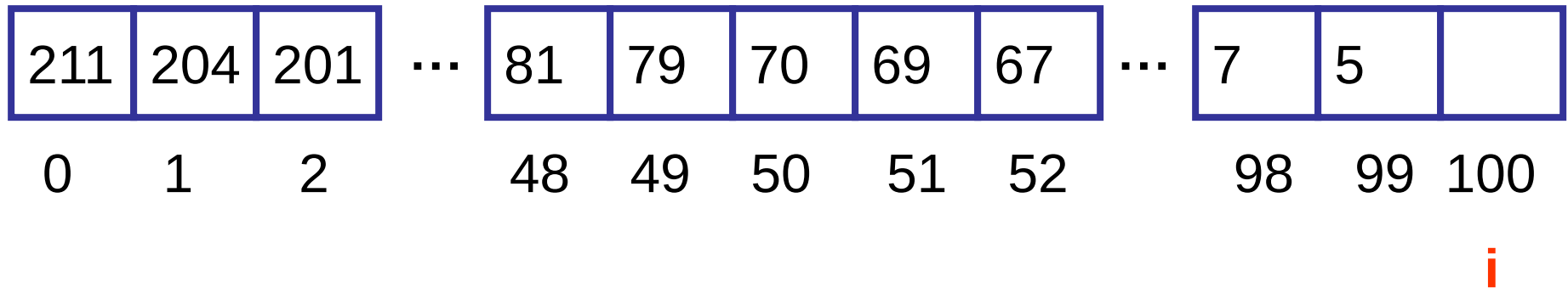
item: 71



**Suppose that element 71 needs to be enqueued.
We could enqueue 71 by using a loop.**

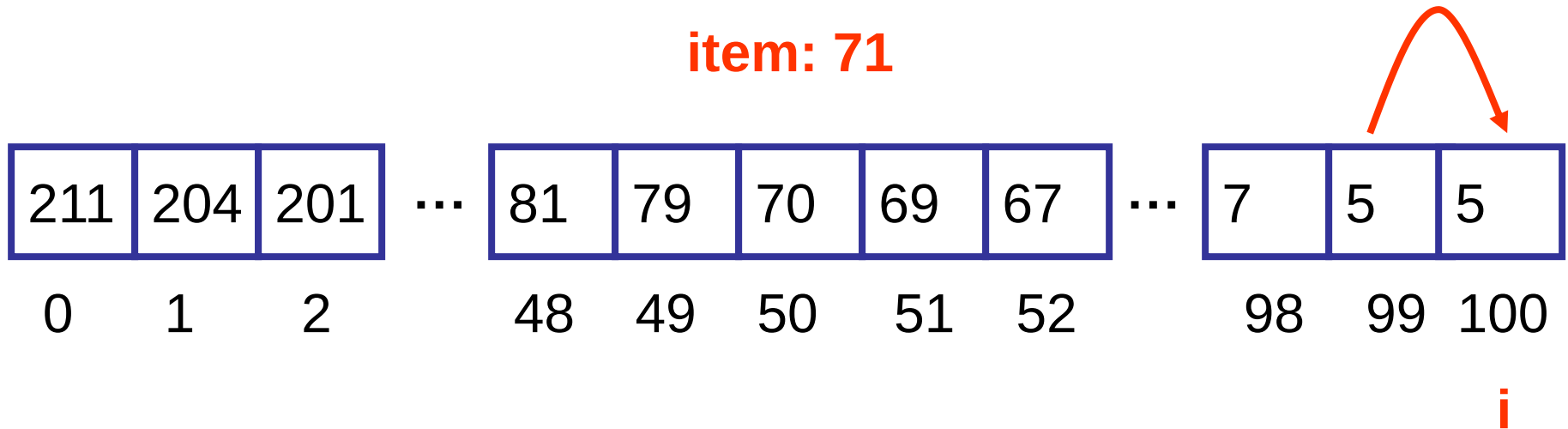
Enqueueing an Element (cont.)

item: 71



```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- )  
    arr[ i ] = arr[ i - 1 ];
```

Enqueueing an Element (cont.)

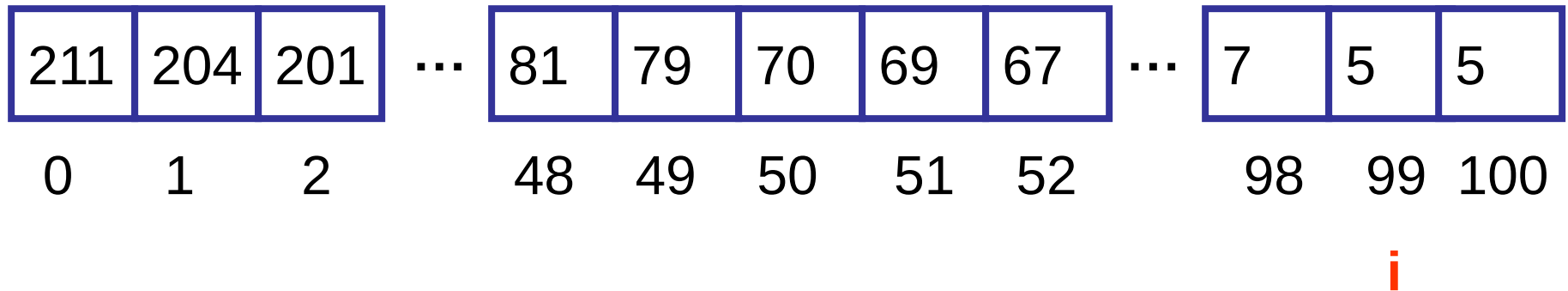


```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- )
```

```
    arr[ i ] = arr[ i - 1 ];
```

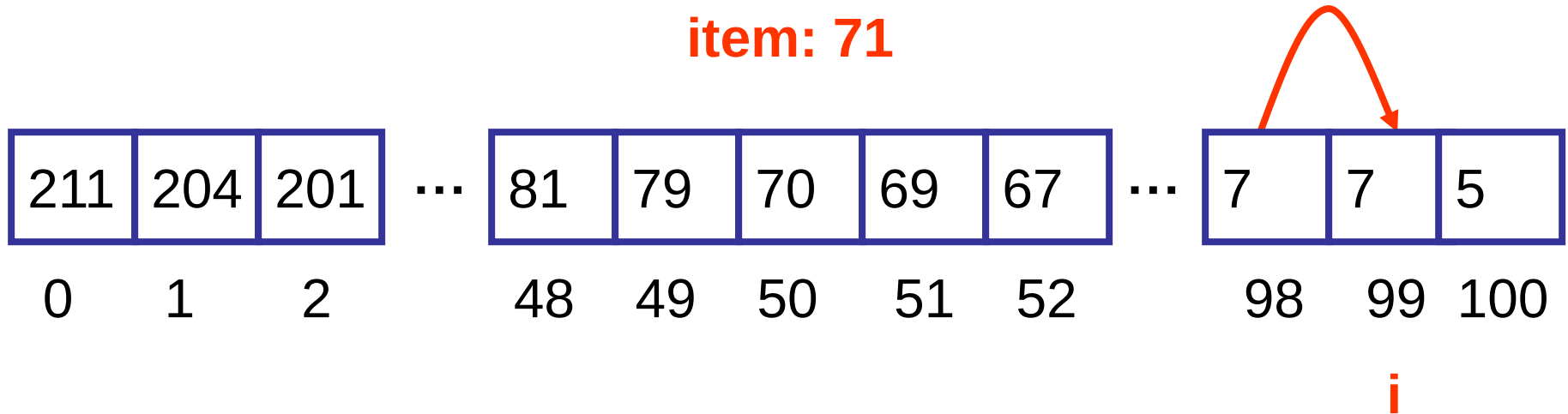
Enqueueing an Element (cont.)

item: 71



```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- )  
    arr[ i ] = arr[ i - 1 ];
```

Enqueueing an Element (cont.)

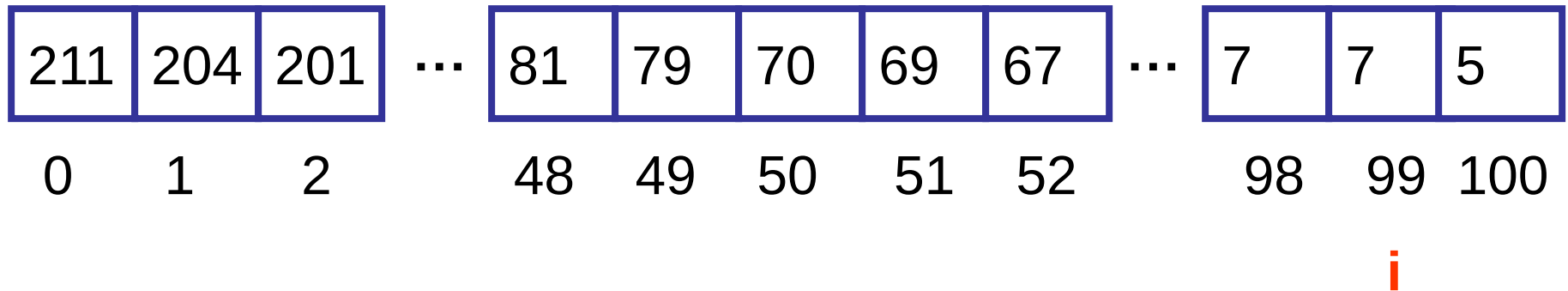


```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- )
```

```
    arr[ i ] = arr[ i - 1 ];
```

Enqueueing an Element (cont.)

item: 71

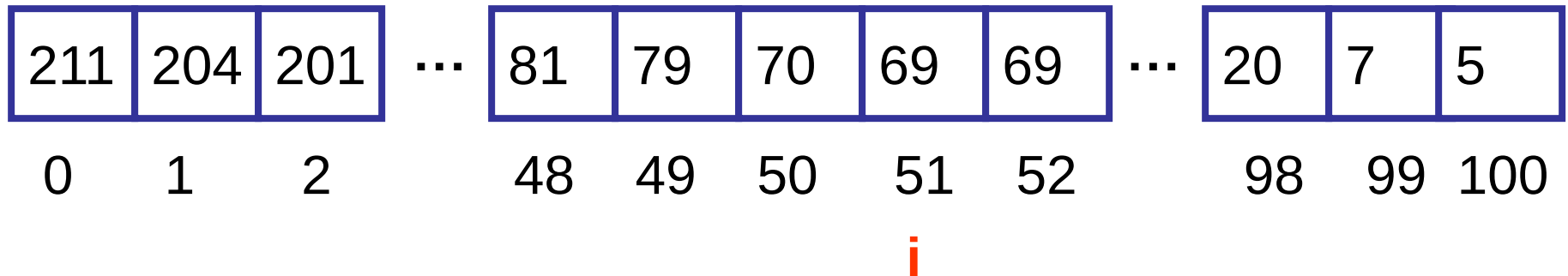


```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- )  
    arr[ i ] = arr[ i - 1 ];
```

**This process
continues and i
eventually
becomes 51**

Enqueueing an Element (cont.)

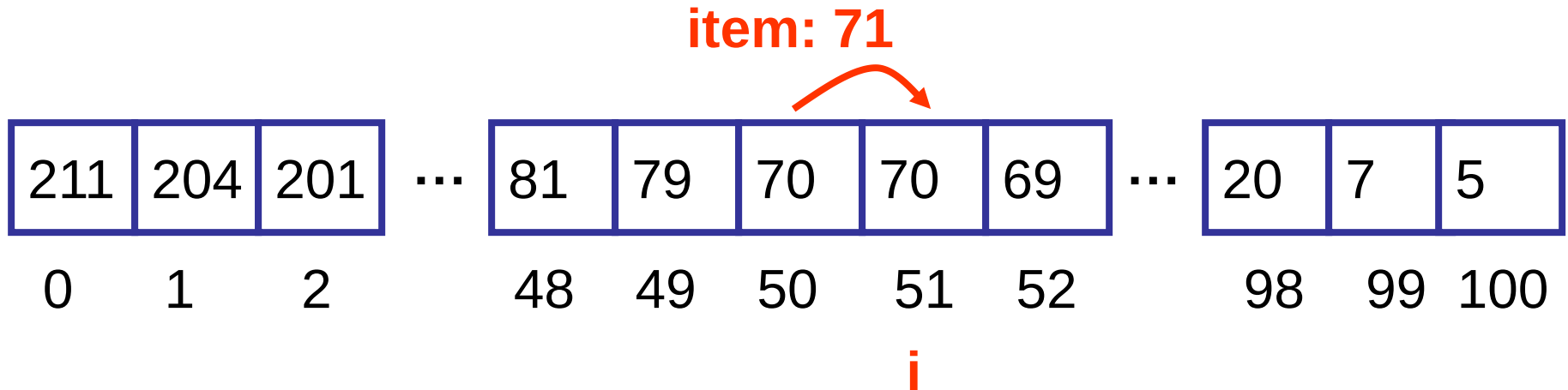
item: 71



```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- )  
    arr[ i ] = arr[ i - 1 ];
```

This process continues and i eventually becomes 51

Enqueueing an Element (cont.)

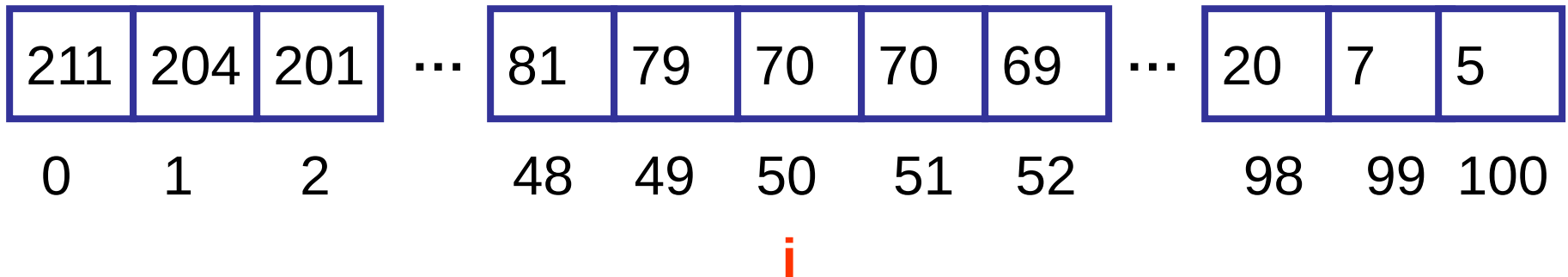


```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- )
```

```
    arr[ i ] = arr[ i - 1 ];
```

Enqueueing an Element (cont.)

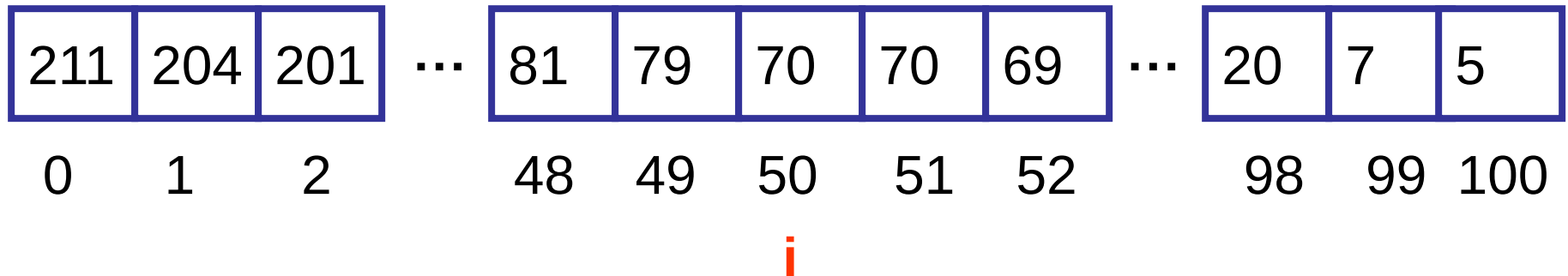
item: 71



```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- )  
    arr[ i ] = arr[ i - 1 ];
```

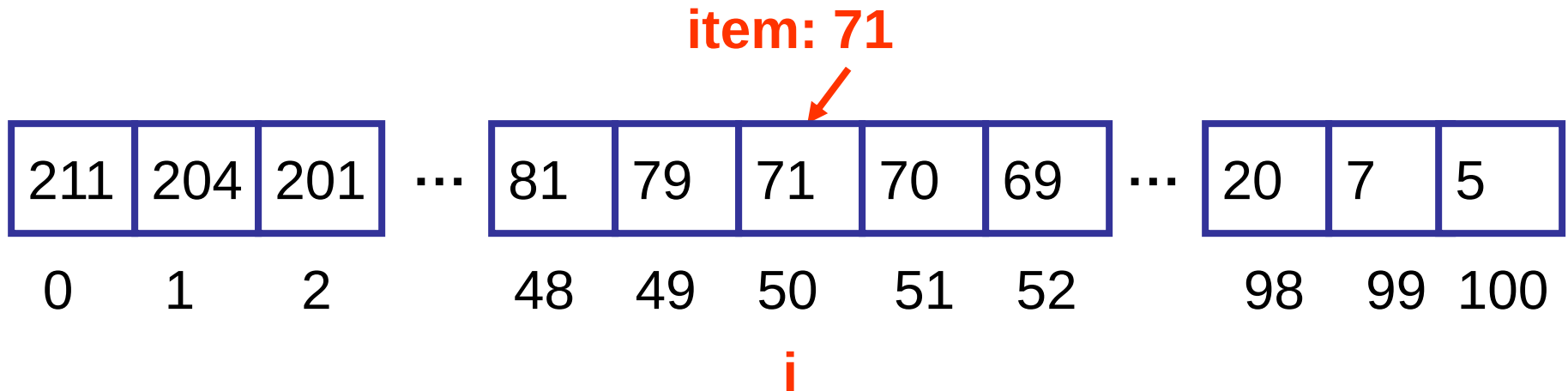
Enqueueing an Element (cont.)

item: 71



```
for ( i = 100; i > 0 && arr[ i - 1 ] < item; i-- ) FALSE  
    arr[ i ] = arr[ i - 1 ];
```

Enqueueing an Element (cont.)



Now we can use:

```
arr[ i ] = item;
```

to enqueue the element

Enqueueing an Element (cont.)

- If we assume that, on average, half the elements in an array need to be shifted to insert an element, then the enqueue for an array is a $\Theta(n)$ algorithm.
- In summary, when using an array for a priority queue:
 - dequeue is $\Theta(1)$
 - enqueue (on average) is $\Theta(n)$

Using a Heap to Implement a Priority Queue

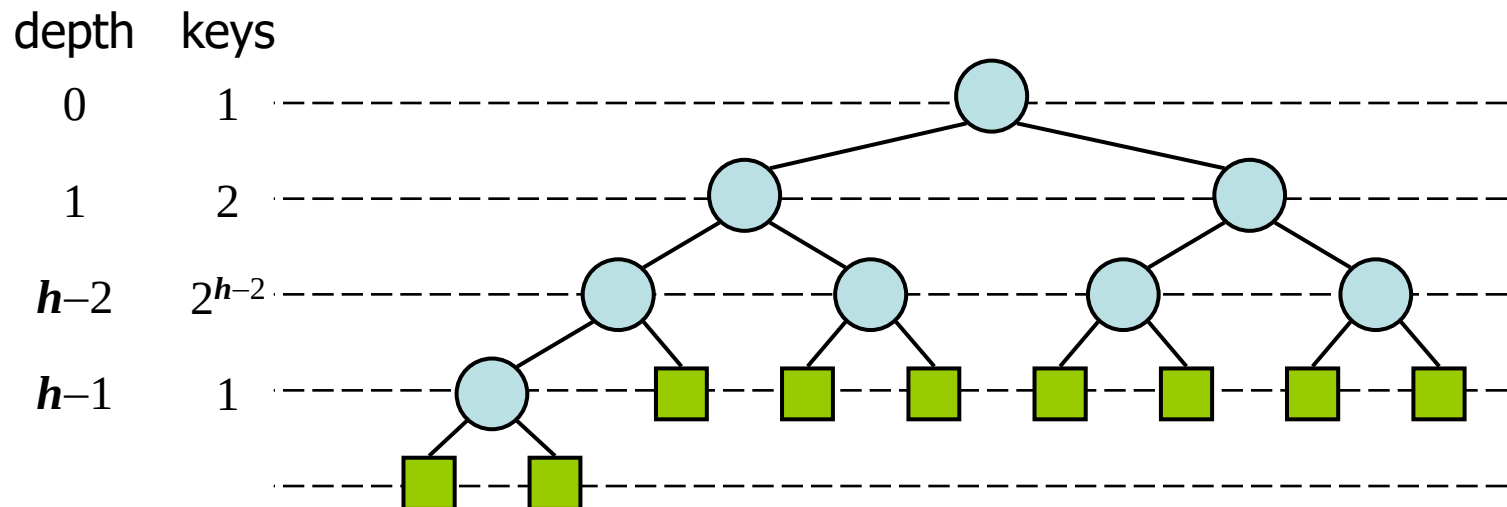
- An alternative to using a sorted array for a priority queue is to use a *heap*.
- Here, heap does not mean an area of memory used for dynamic allocation – rather, it is a data structure (complete binary tree).
- Dequeue in a heap is a $O(\lg n)$ operation
- Enqueue in a heap is a $O(\lg n)$ operation

Height of a Heap

- Theorem: A heap storing n keys has height $O(\log n)$

Proof: (we apply the complete binary tree property)

- Let h be the height of a heap storing n keys
- Since there are 2^i keys at depth $i = 0, \dots, h-2$ and at least one key at depth $h-1$, we have $n \geq 1 + 2 + 4 + \dots + 2^{h-2} + 1$
- Thus, $n \geq 2^{h-1}$, i.e., $h \leq \log n + 1$



Comparing Operations

- Which is better, heap or sorted array?
- We often eliminate a data structure that has a high time complexity in a commonly used operation, even if the other operations have very low time complexities.
- In the array, on average, an enqueue-dequeue pair of operations takes $\Theta(n) + \Theta(1)$ time, but $\Theta(1)$ is absorbed into $\Theta(n)$, leaving us with an overall time complexity of $\Theta(n)$ per pair of operations.

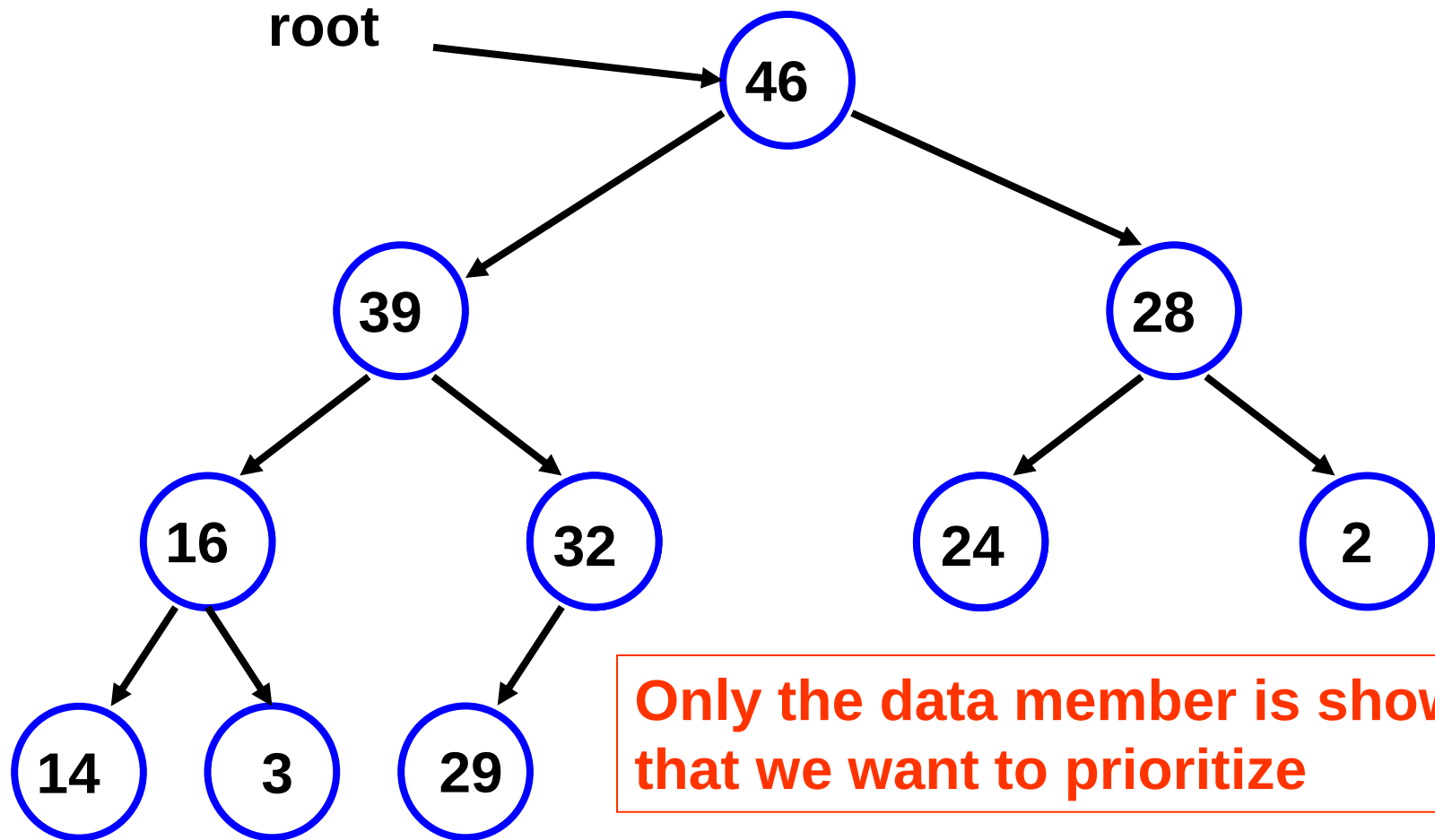
Comparing Operations (cont.)

- In the heap, each enqueue-dequeue pair of operations takes $O(\lg n) + O(\lg n)$ time, giving us an overall time complexity of $O(\lg n)$ per pair of operations.
- The heap is usually better.
- Array can be good if the elements are supplied in sorted manner, then only dequeue operations are performed (no enqueue operations).

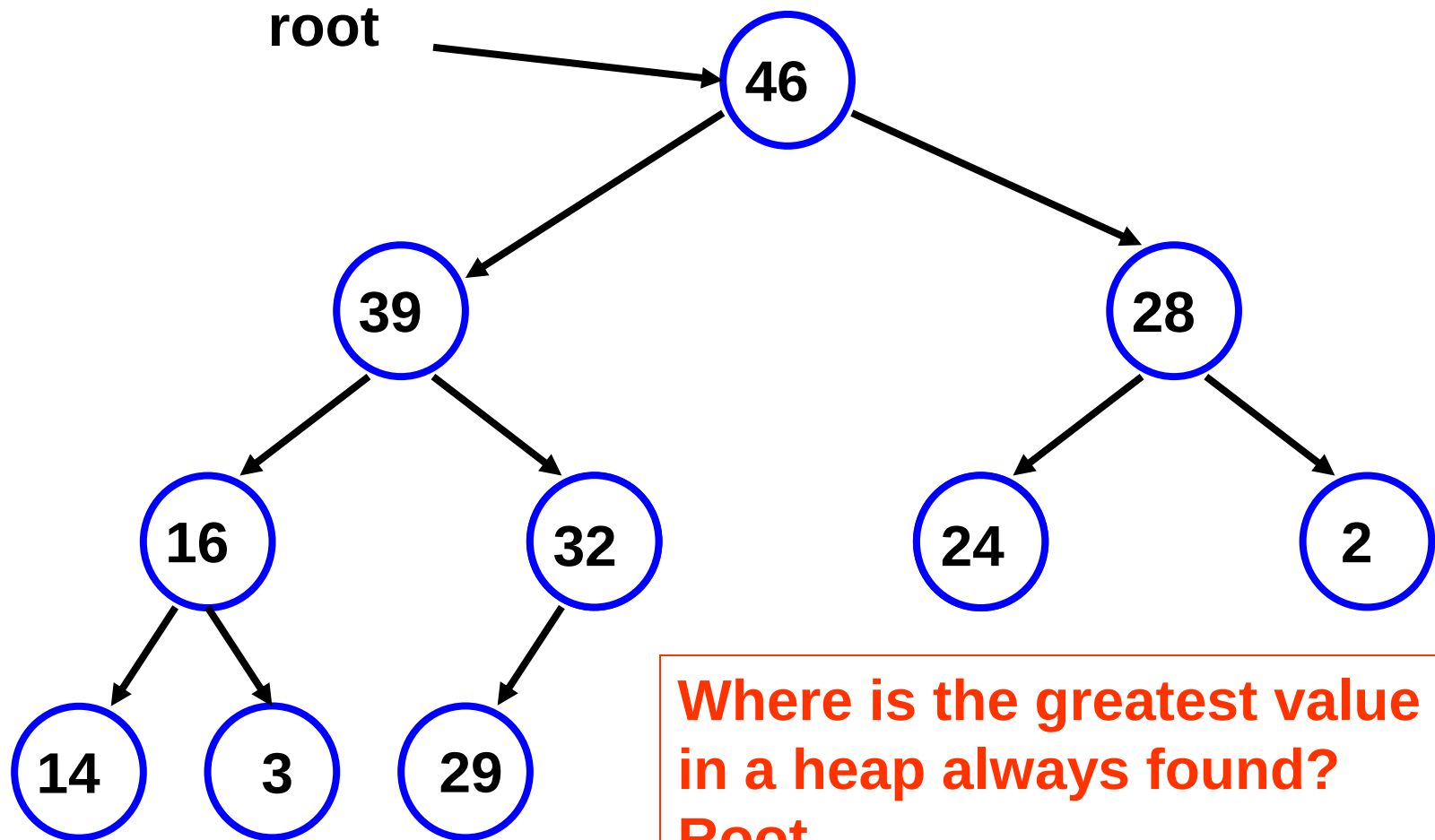
Heaps

- A **heap** is a complete binary tree in which the value of each node is greater than or equal to the values of its children (if any).
 - Parent $\geq$ Children
- Technically, this is called a **maxheap**.
- In a **minheap**, the value of each node is less than or equal to the values of its children.
- A maxheap can be easily modified to make a minheap.
- In our discussion, the word “heap” will refer to a maxheap.

Example of a Heap



Example of a Heap (cont.)

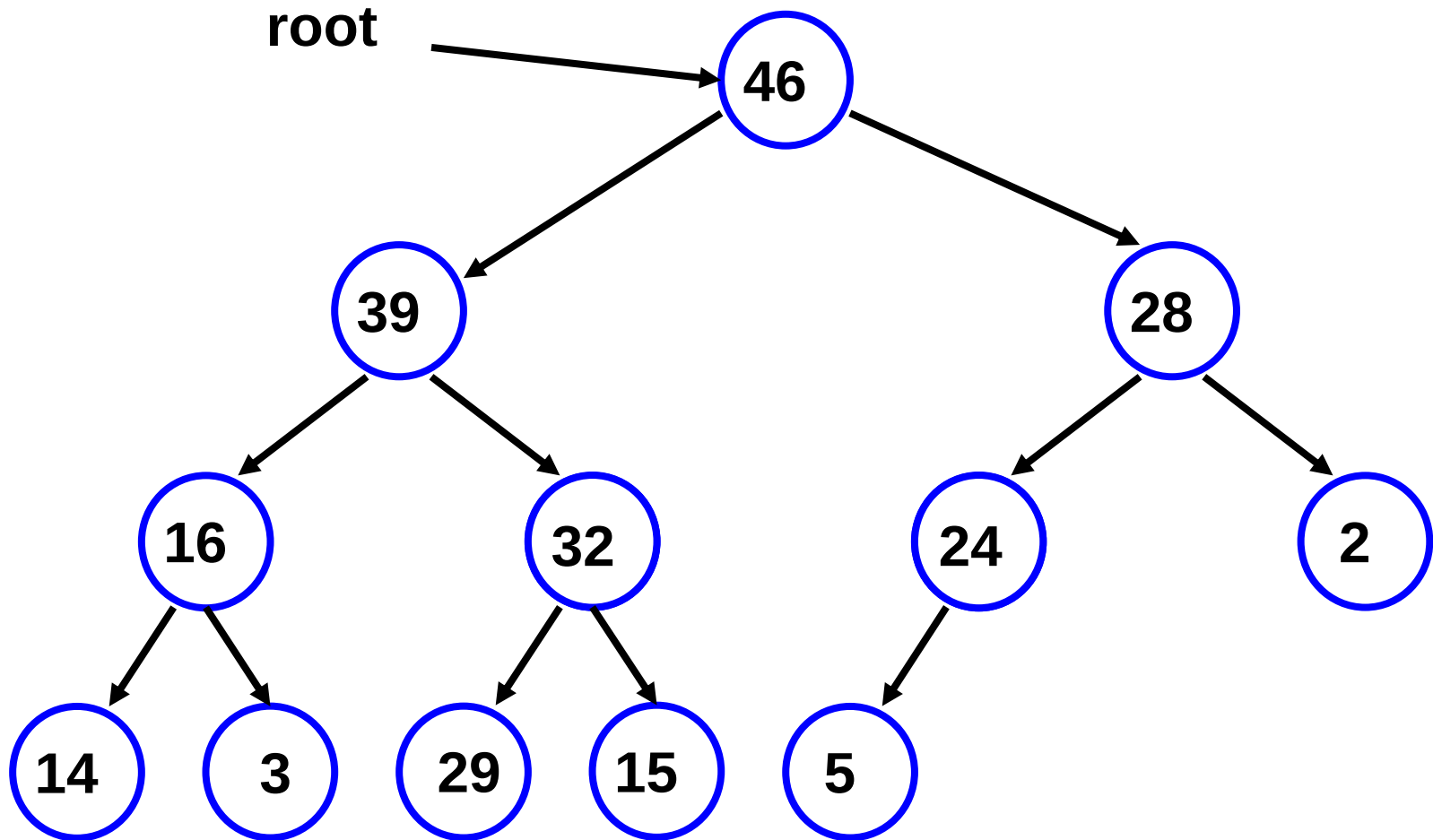


**Where is the greatest value
in a heap always found?
Root**

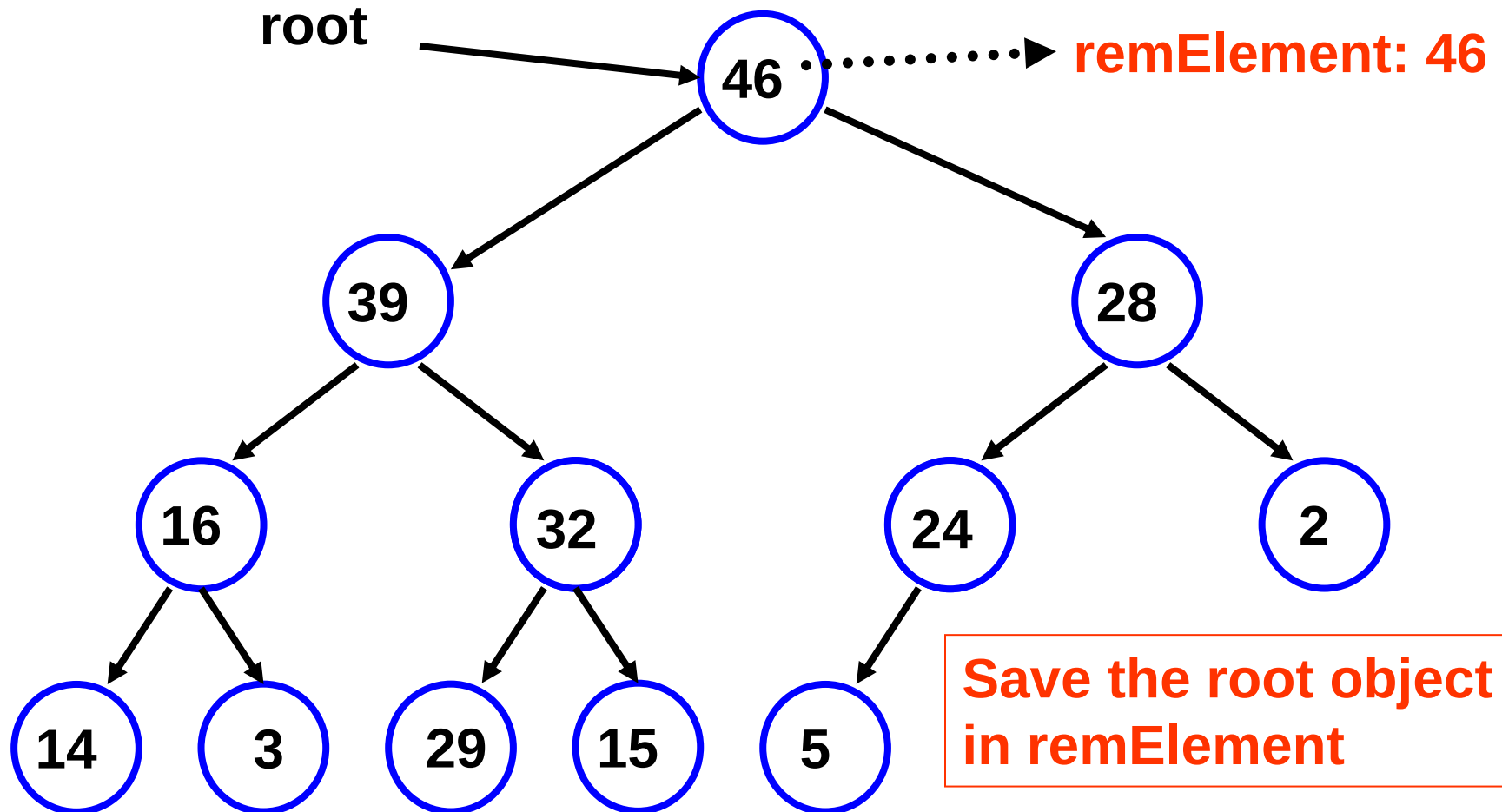
Dequeue

- Dequeueing the object with the greatest value appears to be a $\Theta(1)$ operation.
- However, after removing the object, we must turn the resultant structure into a heap again, for the next dequeuer.
- Fortunately, it only takes $O(\lg n)$ time to turn the structure back into a heap again.
- Hence, the overall dequeue in a heap is a $O(\lg n)$ operation.

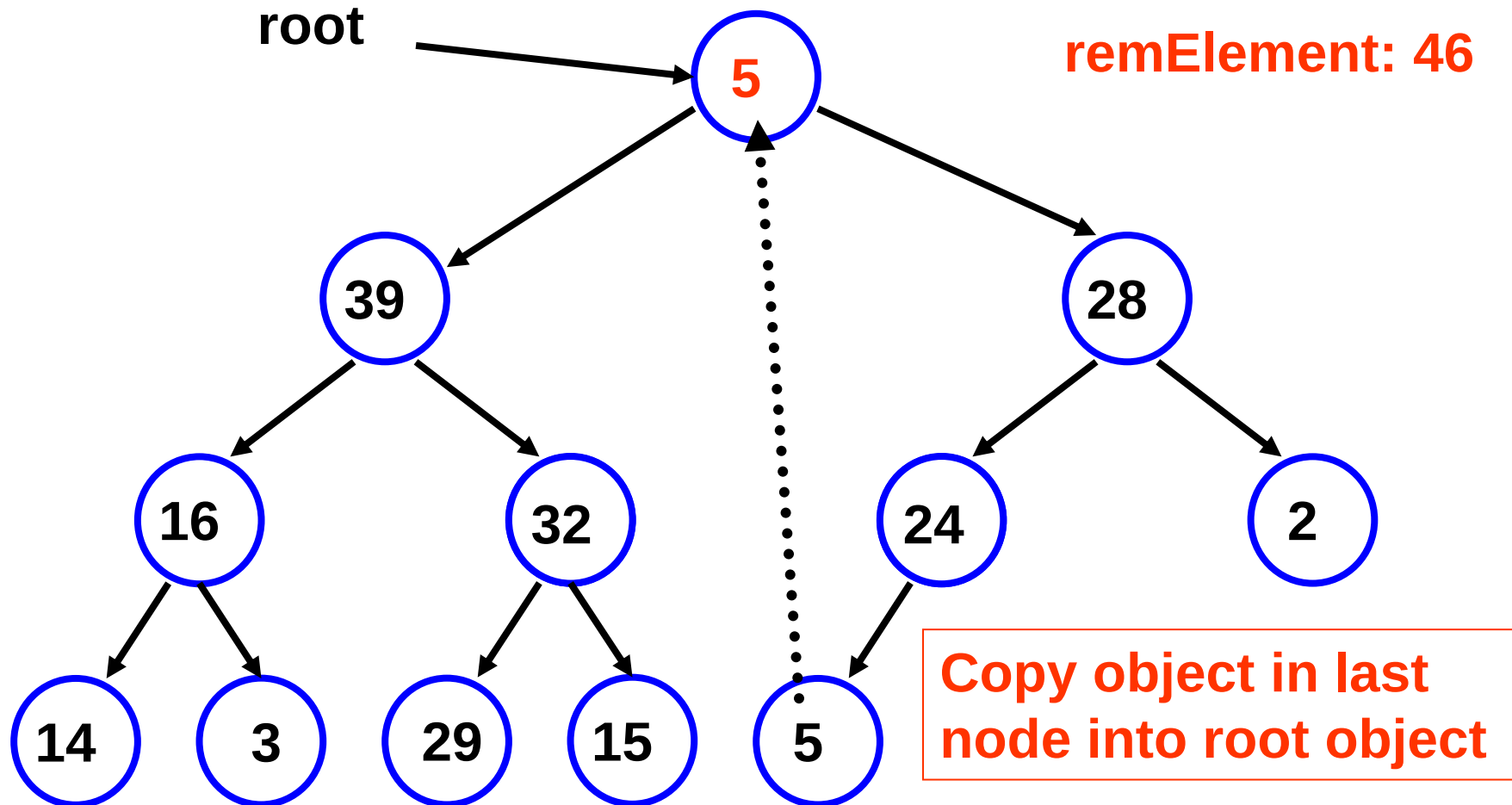
Deque (cont.)



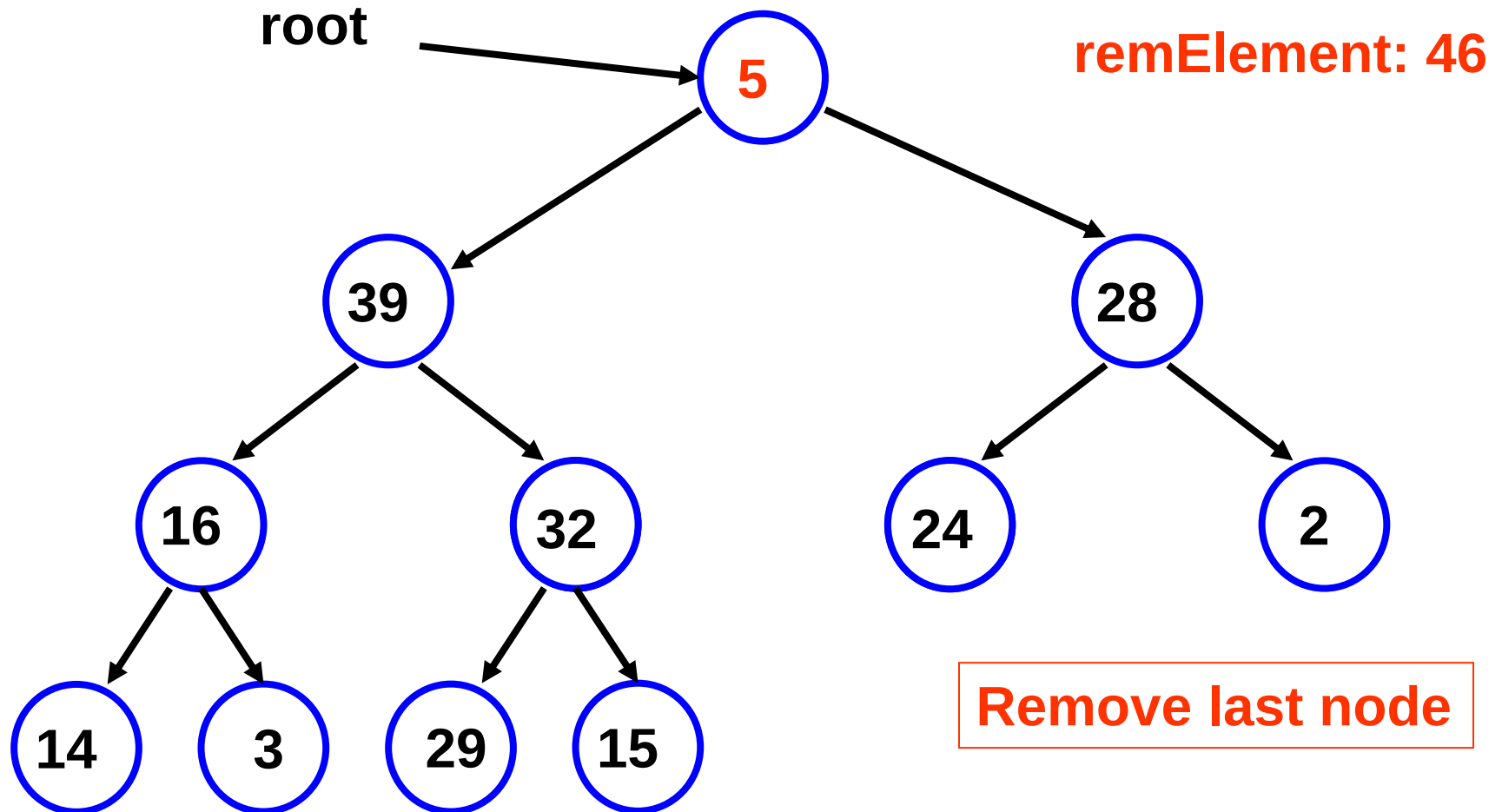
Deque (cont.)



Deque (cont.)



Deque (cont.)

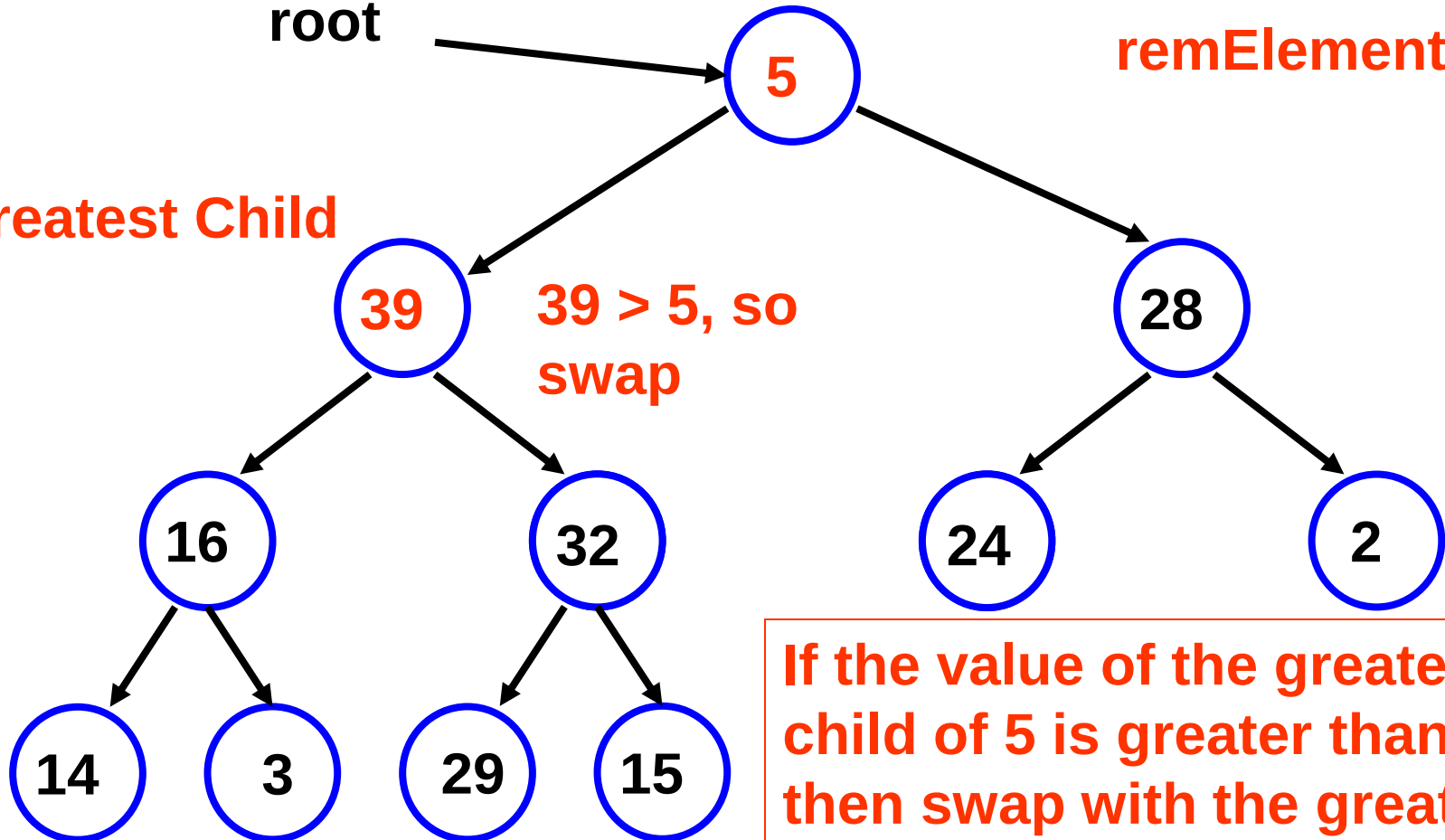


Deque (cont.)

root

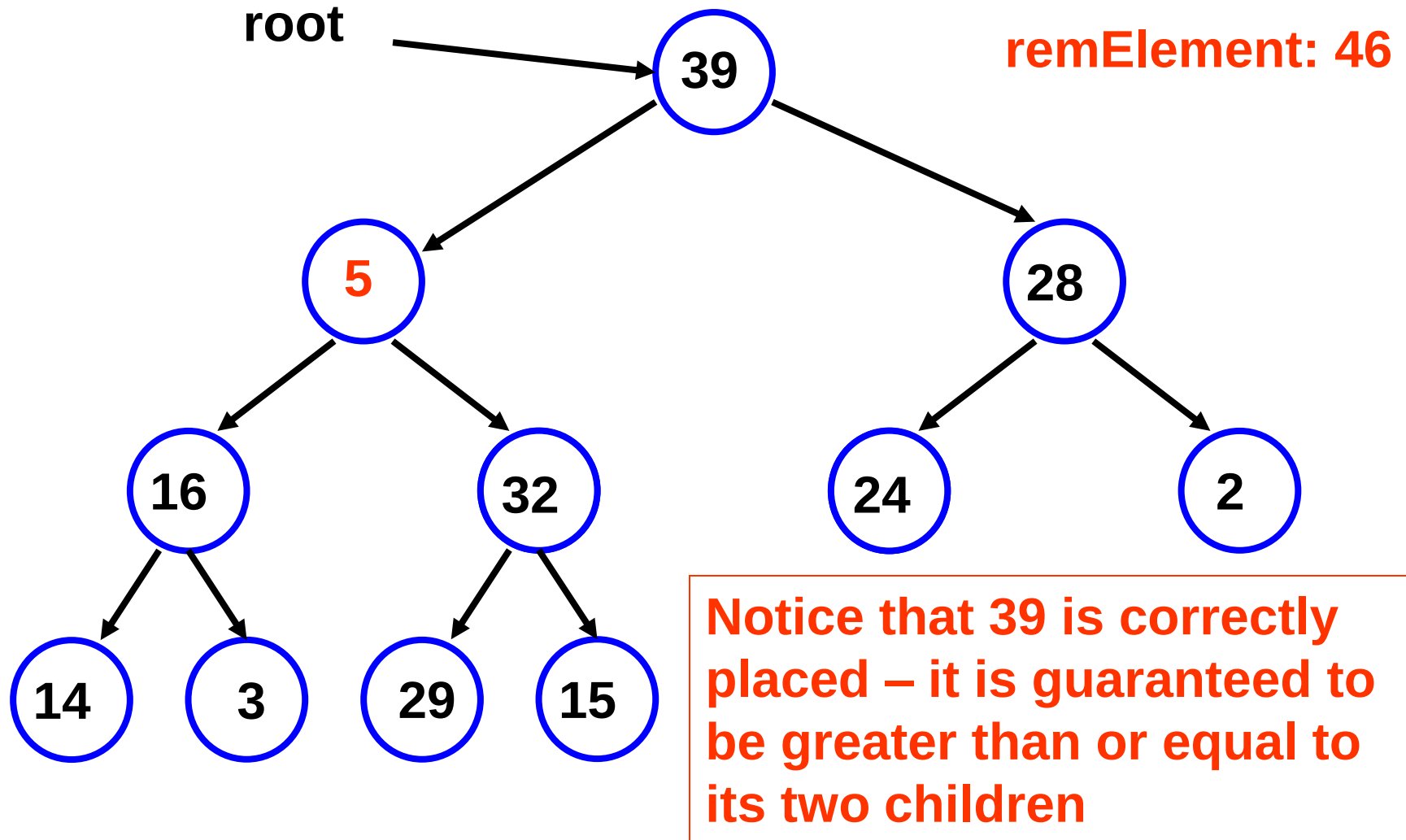
remElement: 46

Greatest Child

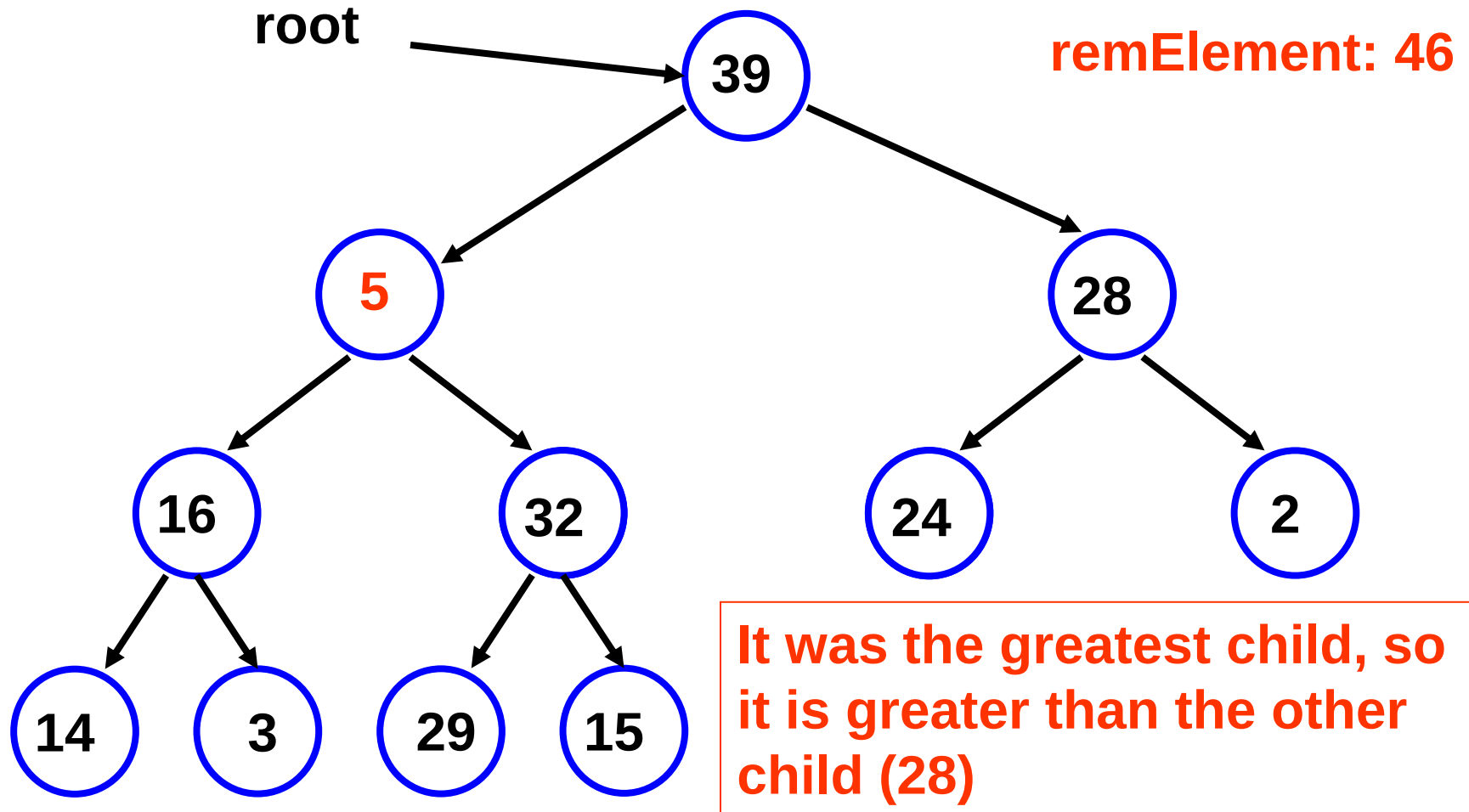


If the value of the greatest child of 5 is greater than 5, then swap with the greatest child

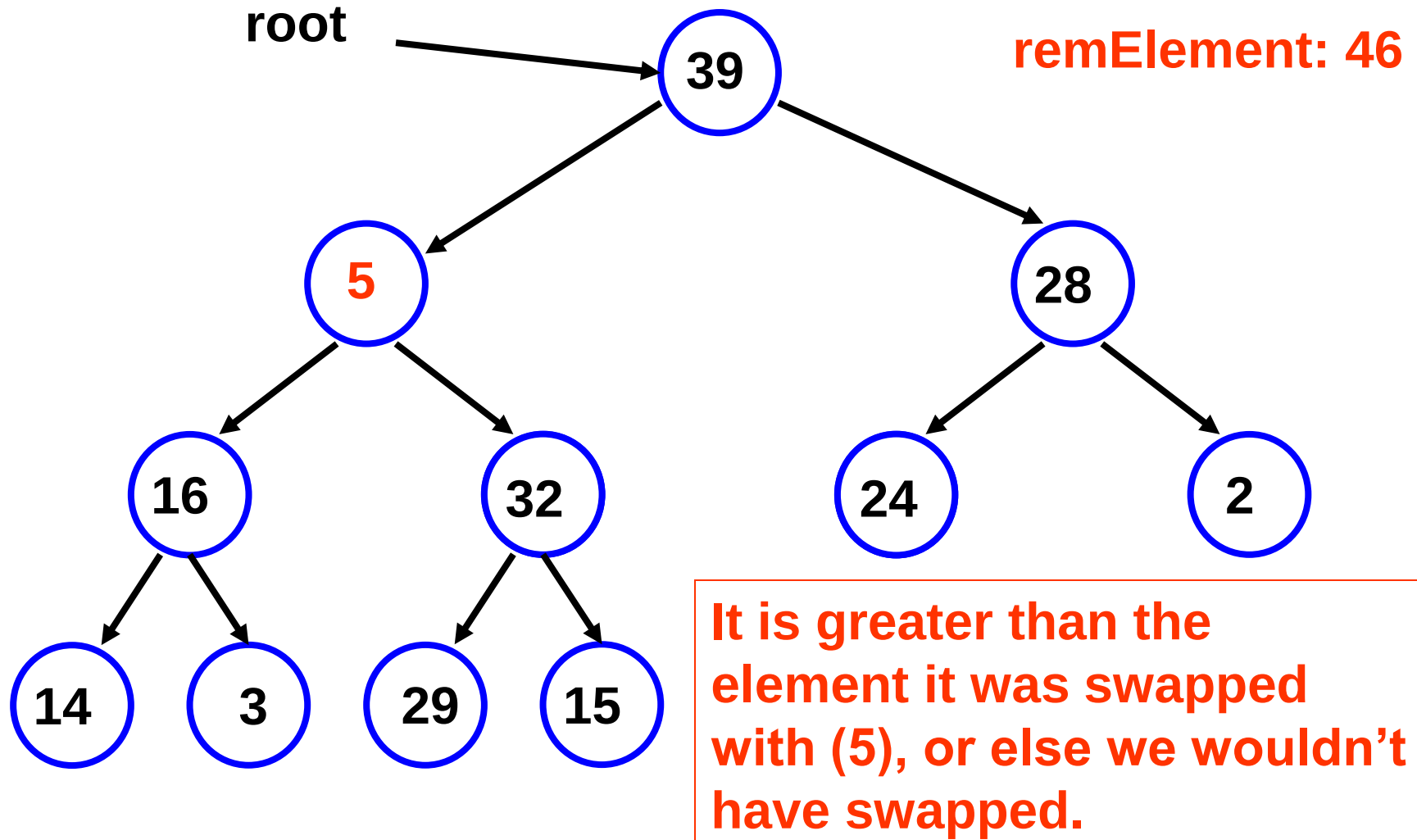
Deque (cont.)



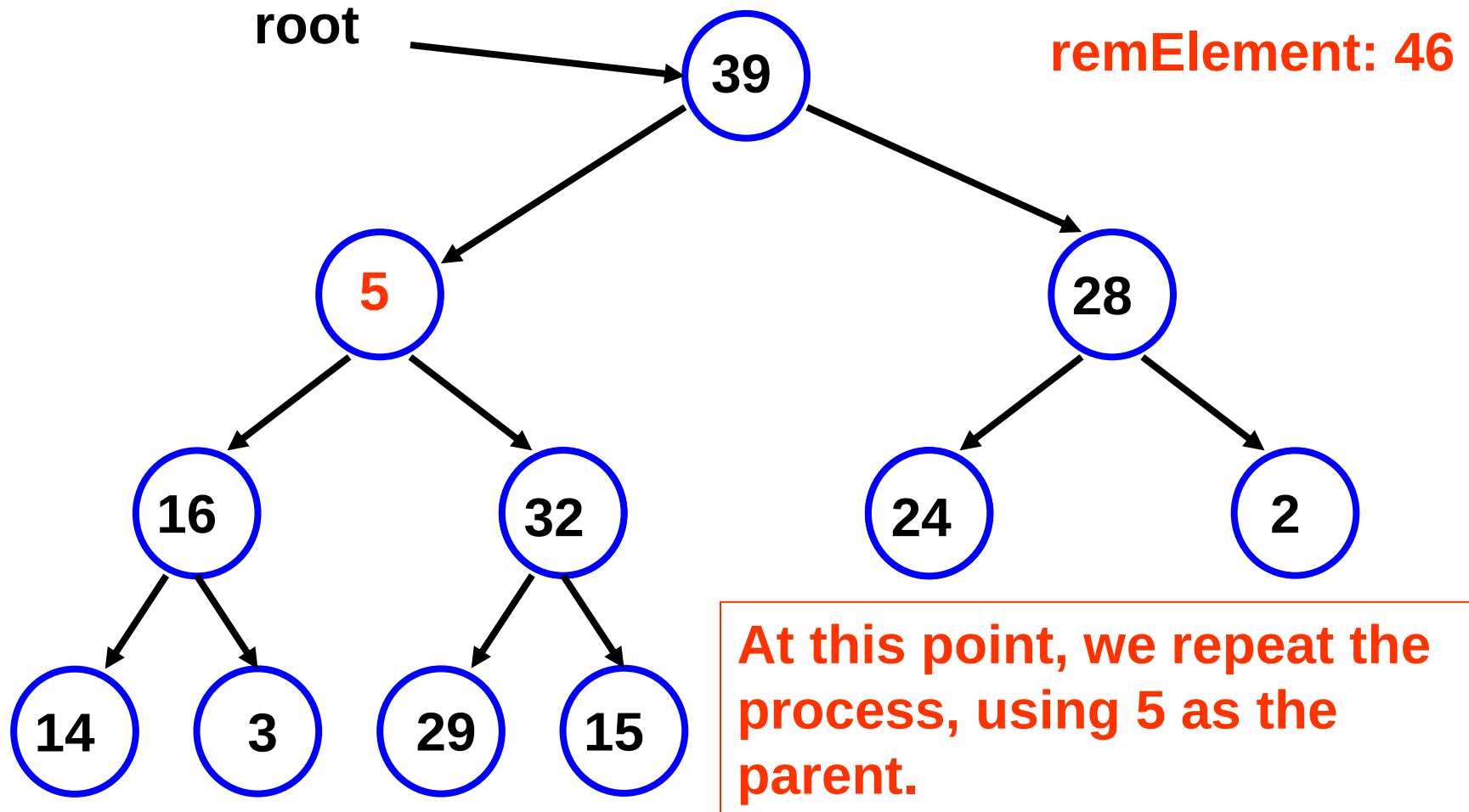
Deque (cont.)



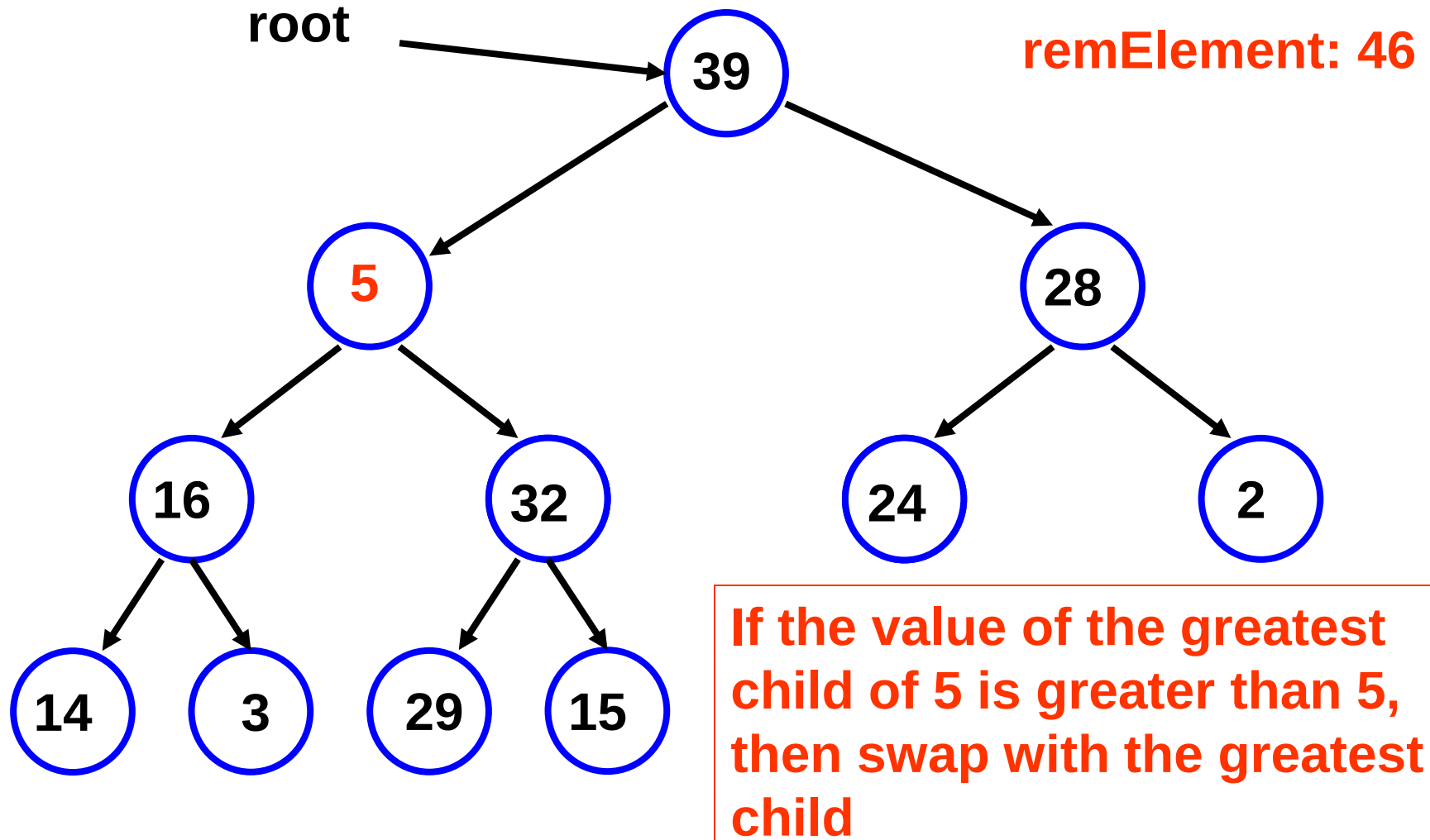
Deque (cont.)



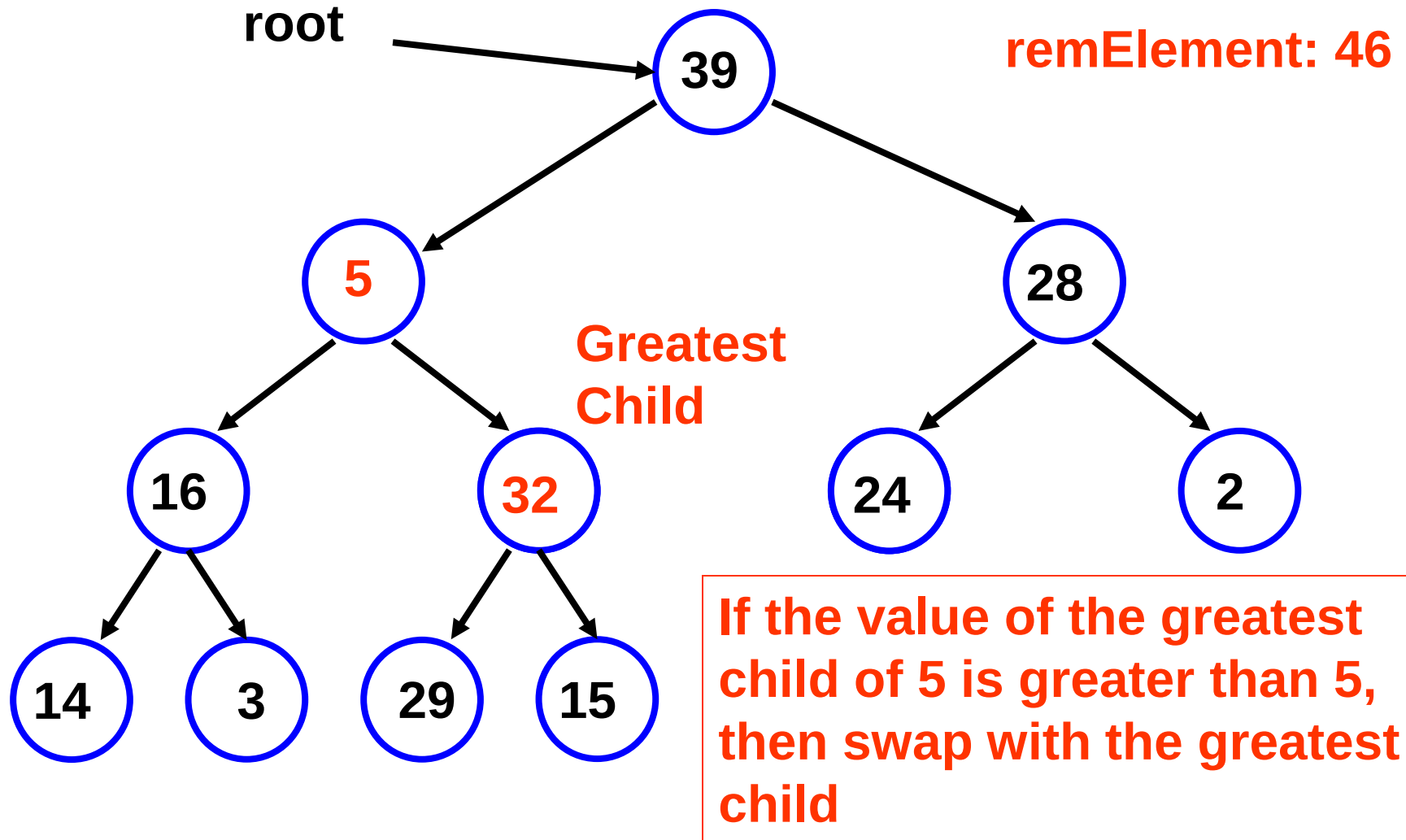
Deque (cont.)



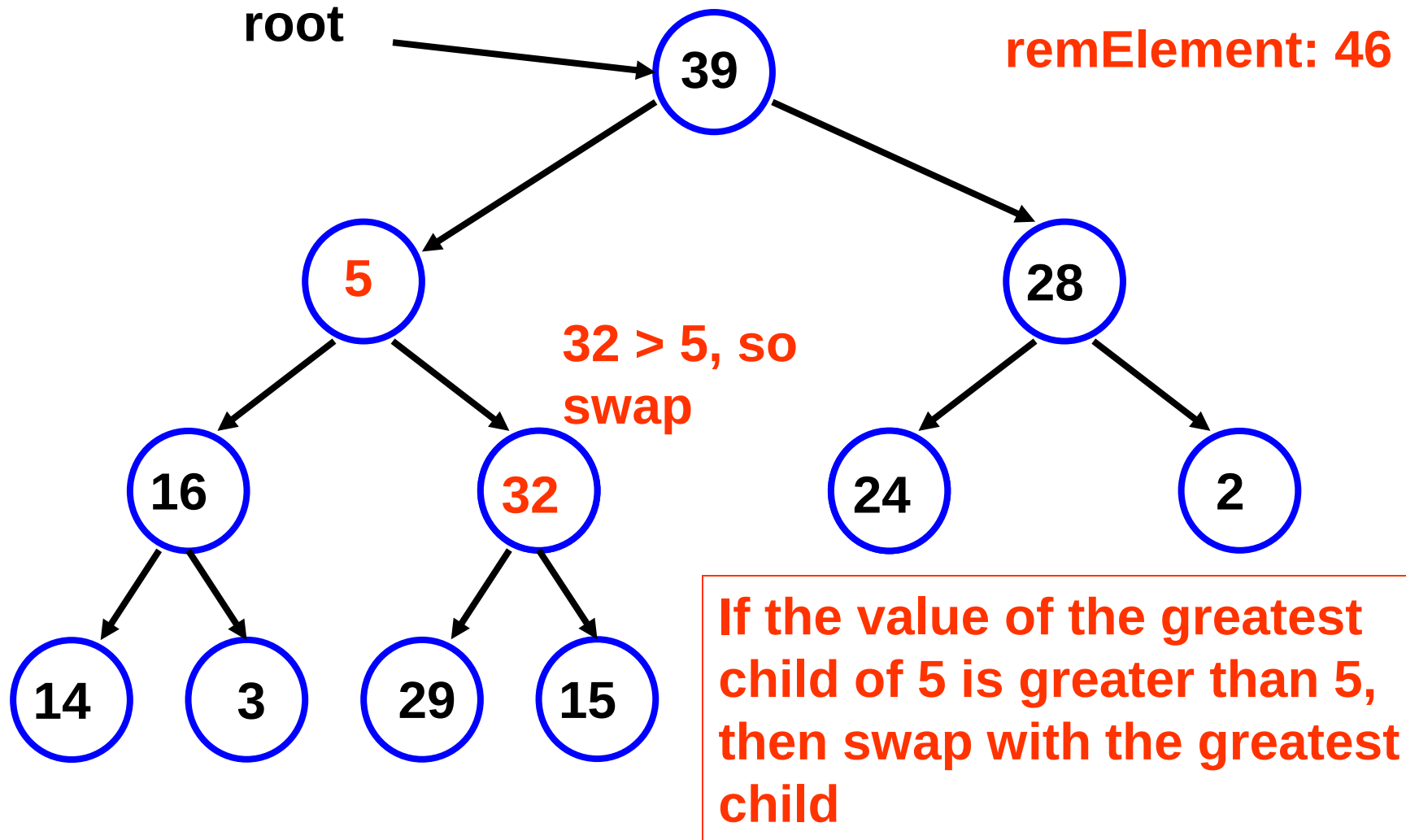
Deque (cont.)



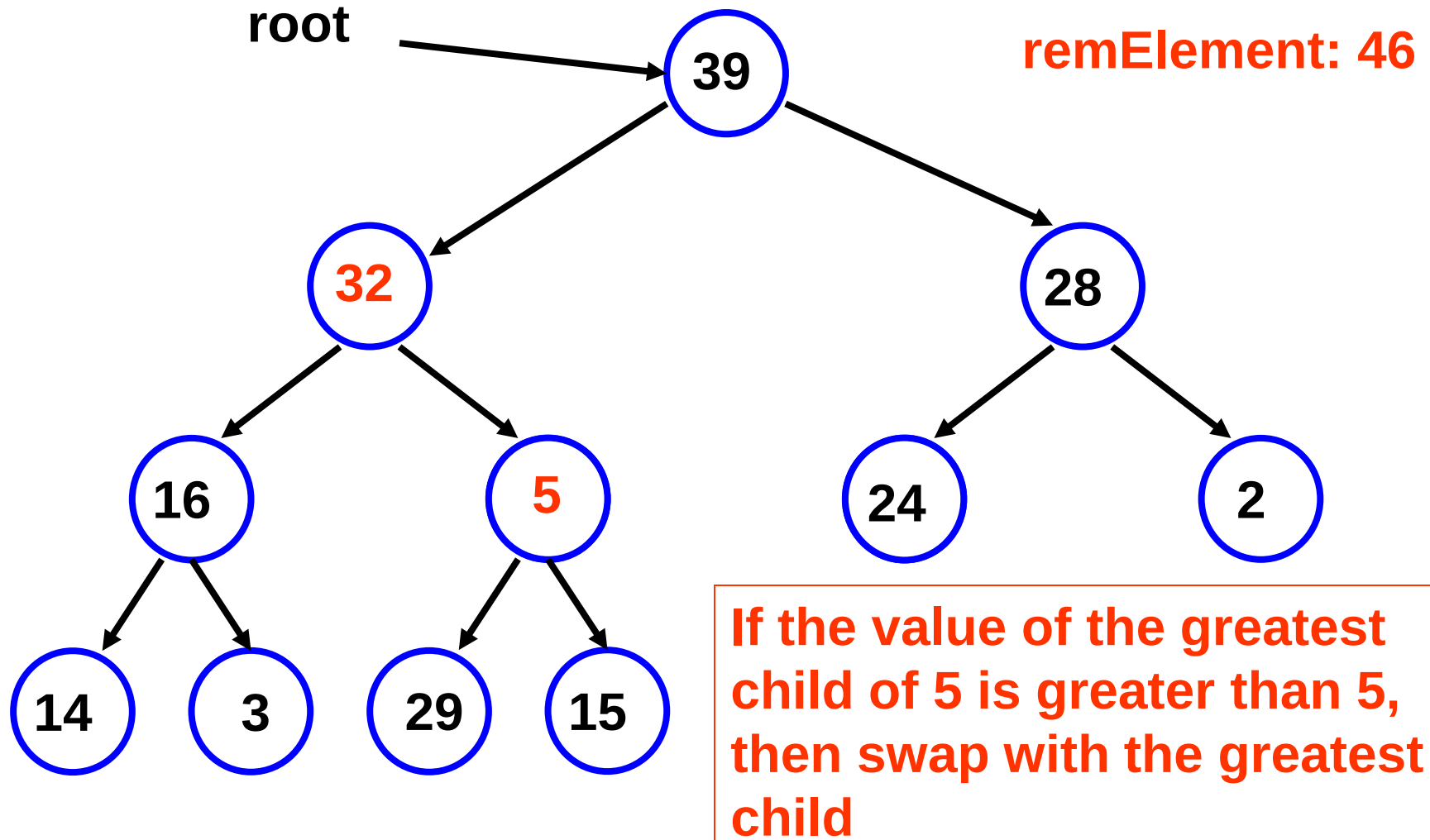
Deque (cont.)



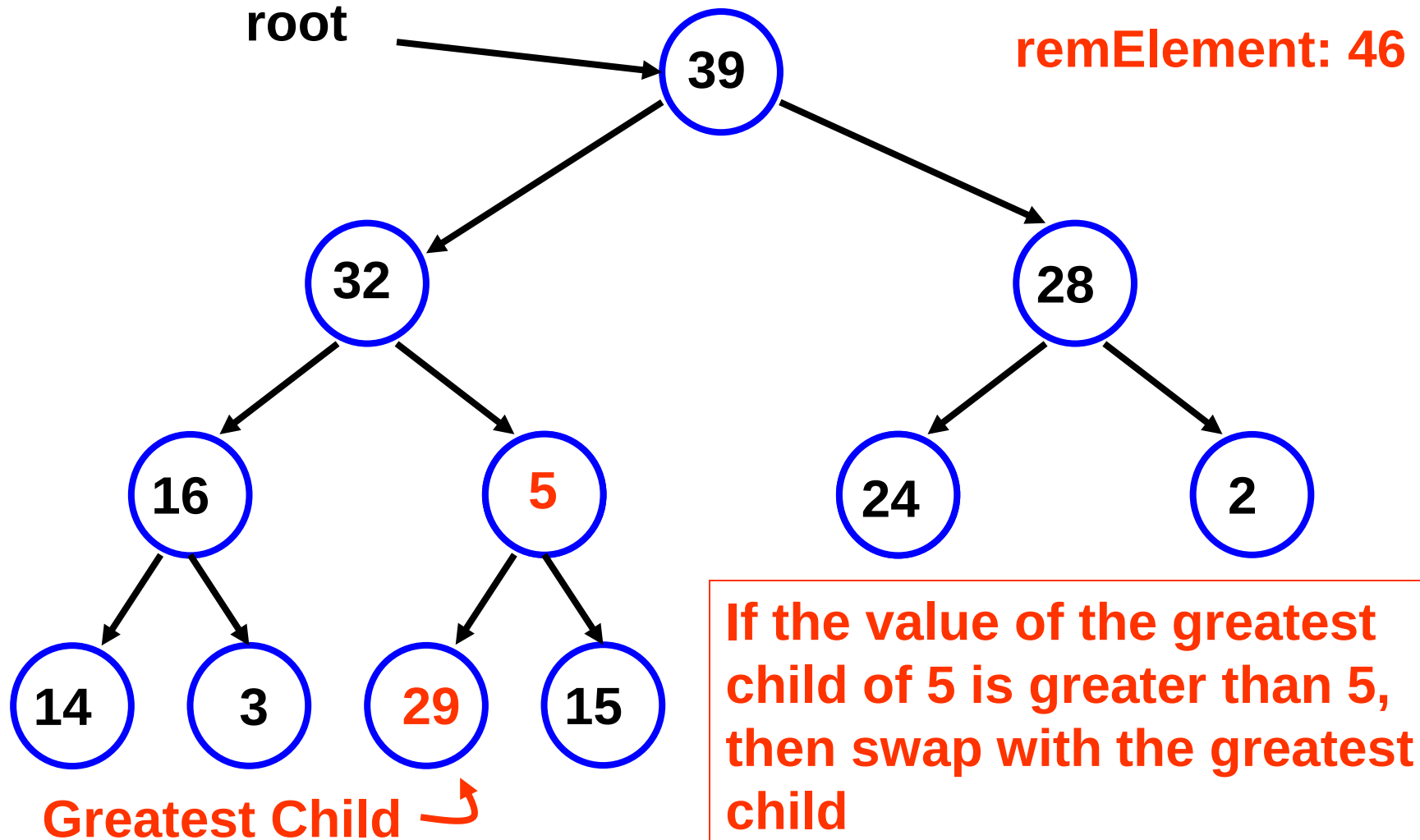
Deque (cont.)



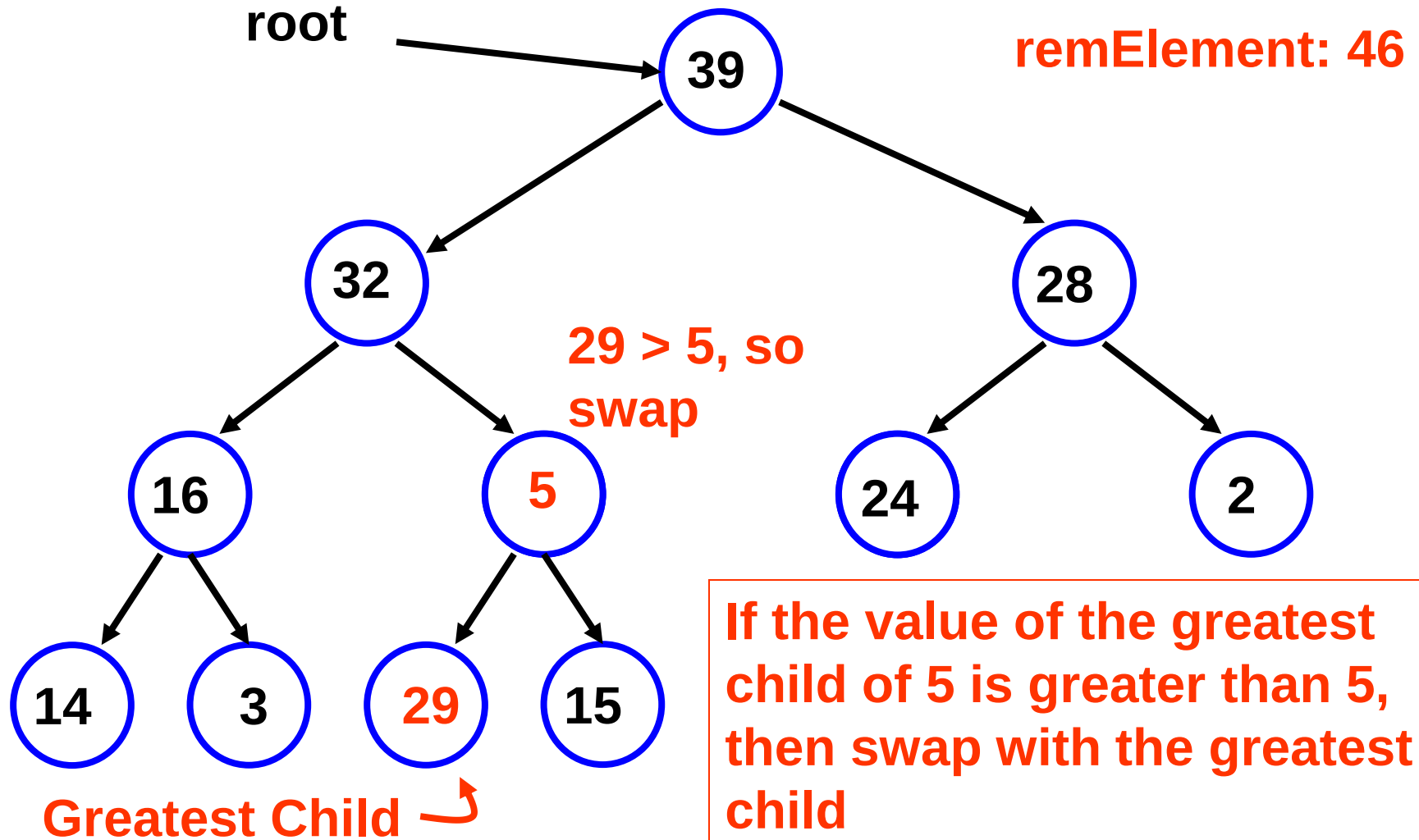
Deque (cont.)



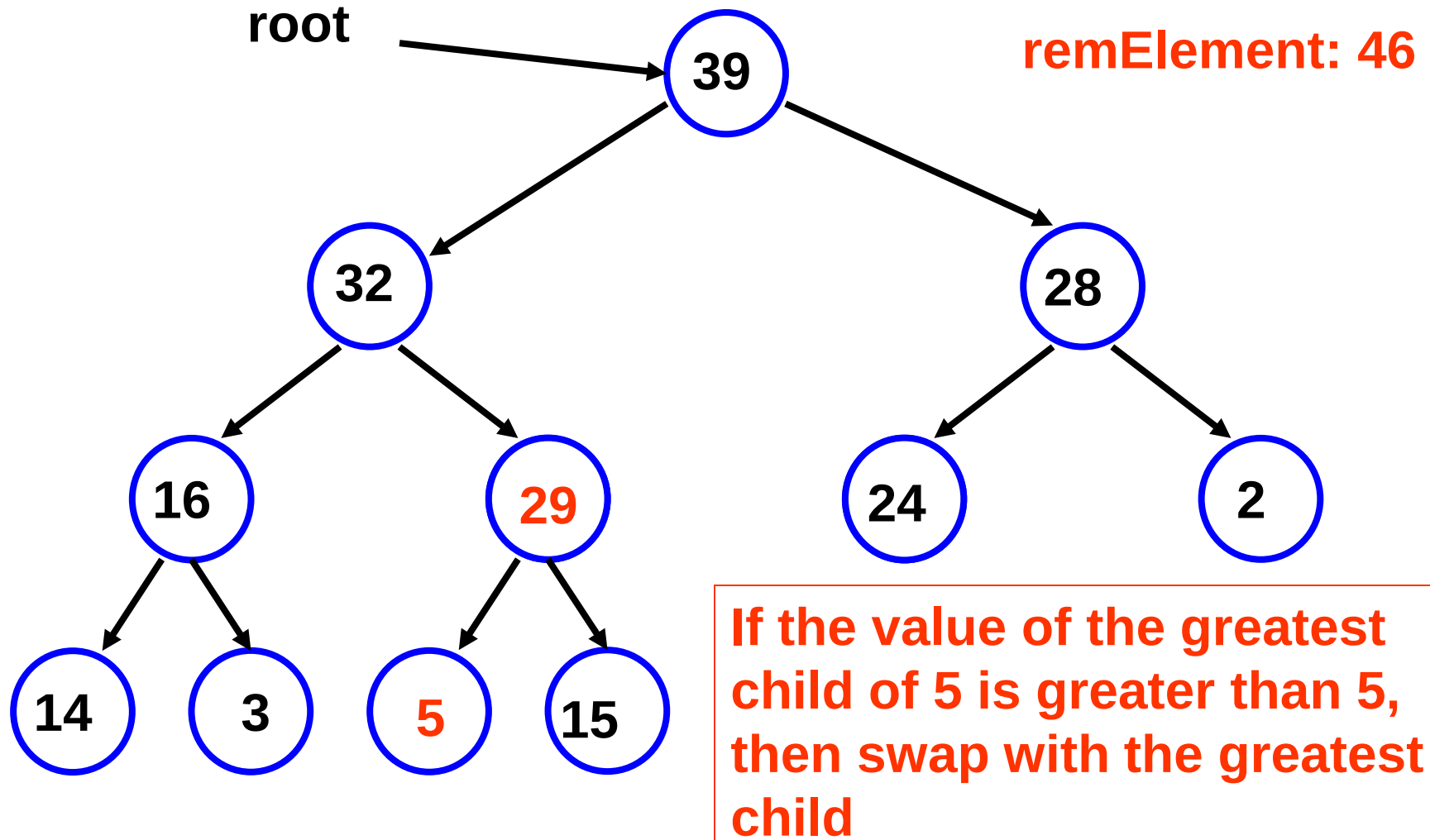
Deque (cont.)



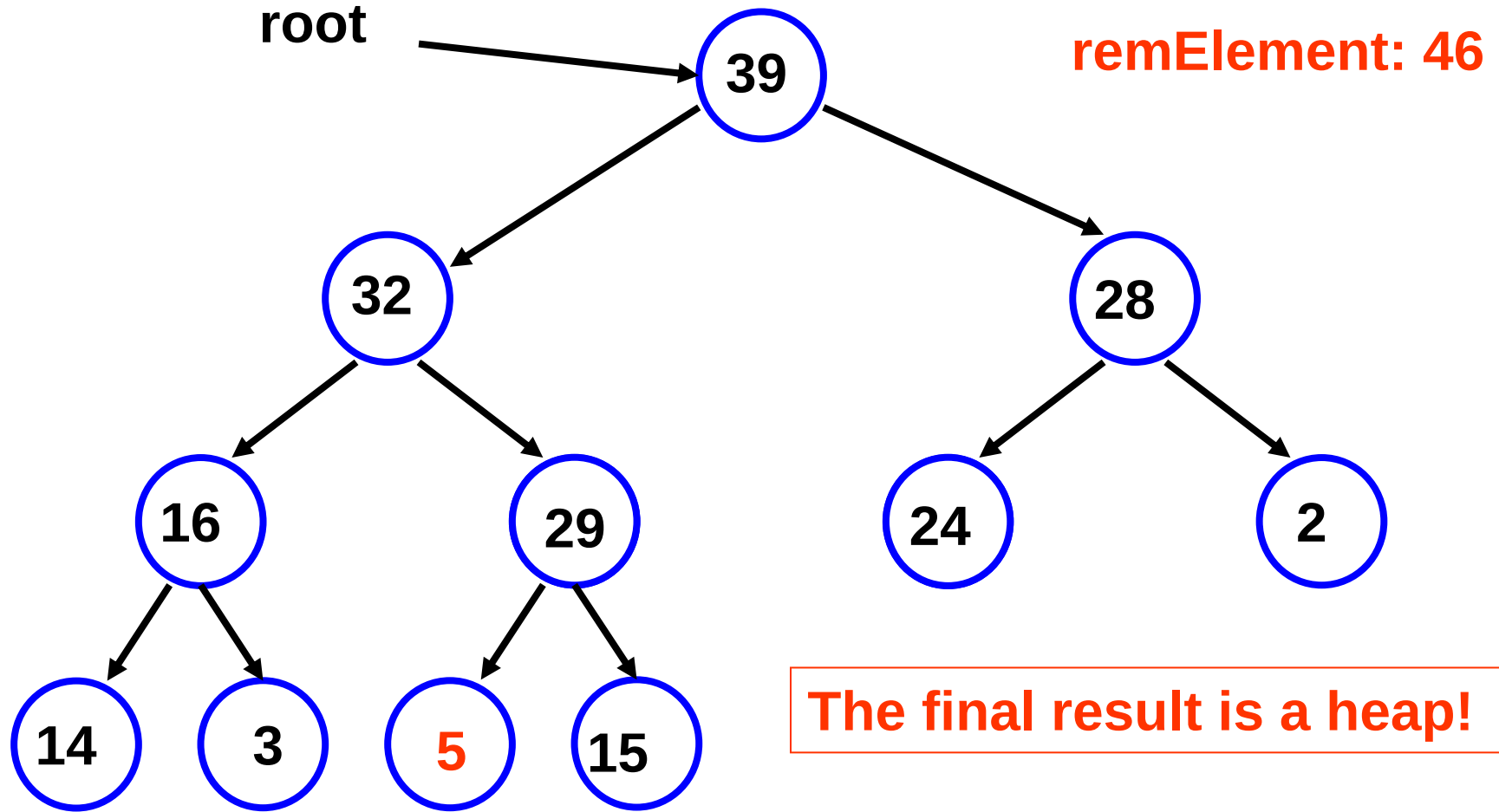
Deque (cont.)



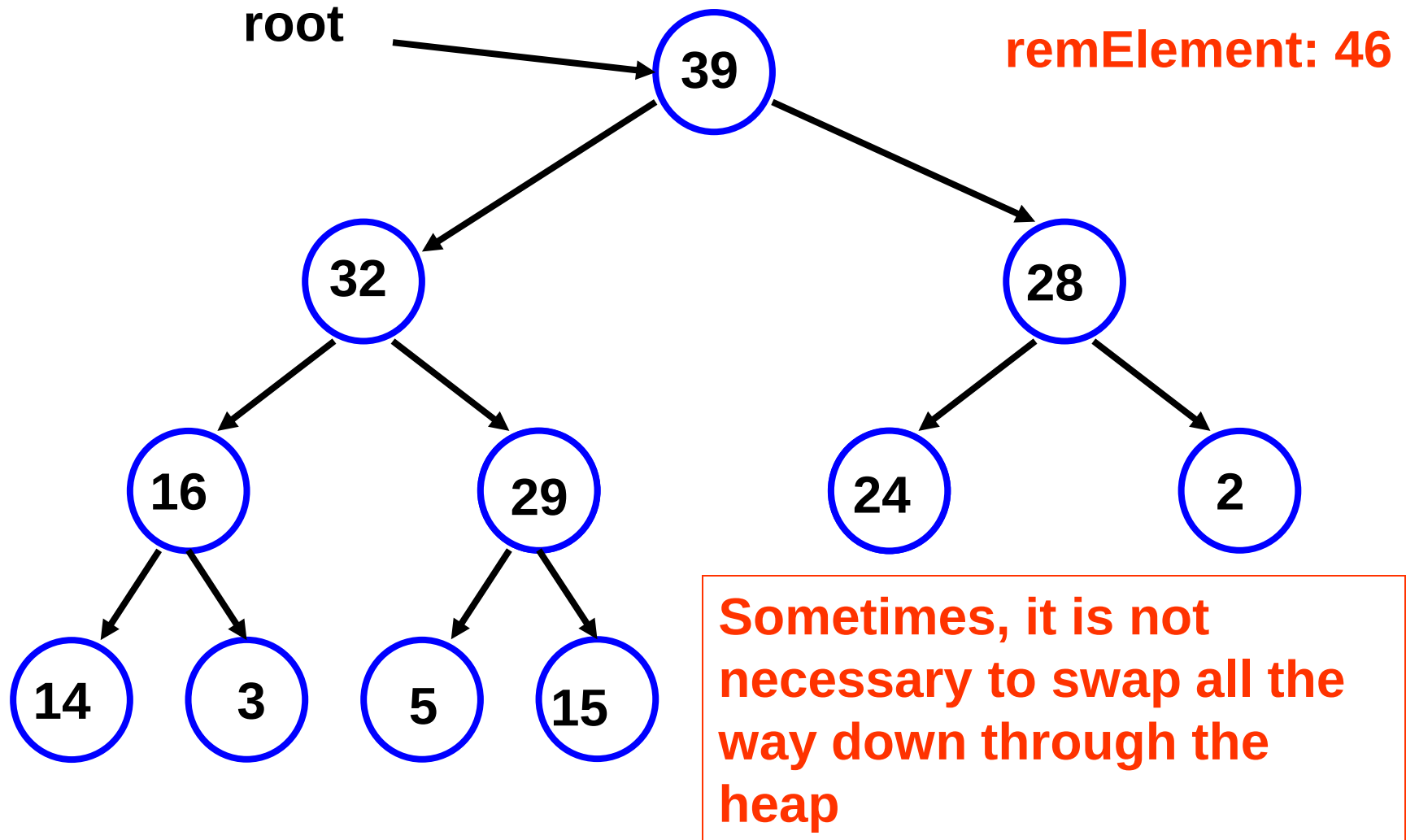
Deque (cont.)



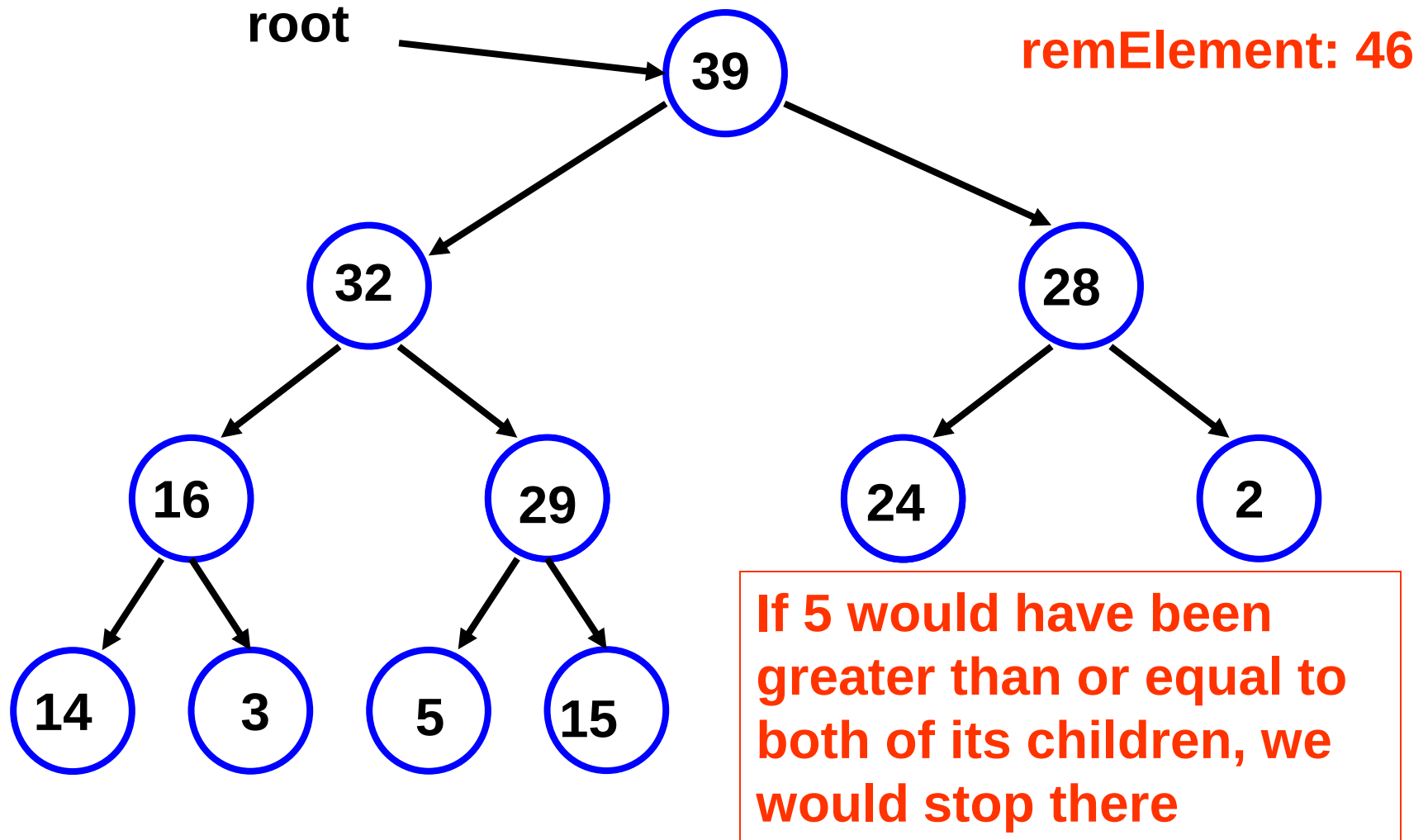
Deque (cont.)



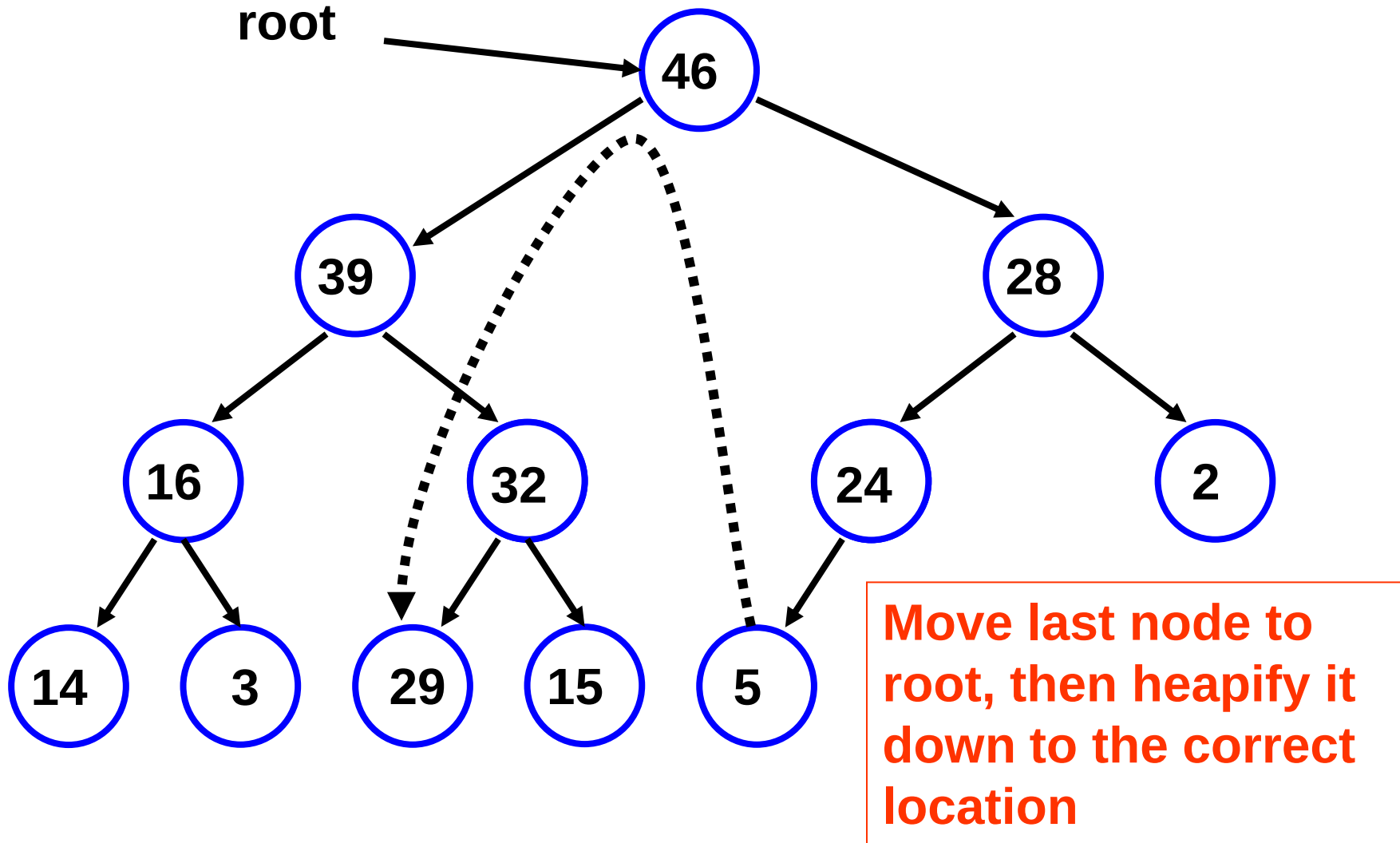
Deque (cont.)



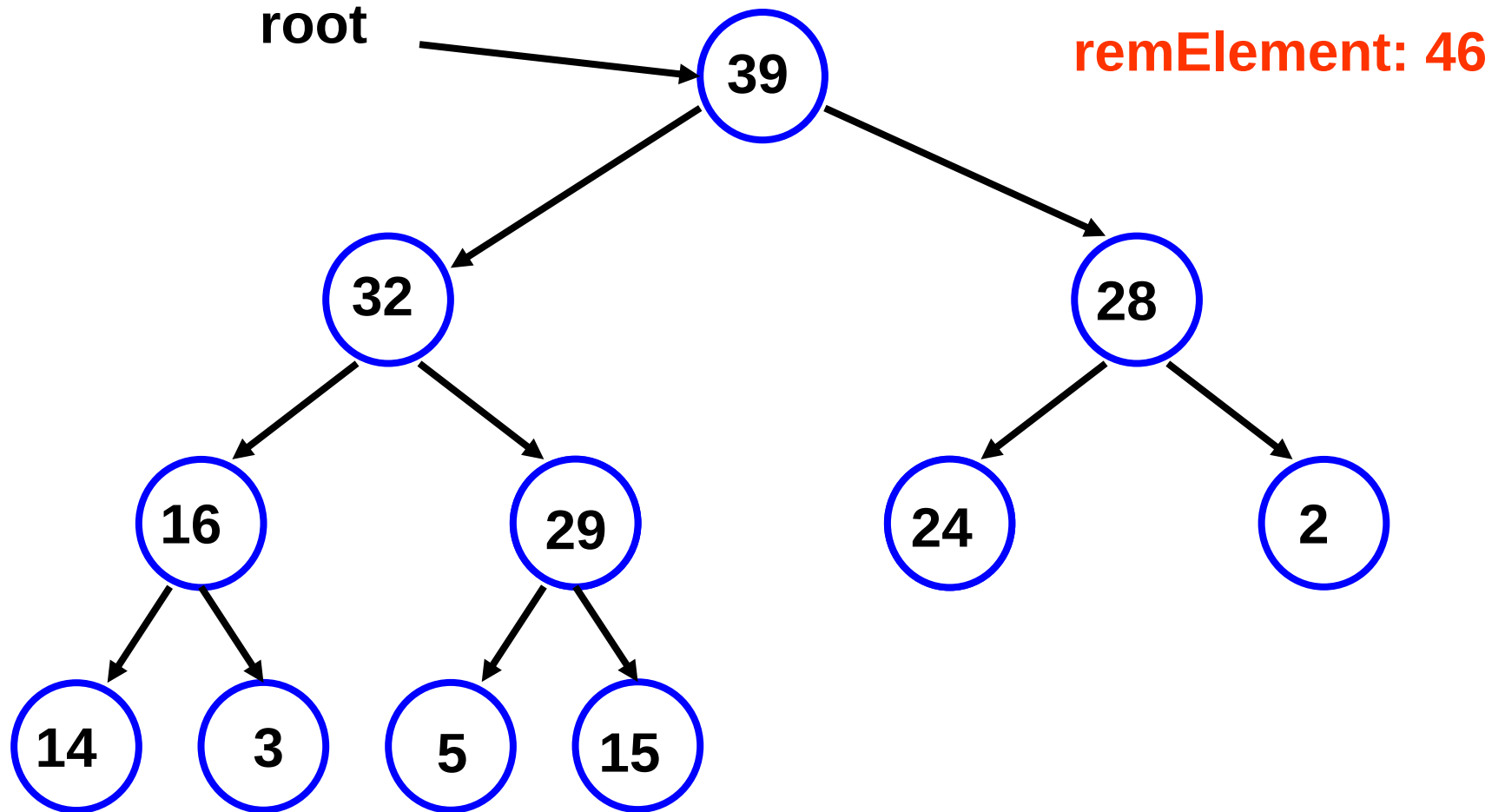
Deque (cont.)



Dequeue In Short



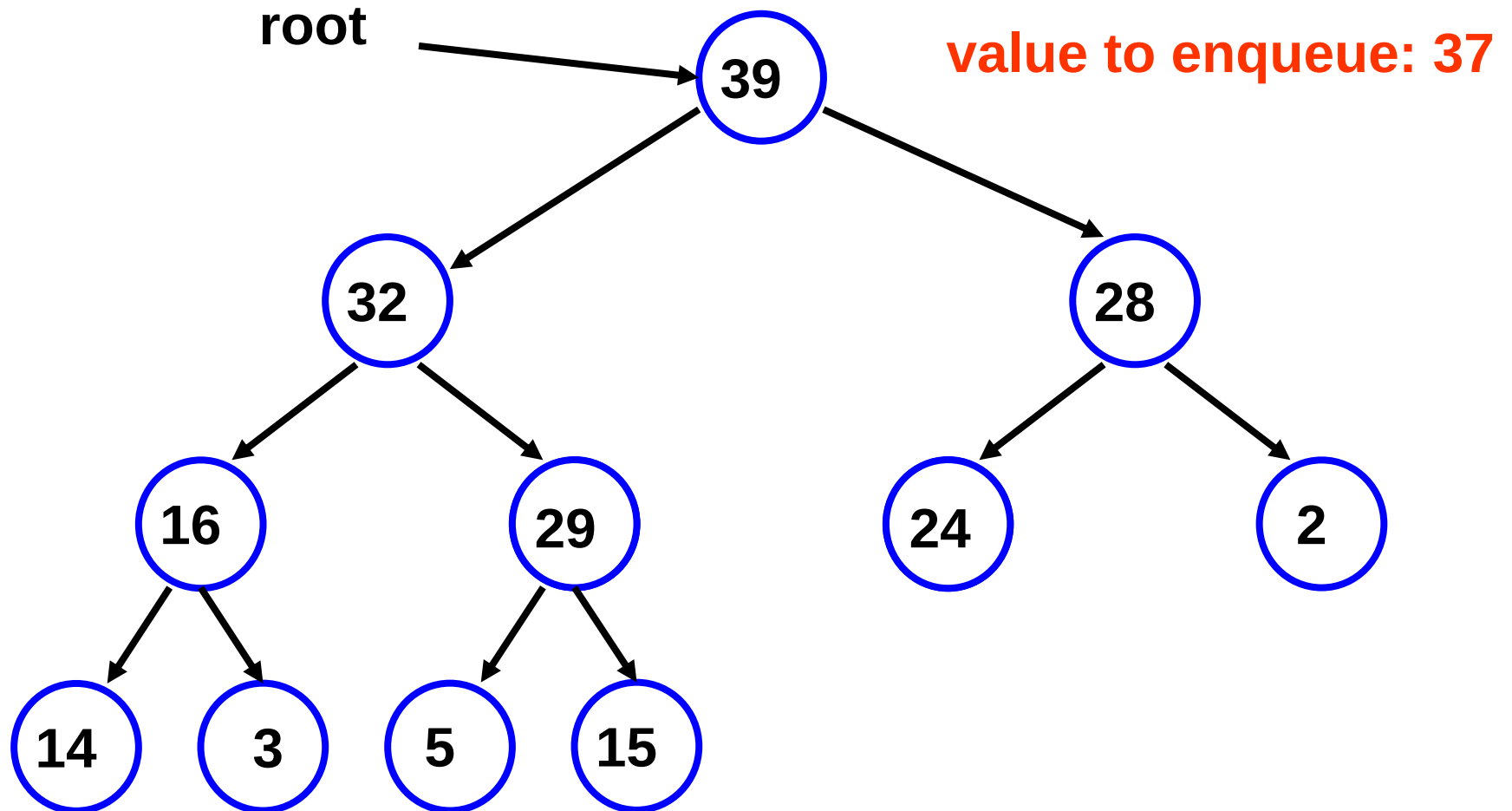
Deque In Short (cont.)



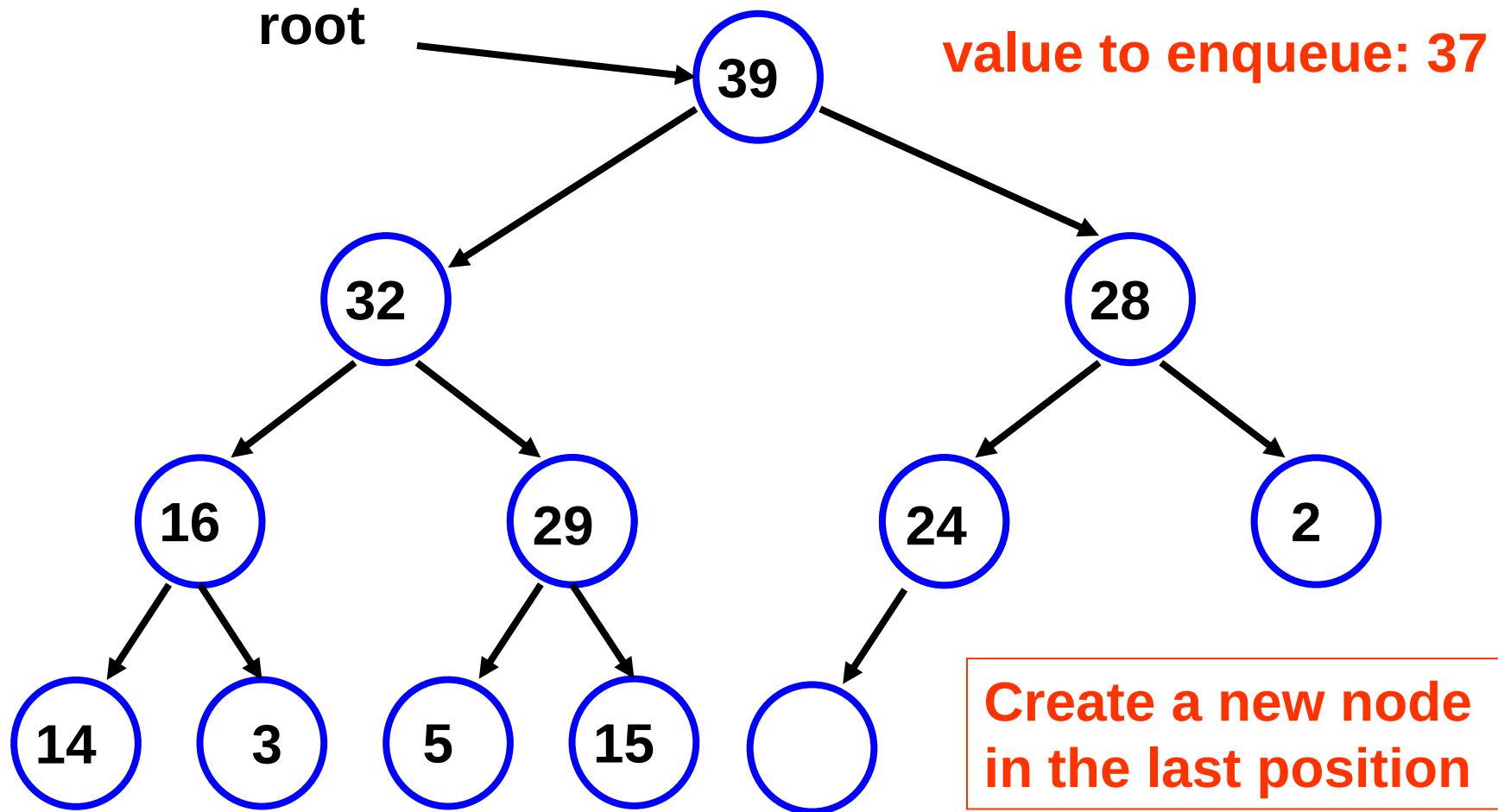
Heapify

- The process of swapping downwards to form a new heap is called *heapifying*.
- When heapifying, it is important that the rest of the structure is a heap, except for the root node that we are starting off with; otherwise, a new heap won't be formed.
- A loop is used for heapifying; the number of times through the loop is always $\lg n$ or less, which gives the $O(\lg n)$ complexity.
- Each time we swap downwards, the number of nodes we can travel to is reduced by approximately half.

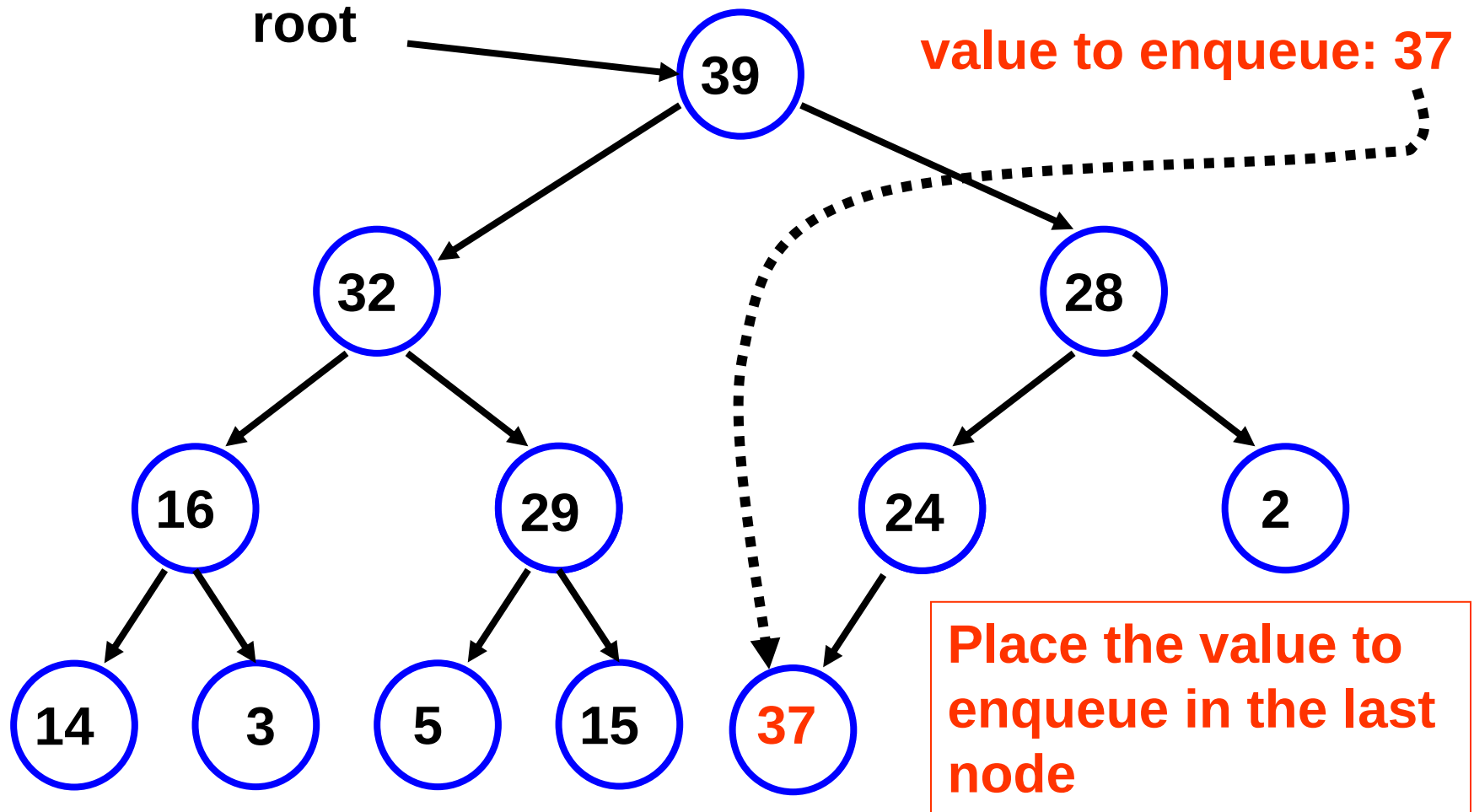
Enqueue



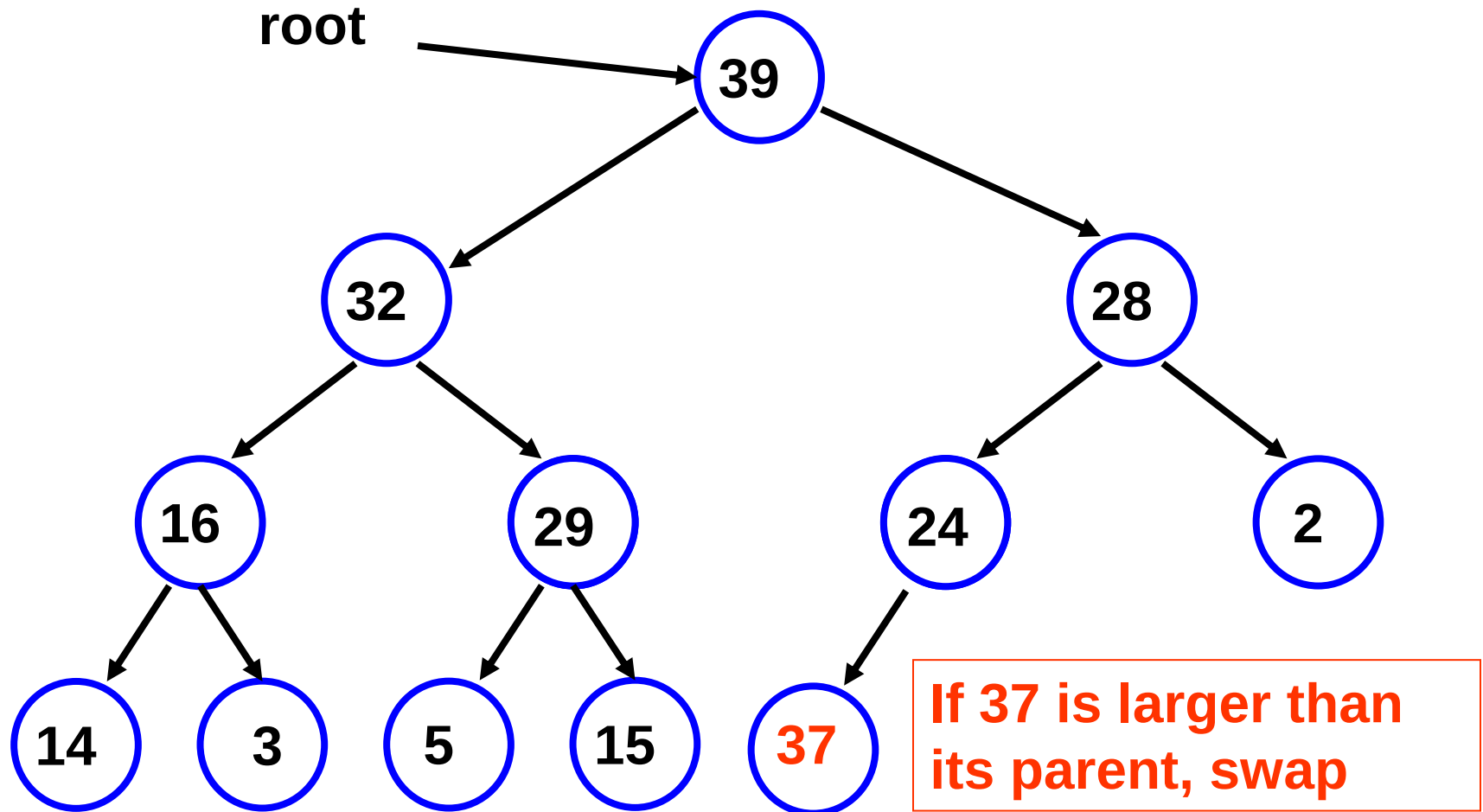
Enqueue (cont.)



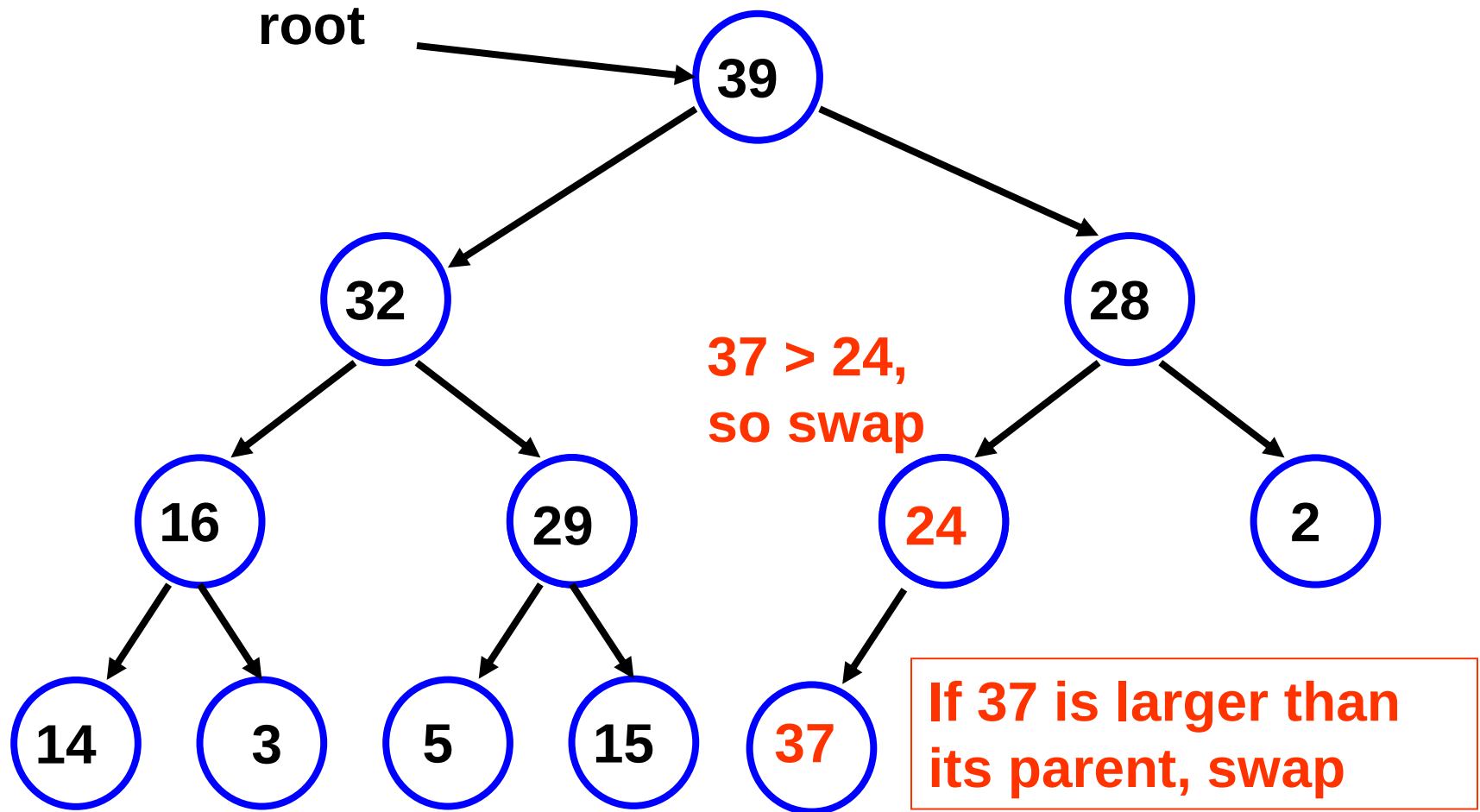
Enqueue (cont.)



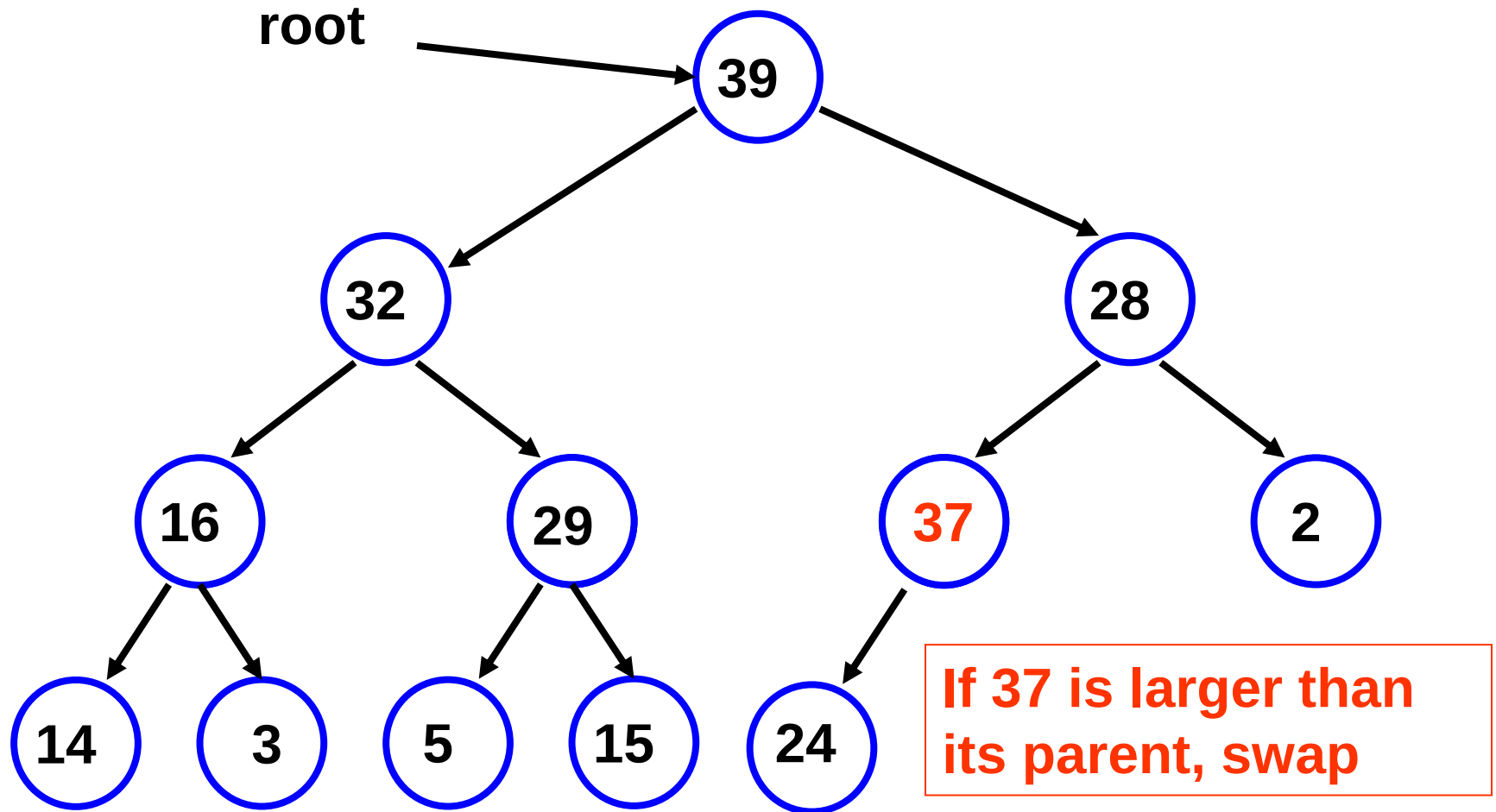
Enqueue (cont.)



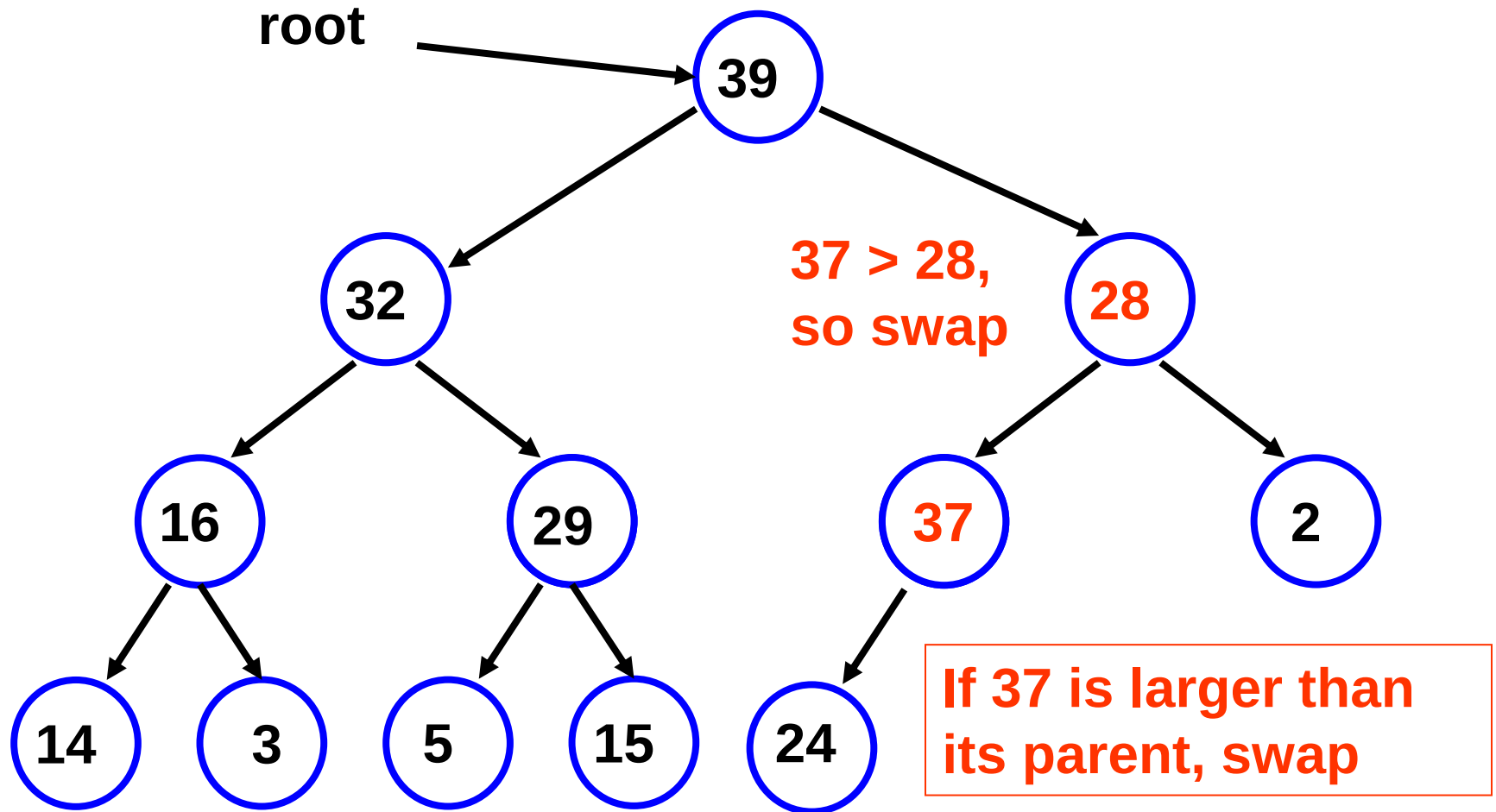
Enqueue (cont.)



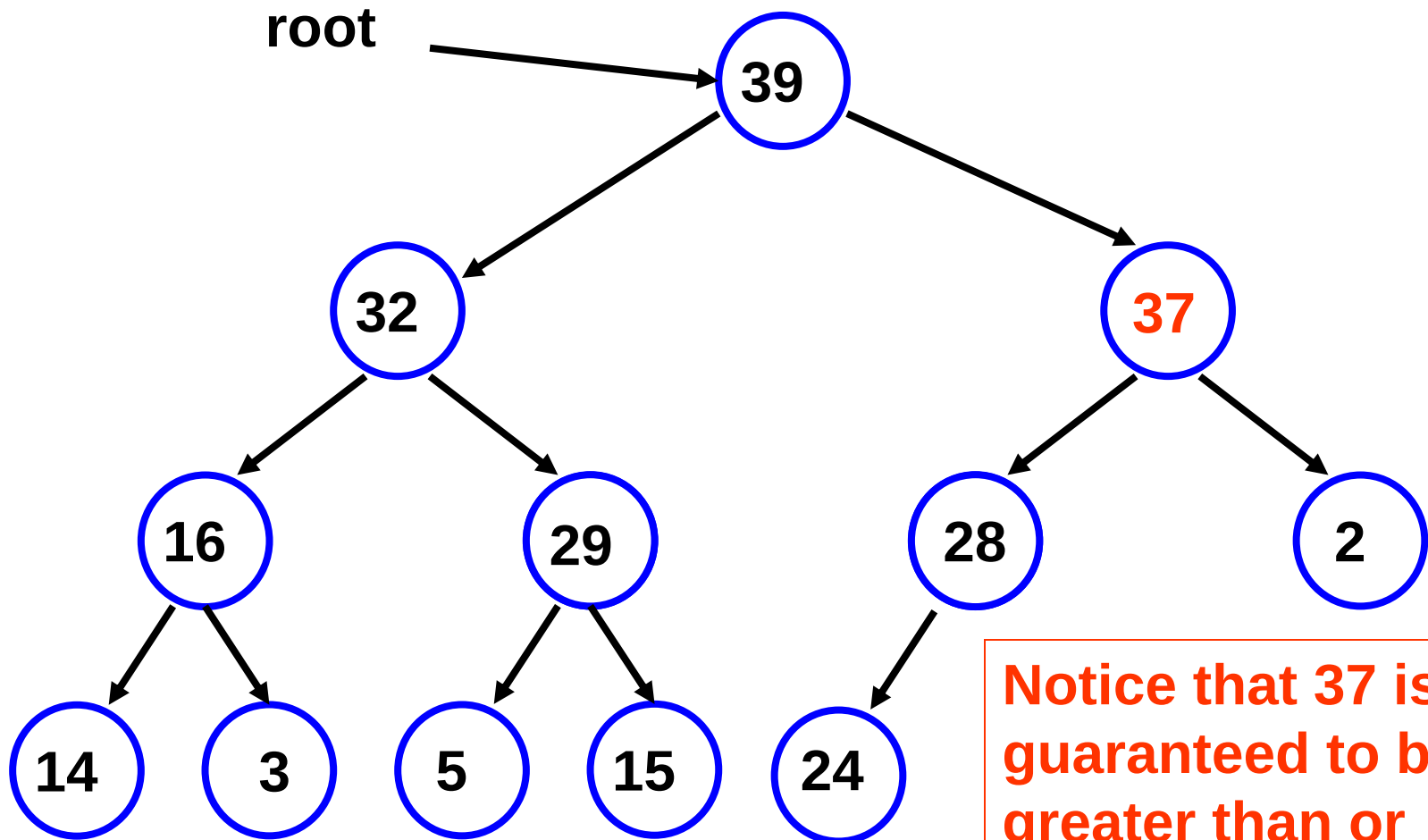
Enqueue (cont.)



Enqueue (cont.)

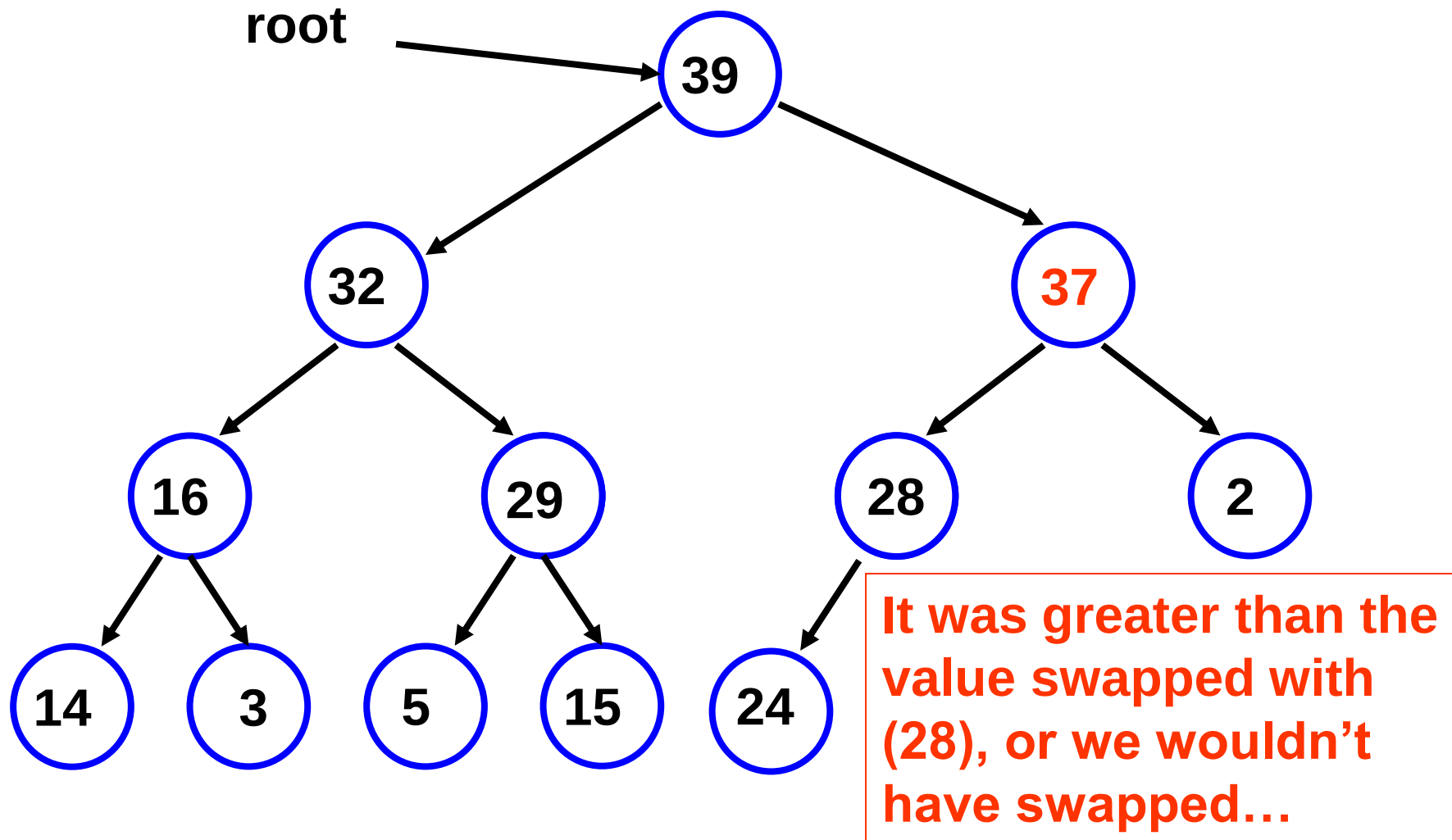


Enqueue (cont.)

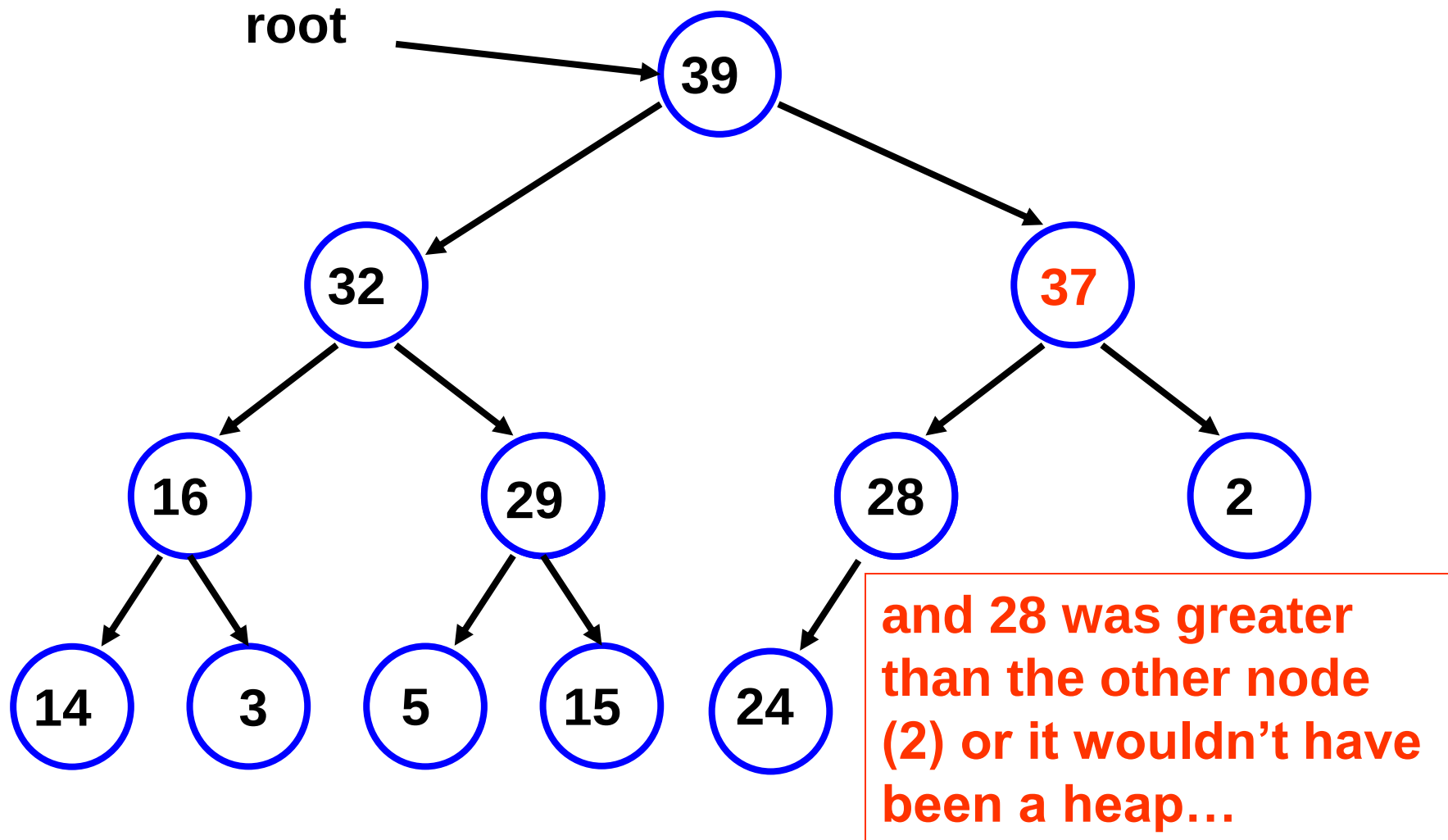


Notice that 37 is guaranteed to be greater than or equal to its children

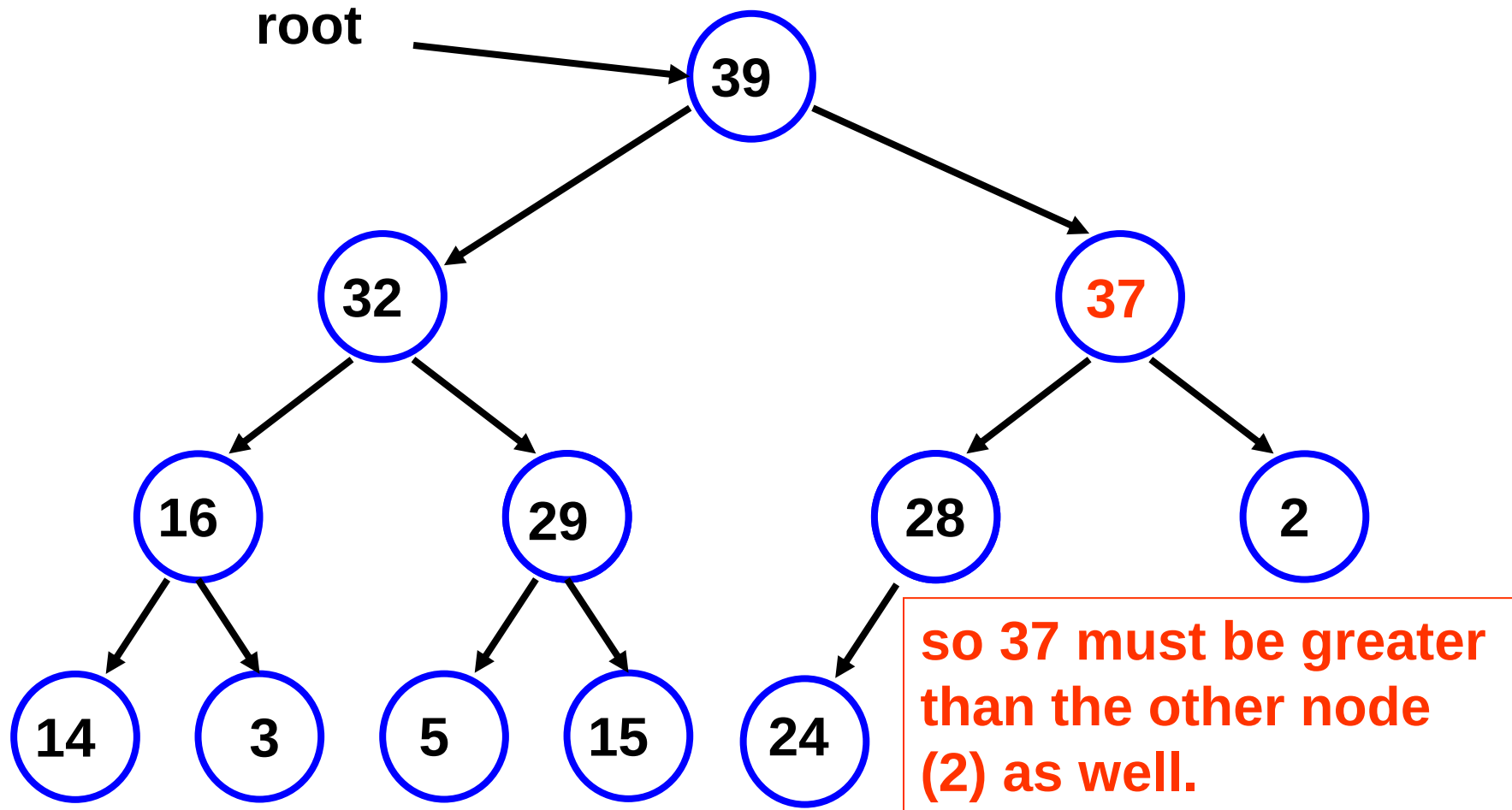
Enqueue (cont.)



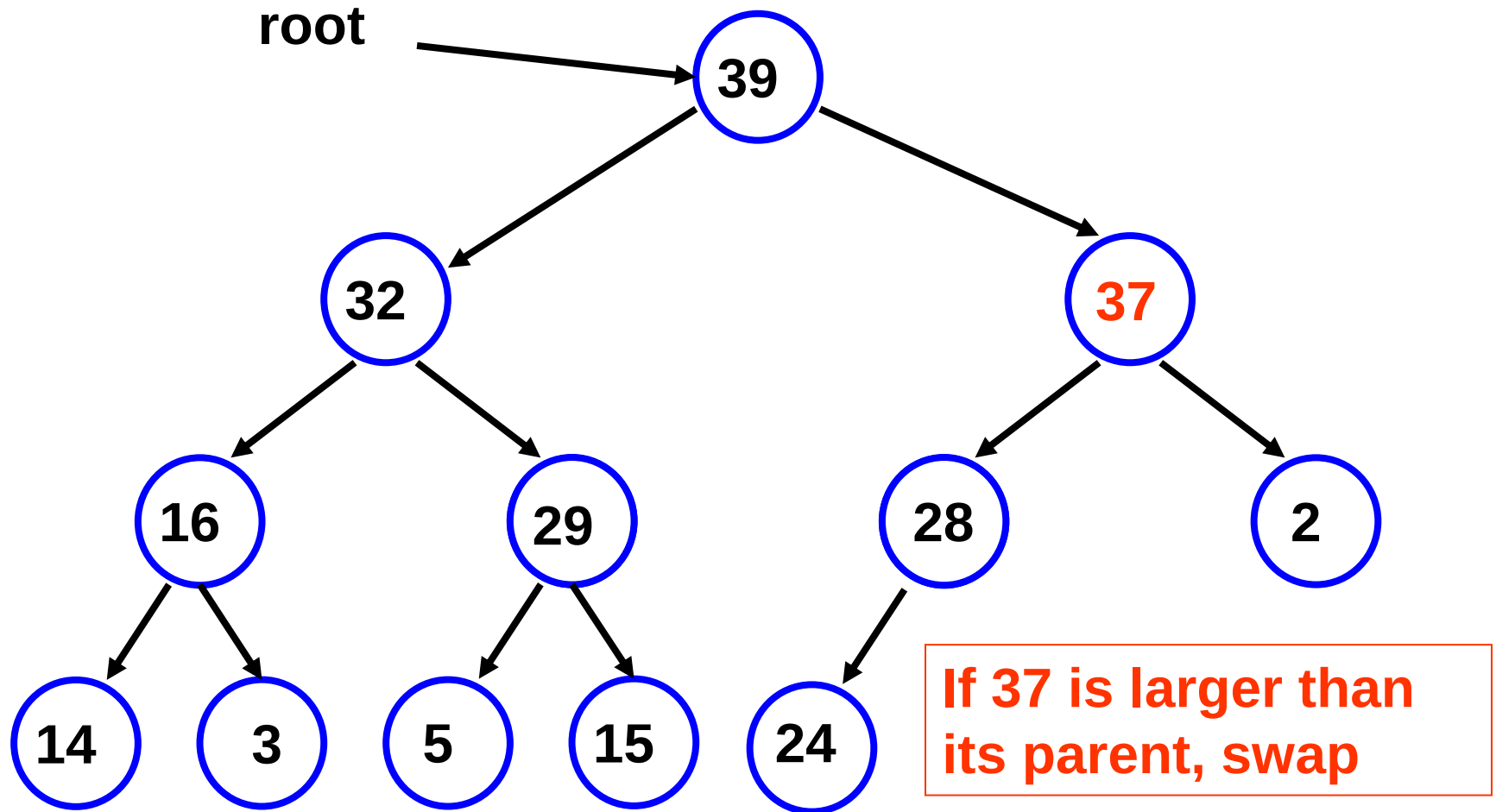
Enqueue (cont.)



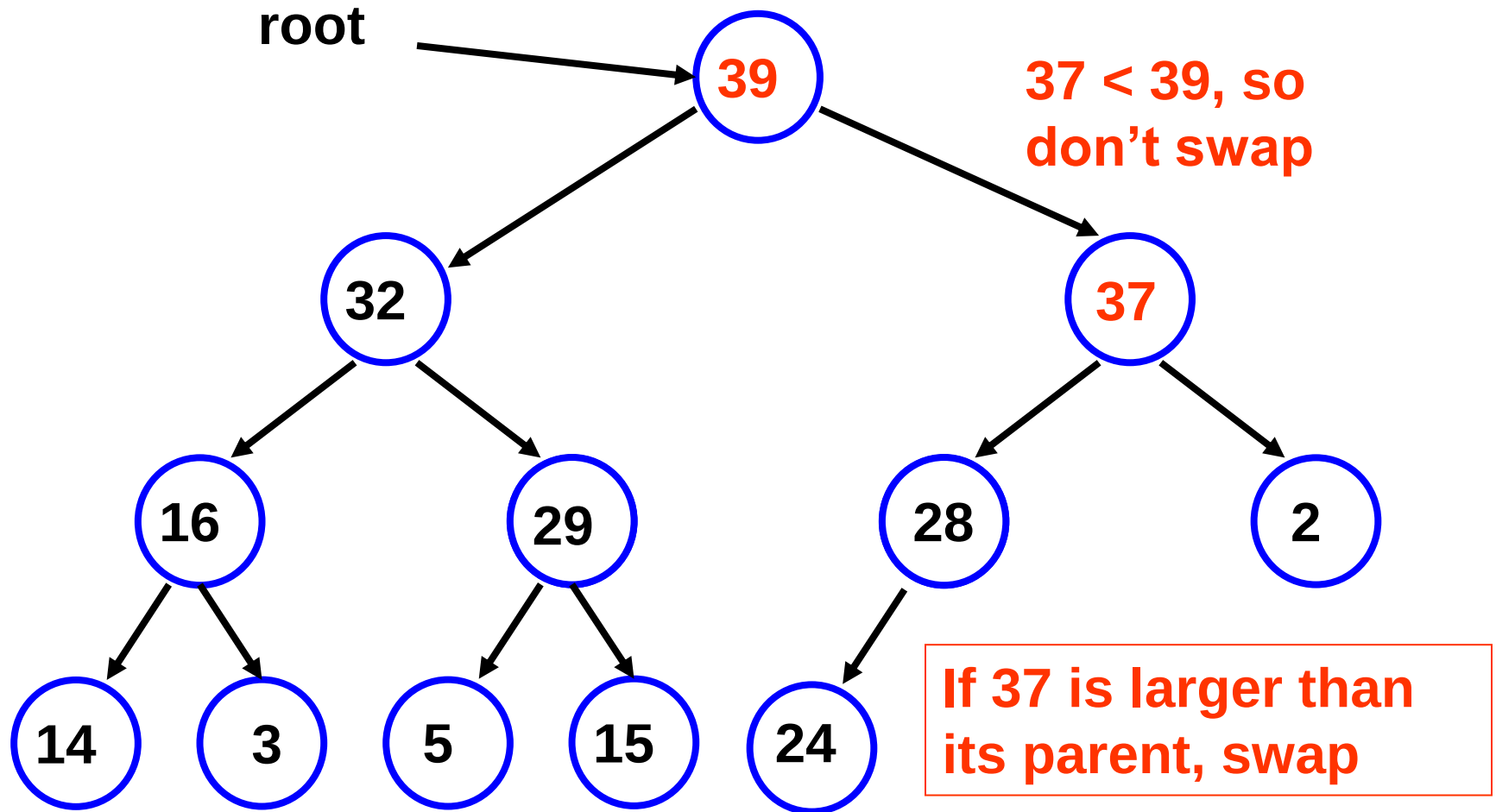
Enqueue (cont.)



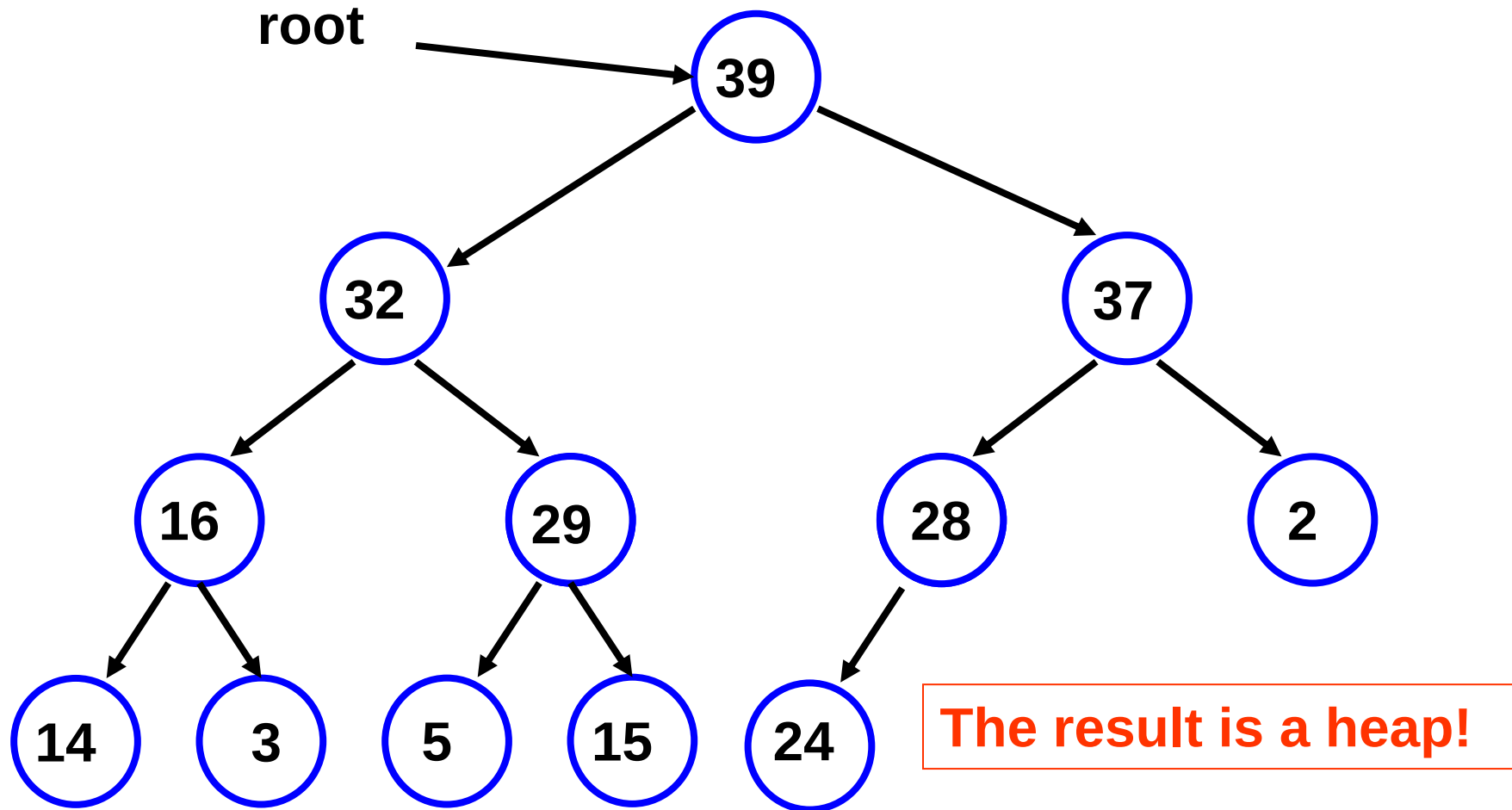
Enqueue (cont.)



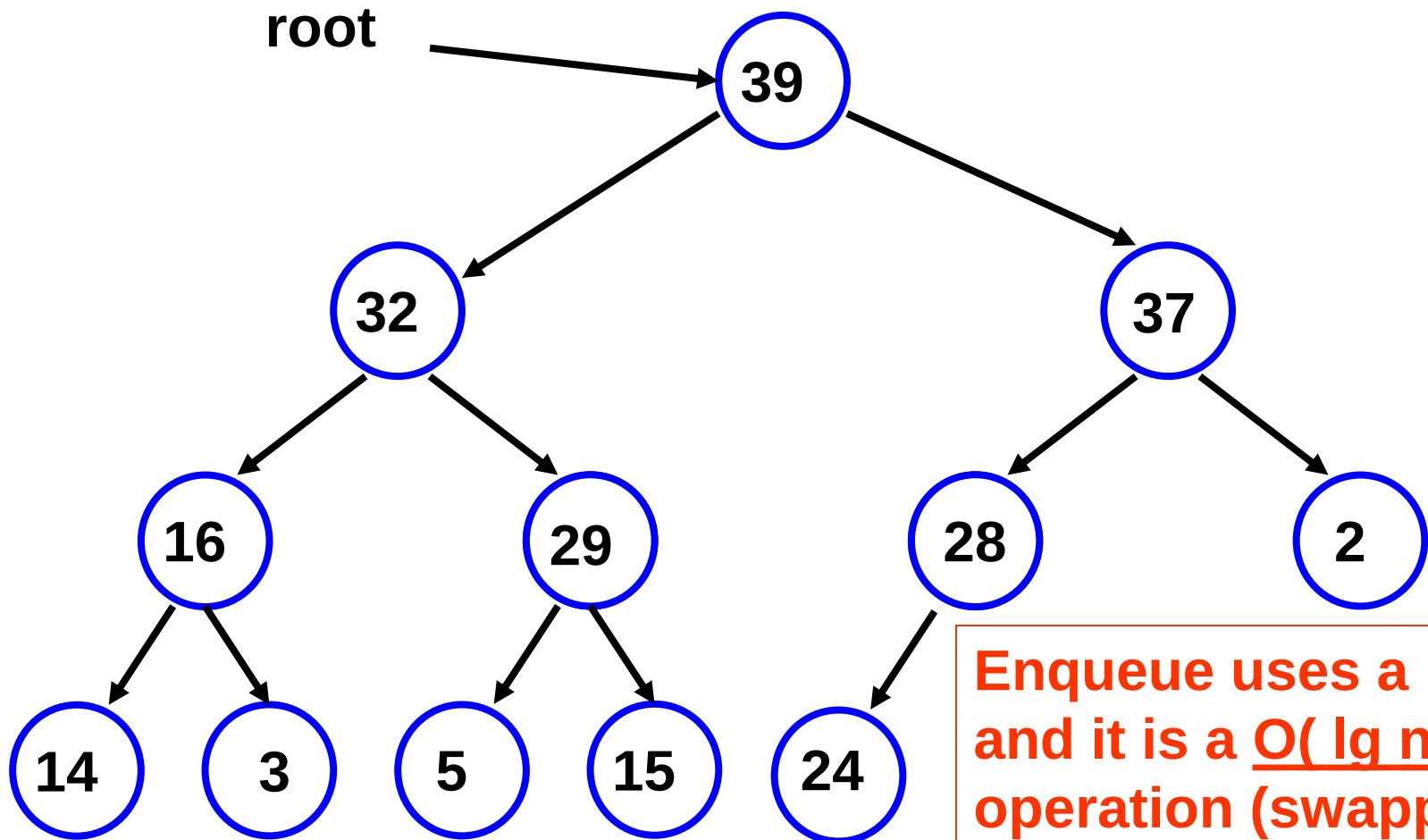
Enqueue (cont.)



Enqueue (cont.)

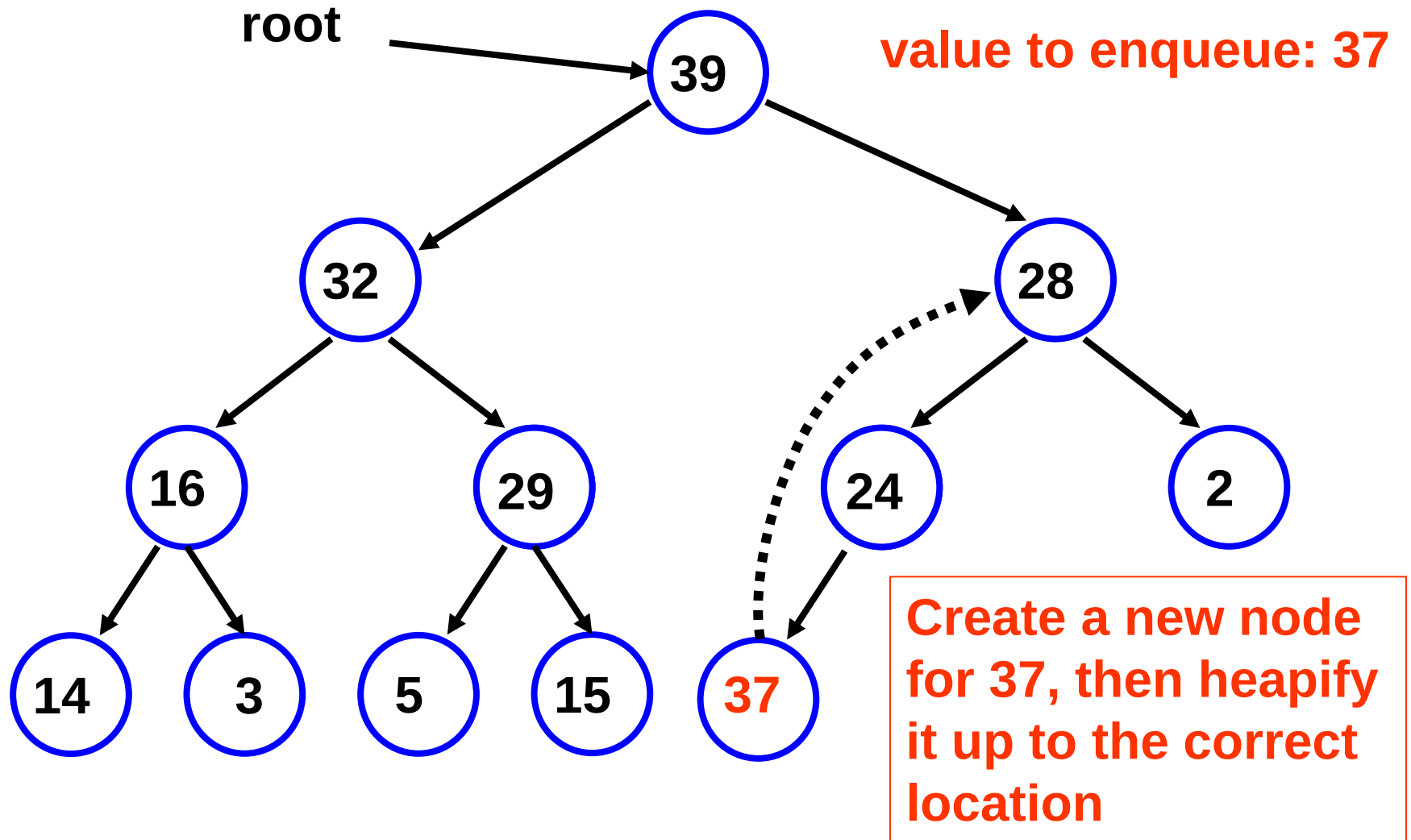


Enqueue (cont.)

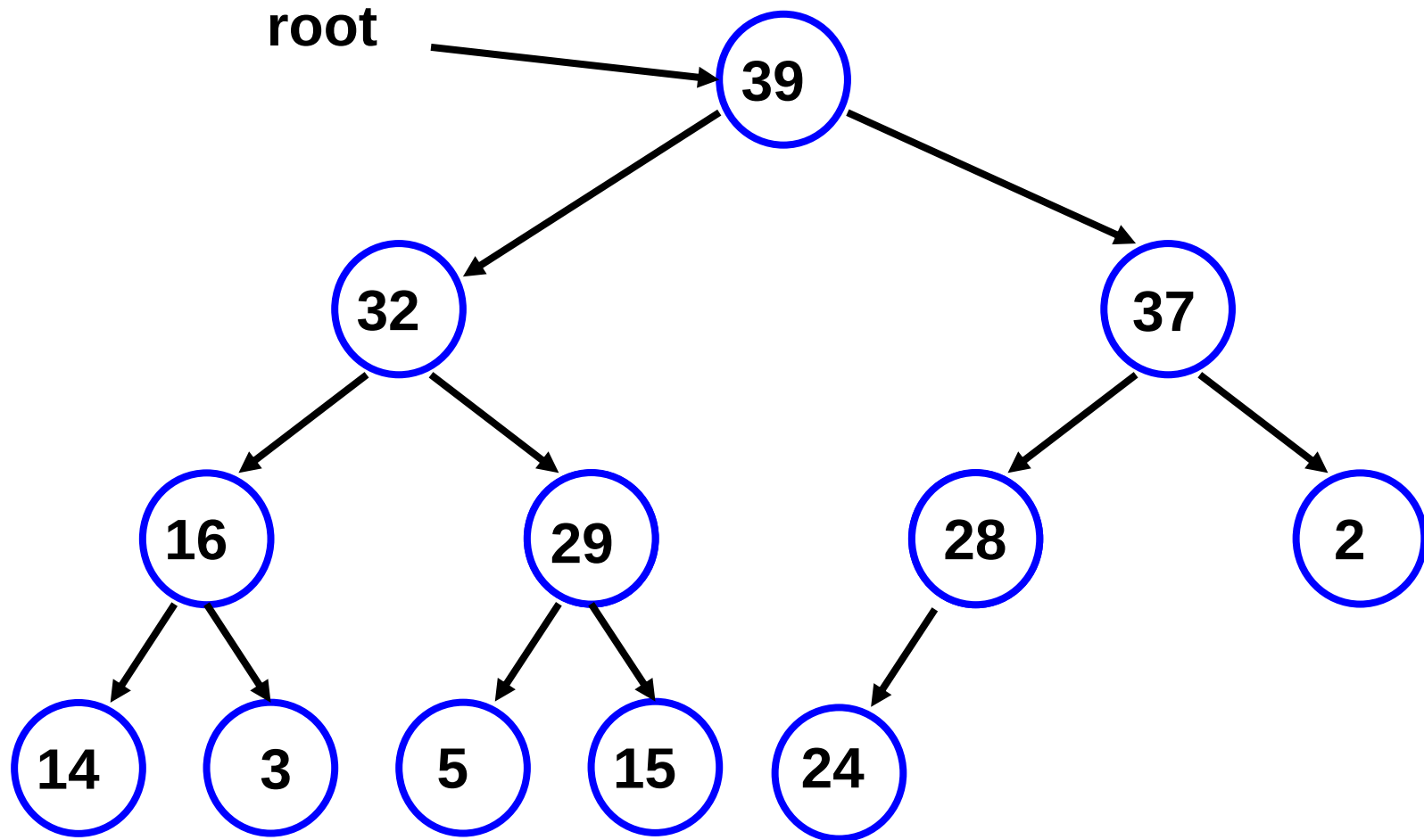


Enqueue uses a loop, and it is a $O(\lg n)$ operation (swapping in reverse)

Enqueue In Short



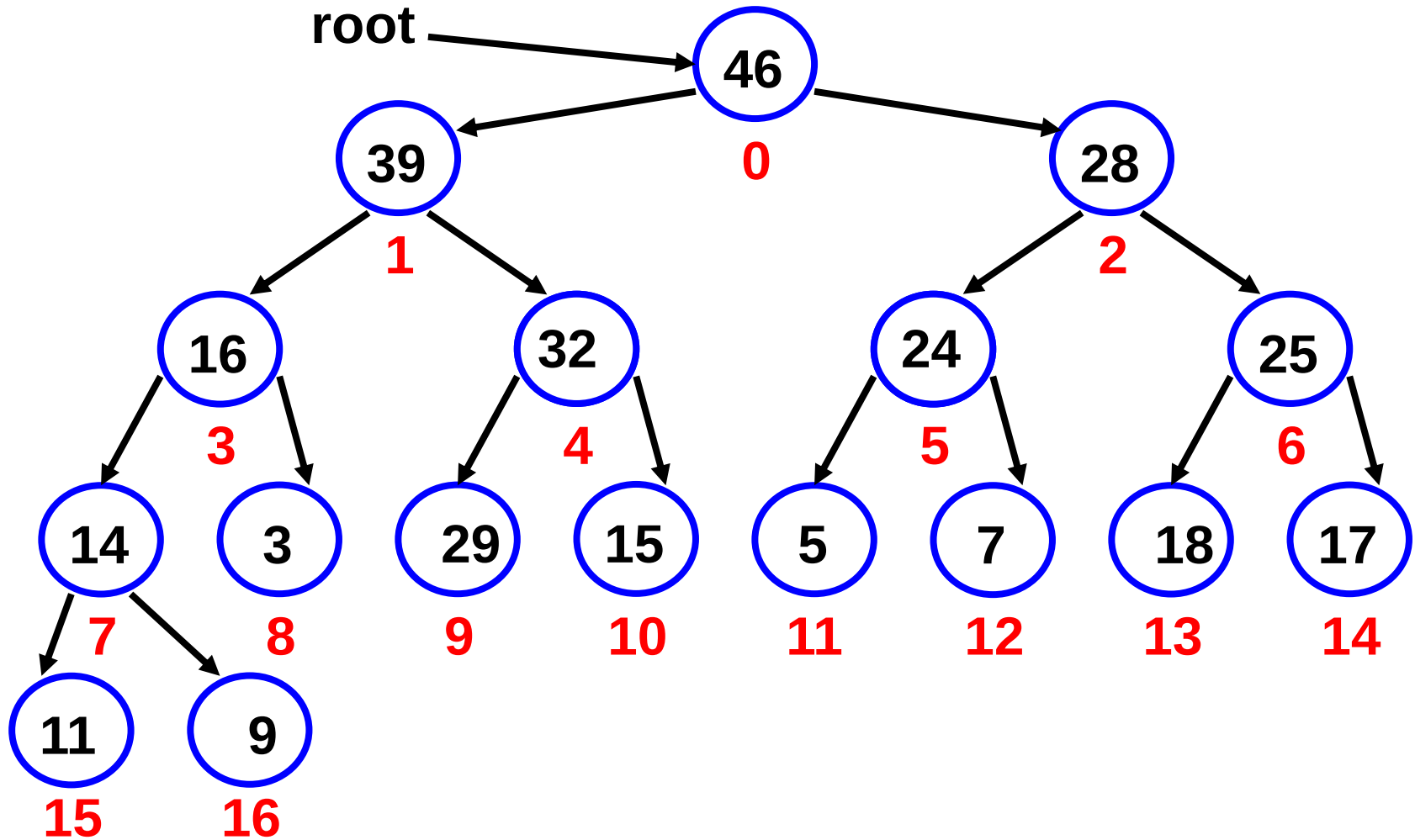
Enqueue In Short (cont.)



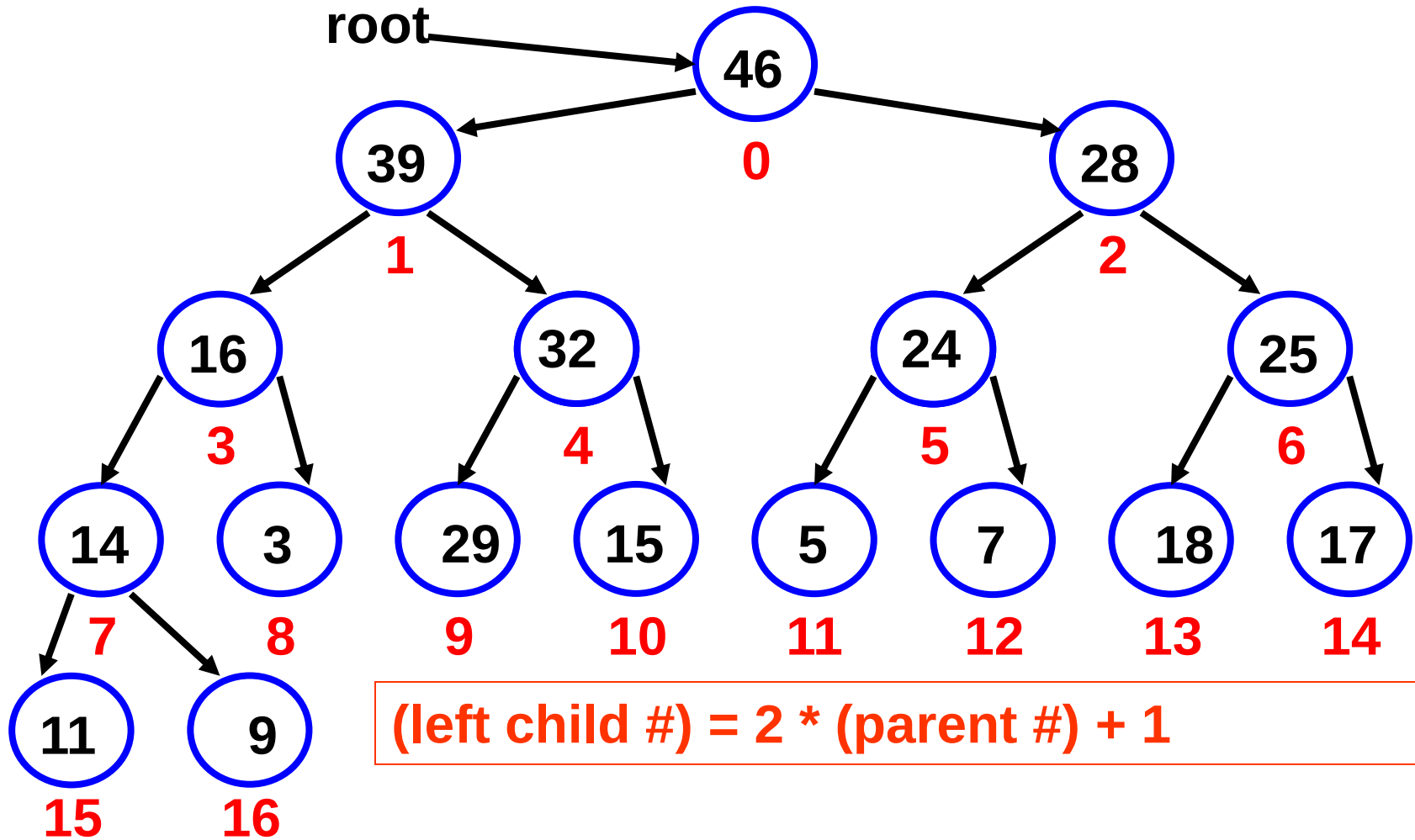
Implementing a Heap

- Although it is helpful to think of a heap as a linked structure when visualizing the enqueue and dequeue operations, it is often implemented with an array.
- Don't get mixed up with implementing a priority queue (PQ) and implementing a heap.
- We have discussed earlier that to implement a PQ, heap is better than array overall.
- We are discussing implementing a heap here
- Let's number the nodes of a heap, starting with 0, going top to bottom and left to right...

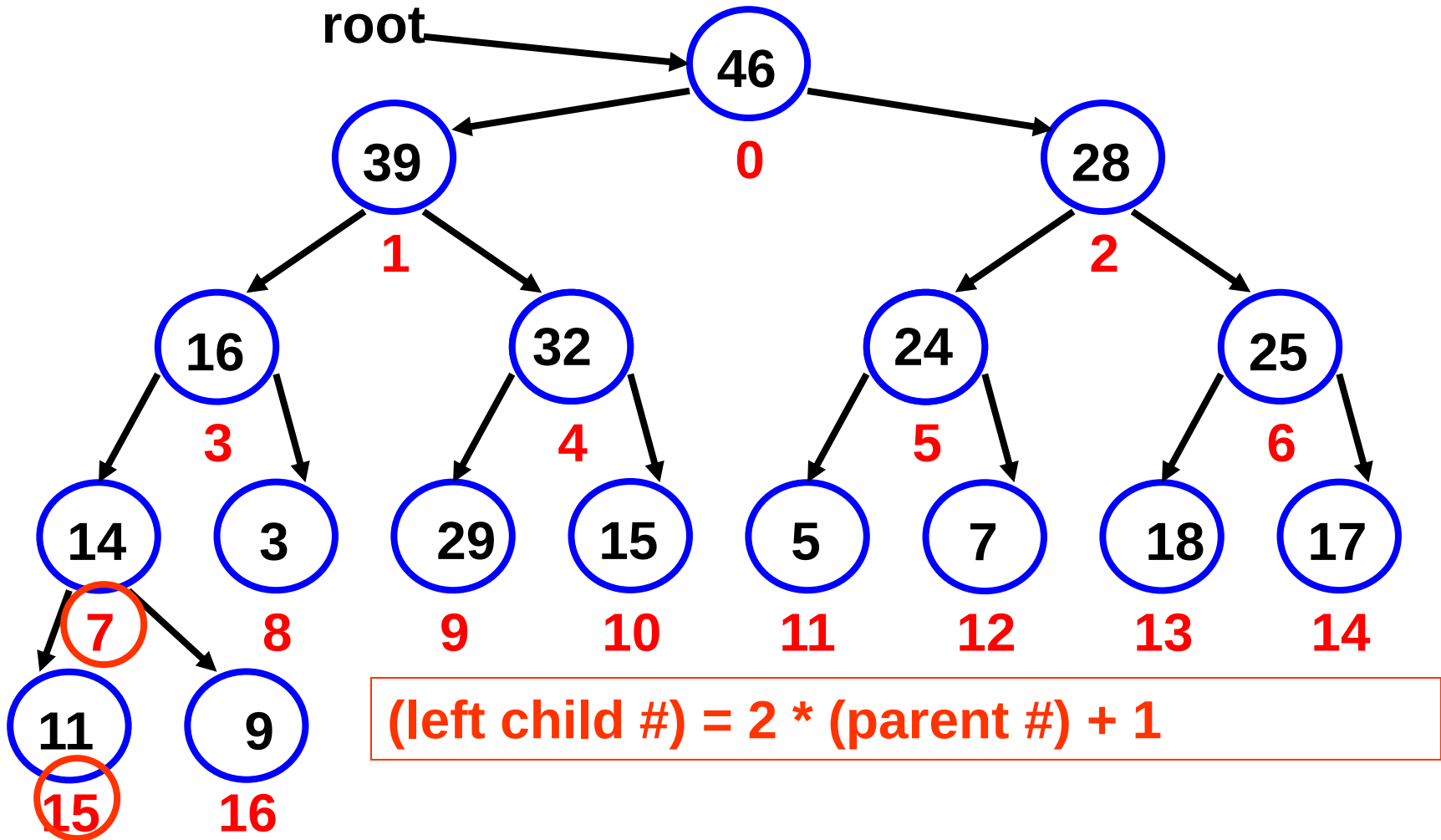
Implementing a Heap with Array



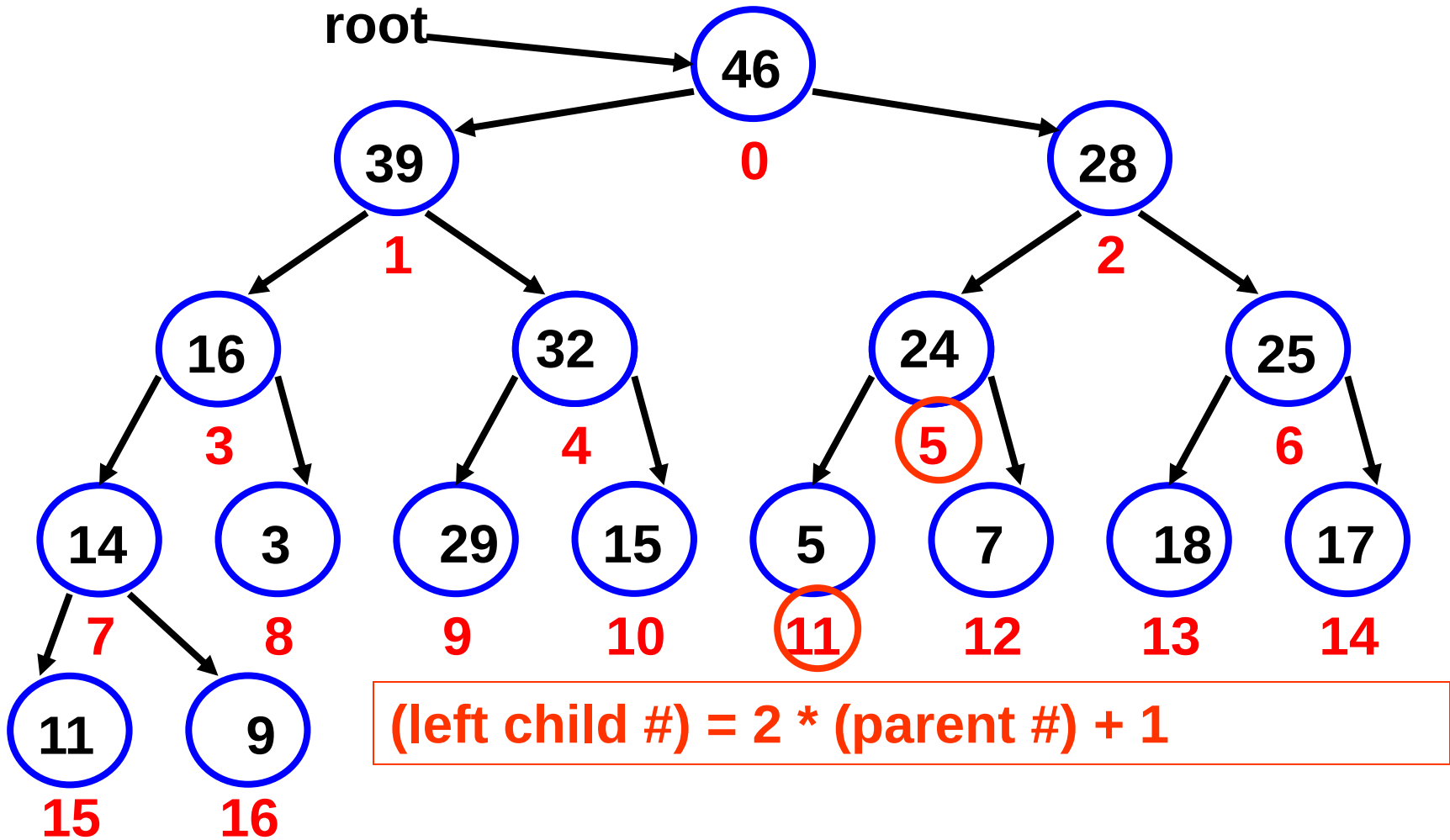
Heap Properties



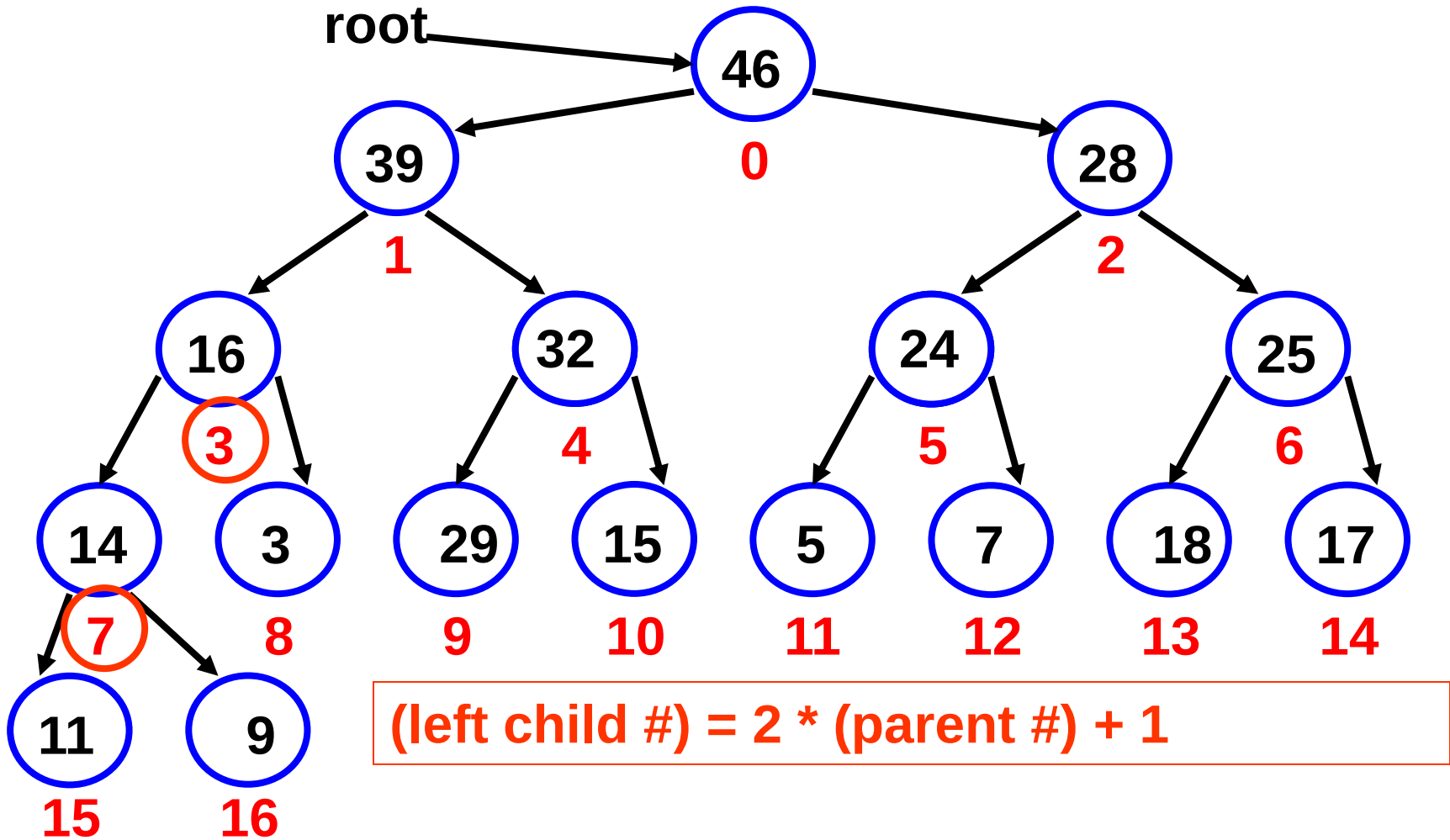
Heap Properties (cont.)



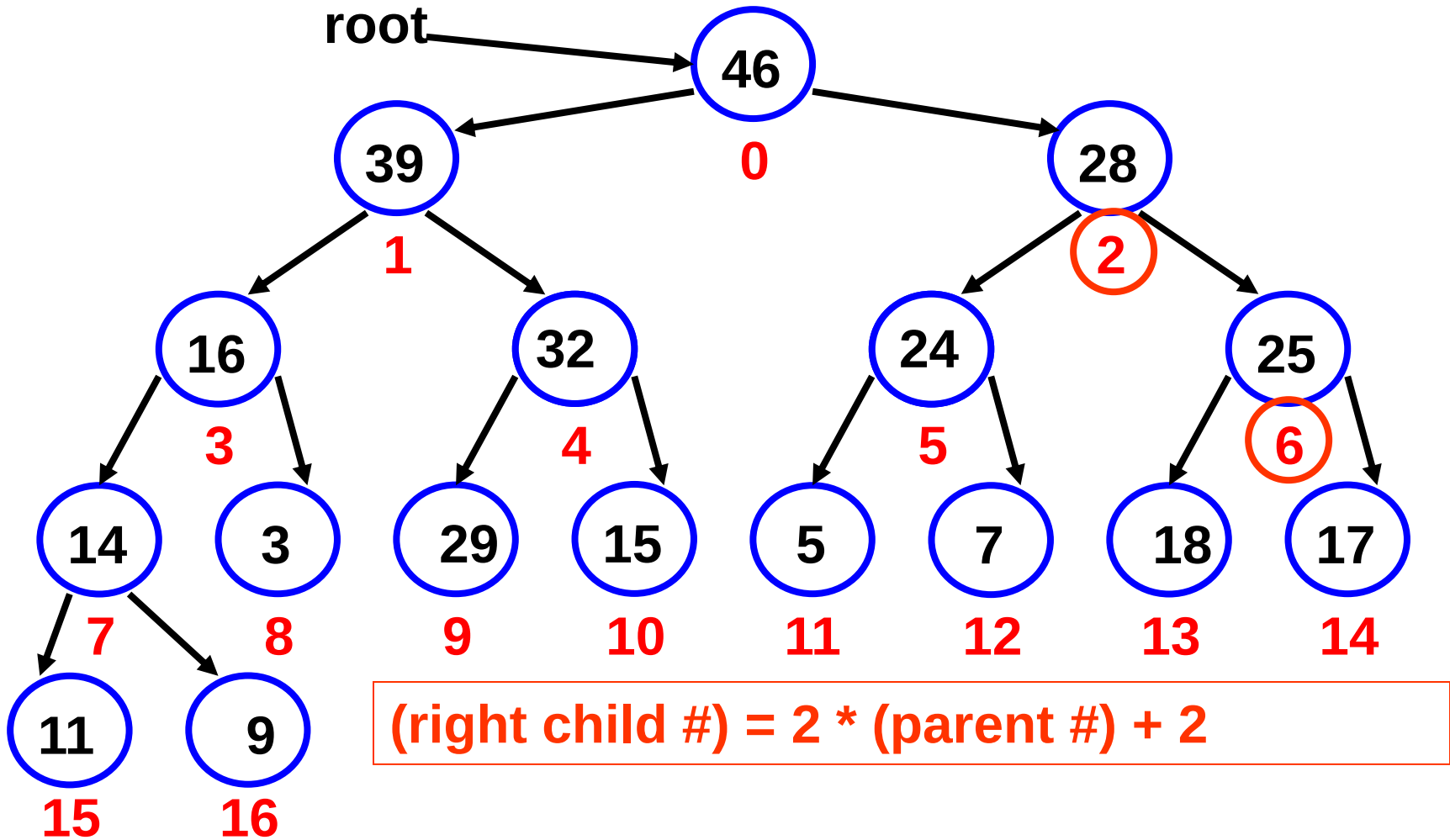
Heap Properties (cont.)



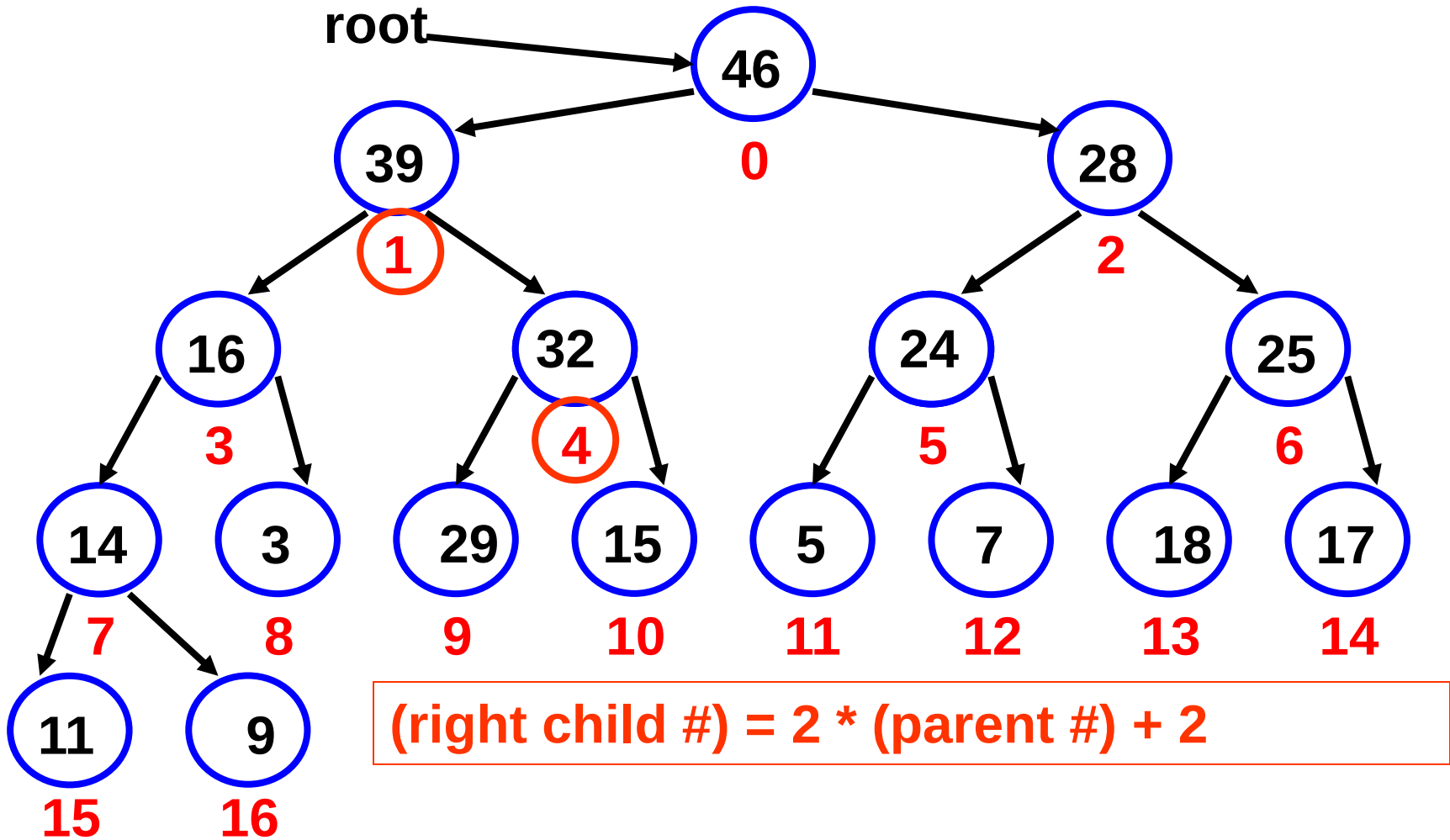
Heap Properties (cont.)



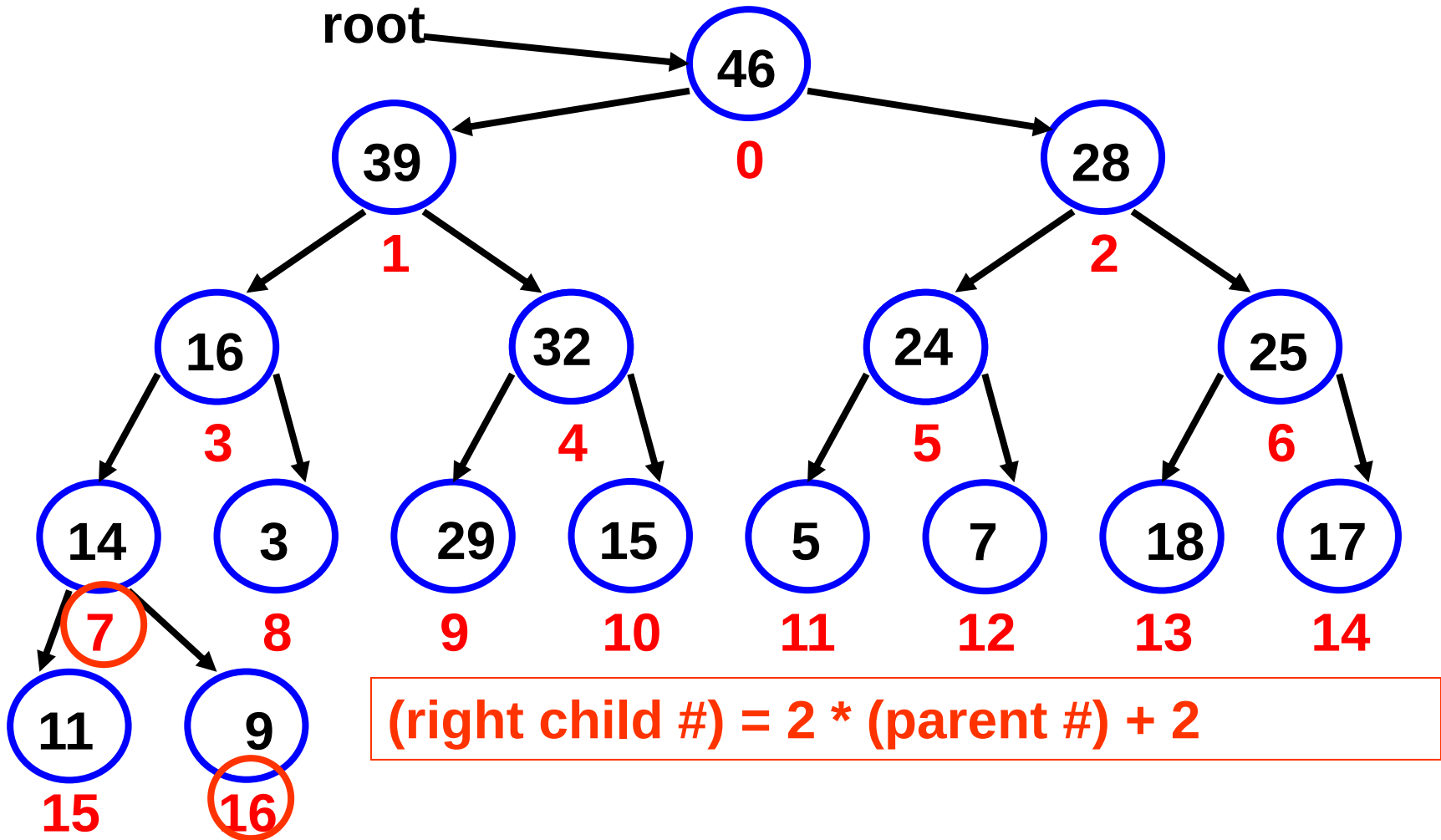
Heap Properties (cont.)



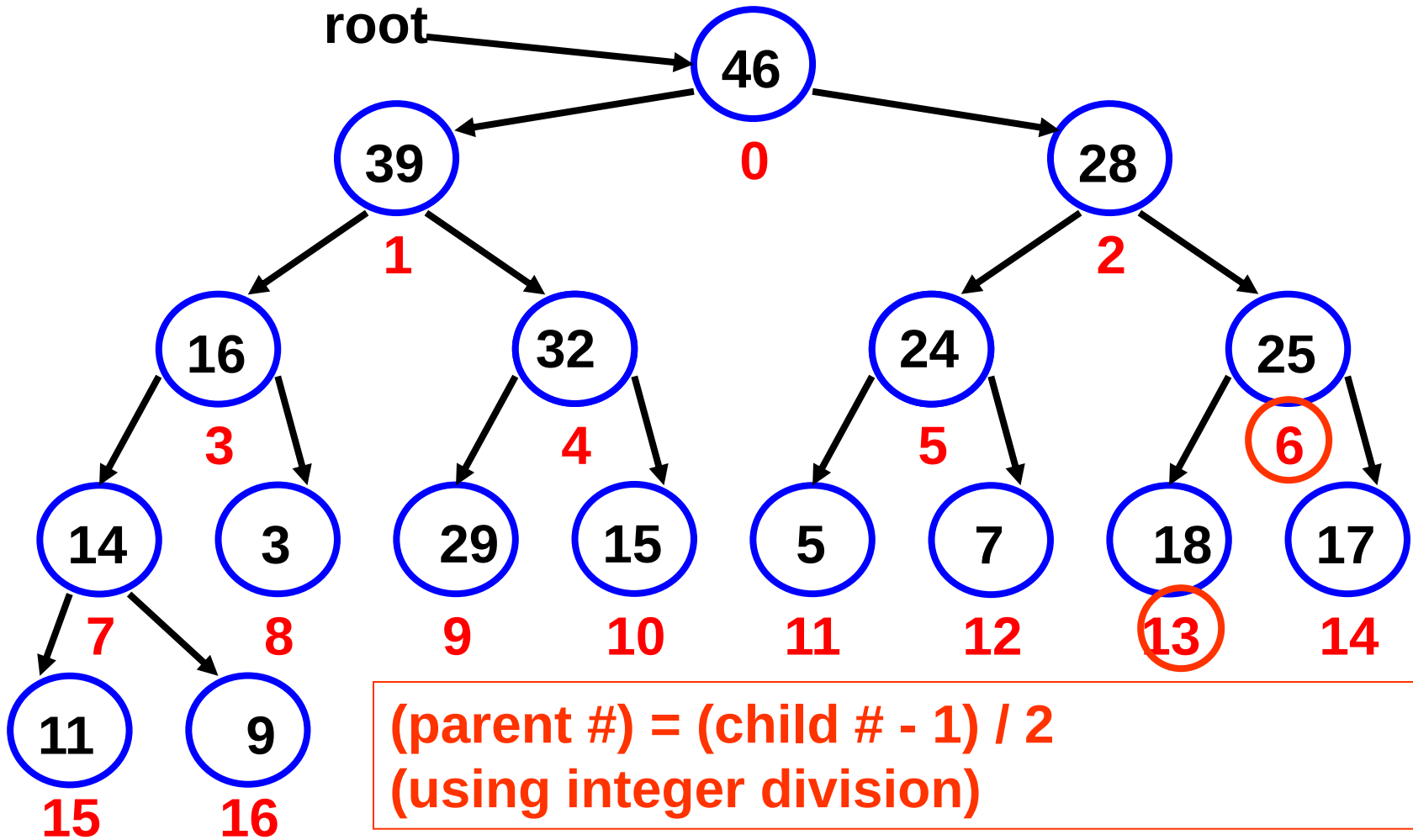
Heap Properties (cont.)



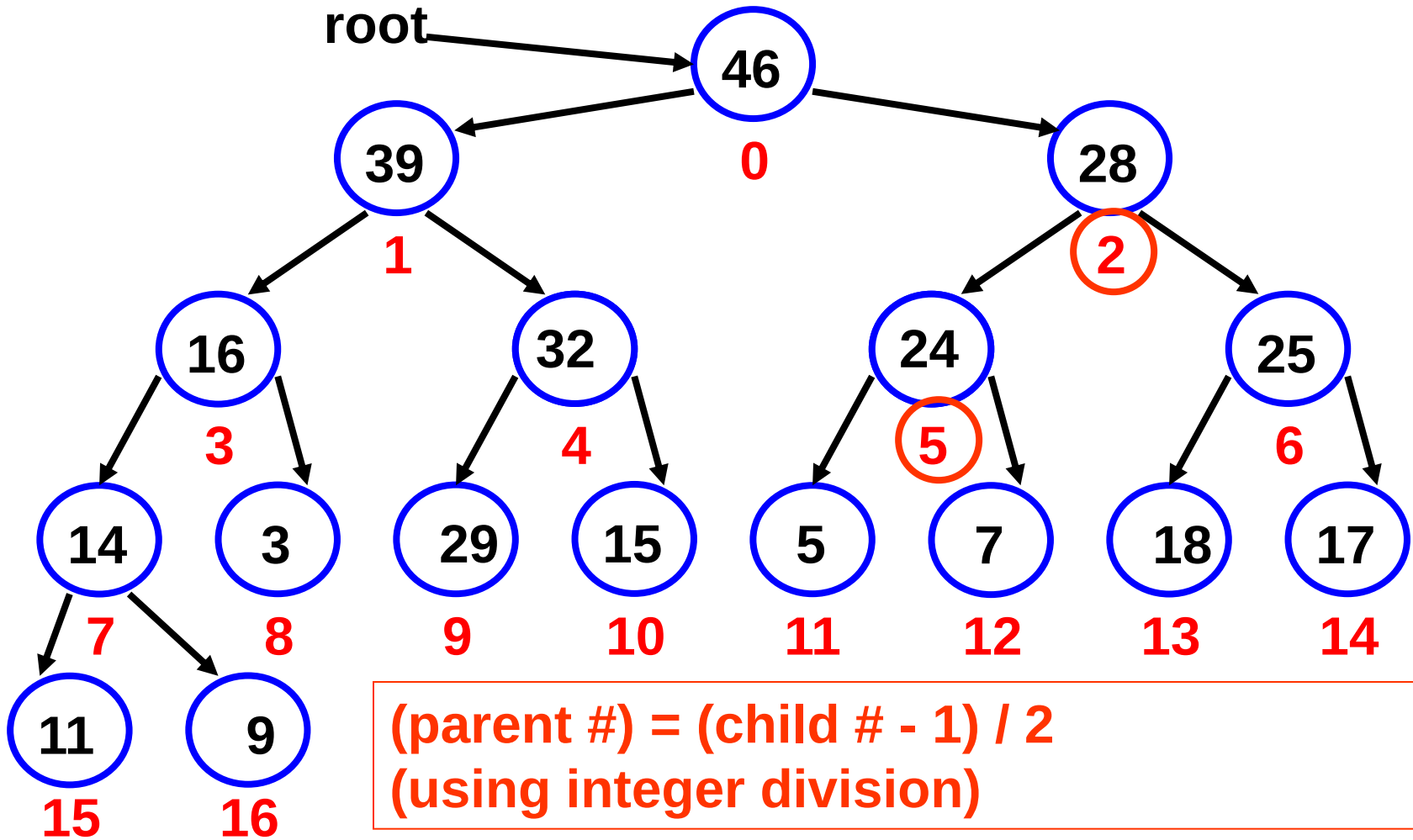
Heap Properties (cont.)



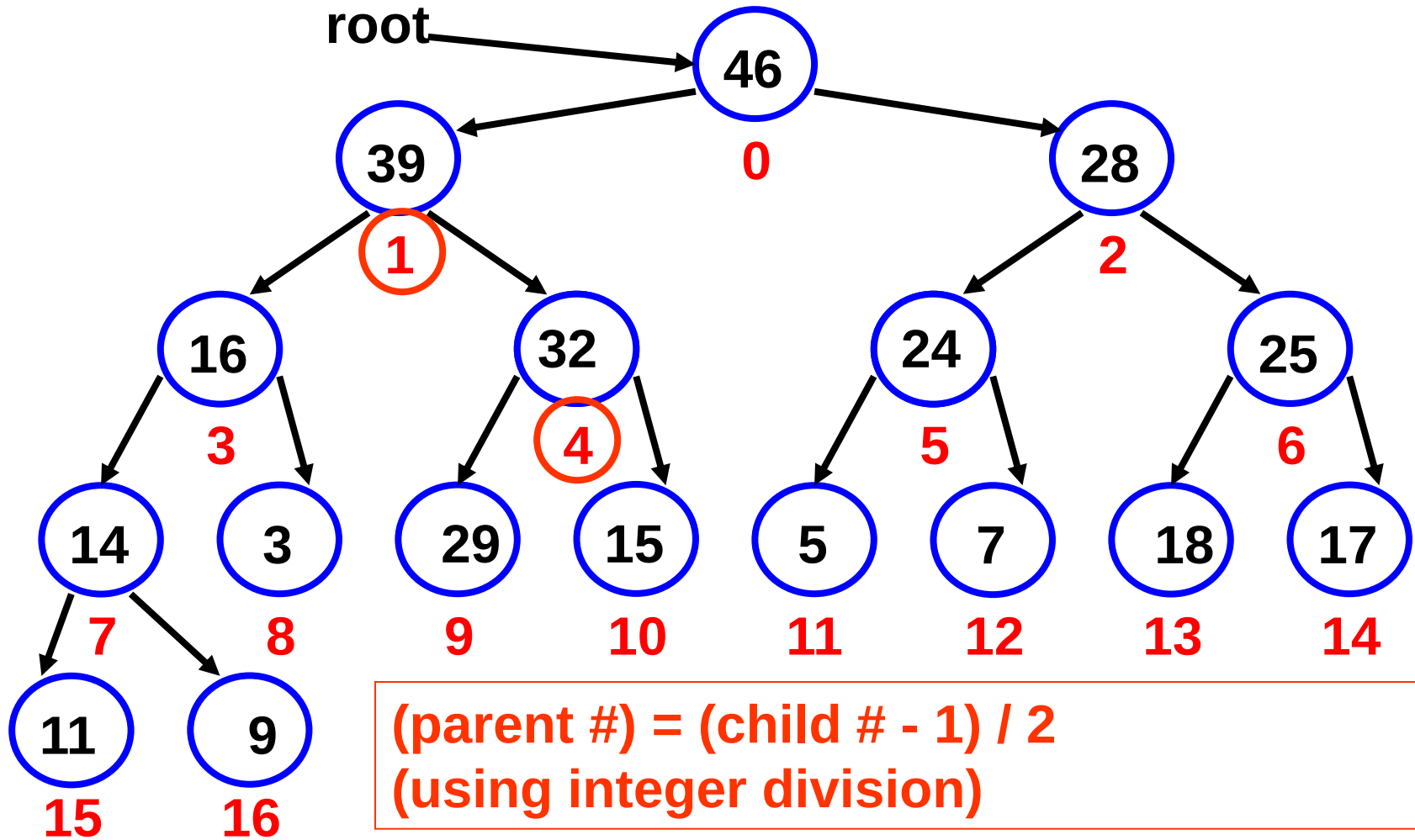
Heap Properties (cont.)



Heap Properties (cont.)



Heap Properties (cont.)



Array Implementation of Heap

- These remarkable properties of the heap allow us to place the elements into an array.
- The red numbers on the previous slide correspond to the array indexes.

Array Implementation of Heap (cont.)

46	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

So now, this is our heap. It has no linked nodes, so it is much easier to work with.

Let's dequeue an element.

The highest value is stored in the root node (index 0).

Array Implementation of Heap (cont.)

46	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

Now we need to move the object at the last node to the root node

We need to keep track of the object in the last position of the heap using a heapsize variable

Array Implementation of Heap (cont.)

46	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 17

Now we need to move the object at the last node to the root node

We need to keep track of the object in the last position of the heap using a heapsize variable

Array Implementation of Heap (cont.)

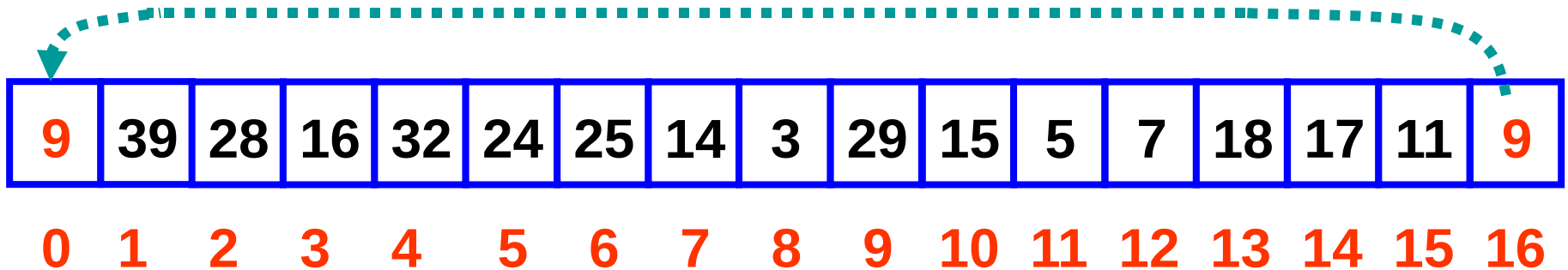
46	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 17

Now we can access the object in the last node using `elements[ heapsize - 1 ]` and assign it to `elements[ 0 ]`

Array Implementation of Heap (cont.)



remElement: 46

heapsize: 17

Now we can access the object in the last node using `elements[ heapsize - 1 ]` and assign it to `elements[ 0 ]`

Array Implementation of Heap (cont.)

9	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 17

Next, decrement the heap size

Array Implementation of Heap (cont.)

9	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

Next, decrement the heap size

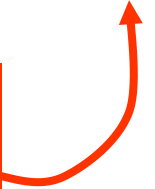
Array Implementation of Heap (cont.)

9	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

The value at index 16 can't be used anymore; it will be overwritten on the next enqueue



Array Implementation of Heap (cont.)

9	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

Now, we need to find the
greatest child of node 0
and compare it to 9.

But how do we get the
greatest child?

Array Implementation of Heap (cont.)

9	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

By using the formulas we noted earlier
(this is why an array can be used)...

Array Implementation of Heap (cont.)

Greatest Child $39 > 9$, so swap

9	39	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

(left child #) =
 $2 * (\text{parent \#}) + 1 =$
 $2 * 0 + 1 =$
1

(right child #) =
 $2 * (\text{parent \#}) + 2 =$
 $2 * 0 + 2 =$
2

Array Implementation of Heap (cont.)

39	9	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

If the greatest child of 9 is
greater than 9, then swap

Array Implementation of Heap (cont.)

Greatest Child $32 > 9$, so swap

39	9	28	16	32	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

(left child #) =
 $2 * (\text{parent \#}) + 1 =$
 $2 * 1 + 1 =$
3

(right child #) =
 $2 * (\text{parent \#}) + 2 =$
 $2 * 1 + 2 =$
4

Array Implementation of Heap (cont.)

39	32	28	16	9	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

If the greatest child of 9 is
greater than 9, then swap

Array Implementation of Heap (cont.)

Greatest Child $29 > 9$, so swap

39	32	28	16	9	24	25	14	3	29	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

(left child #) =
 $2 * (\text{parent \#}) + 1 =$
 $2 * 4 + 1 =$
9

(right child #) =
 $2 * (\text{parent \#}) + 2 =$
 $2 * 4 + 2 =$
10

Array Implementation of Heap (cont.)

39	32	28	16	29	24	25	14	3	9	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

If the greatest child of 9 is
greater than 9, then swap

Array Implementation of Heap (cont.)

39	32	28	16	29	24	25	14	3	9	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

remElement: 46

heapsize: 16

19 > heapsize

(left child #) =
 $2 * (\text{parent \#}) + 1 =$
 $2 * 9 + 1 =$
19

Array Implementation of Heap (cont.)

39	32	28	16	29	24	25	14	3	9	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

so 9 must be a leaf node

remElement: 46

heapsize: 16

(left child #) =
 $2 * (\text{parent \#}) + 1 =$
 $2 * 9 + 1 =$
19

(we can stop)

Array Implementation of Heap (cont.)

39	32	28	16	29	24	25	14	3	9	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

heapsize: 16

An enqueue is done by placing the new element at elements[heapsize], then swapping upwards

Array Implementation of Heap (cont.)

39	32	28	16	29	24	25	14	3	9	15	5	7	18	17	11	9
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

heapsize: 16

When enqueueing, the parent is always found by using the parent formula:

$$(\text{parent \#}) = (\text{child \#} - 1) / 2$$

Reducing the Work in a Swap

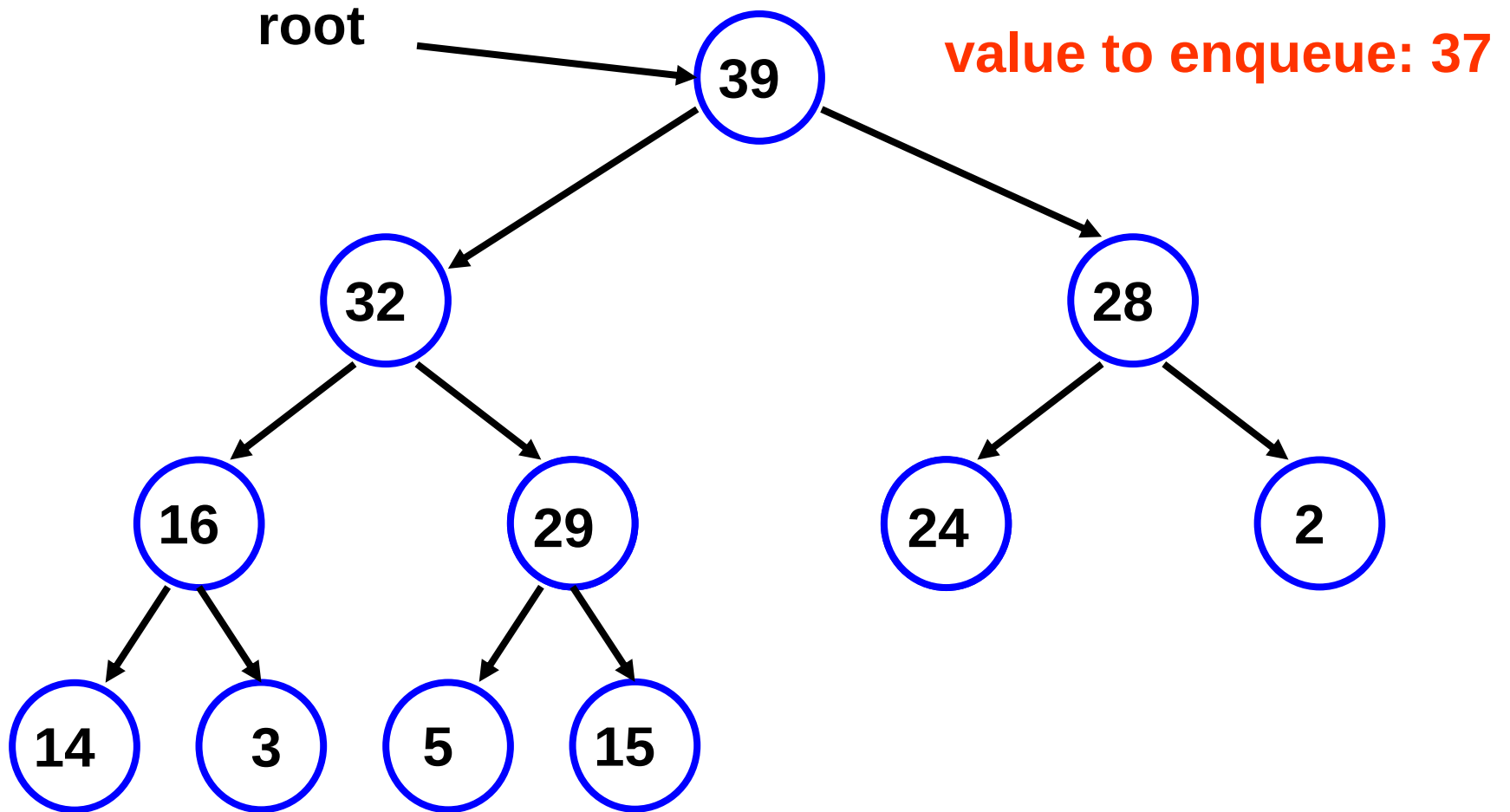
- A swap (using simple assignments) would normally involve three statements:

```
temp = elements[ i ];  
elements[ i ] = elements[ j ];  
elements[ j ] = temp;
```

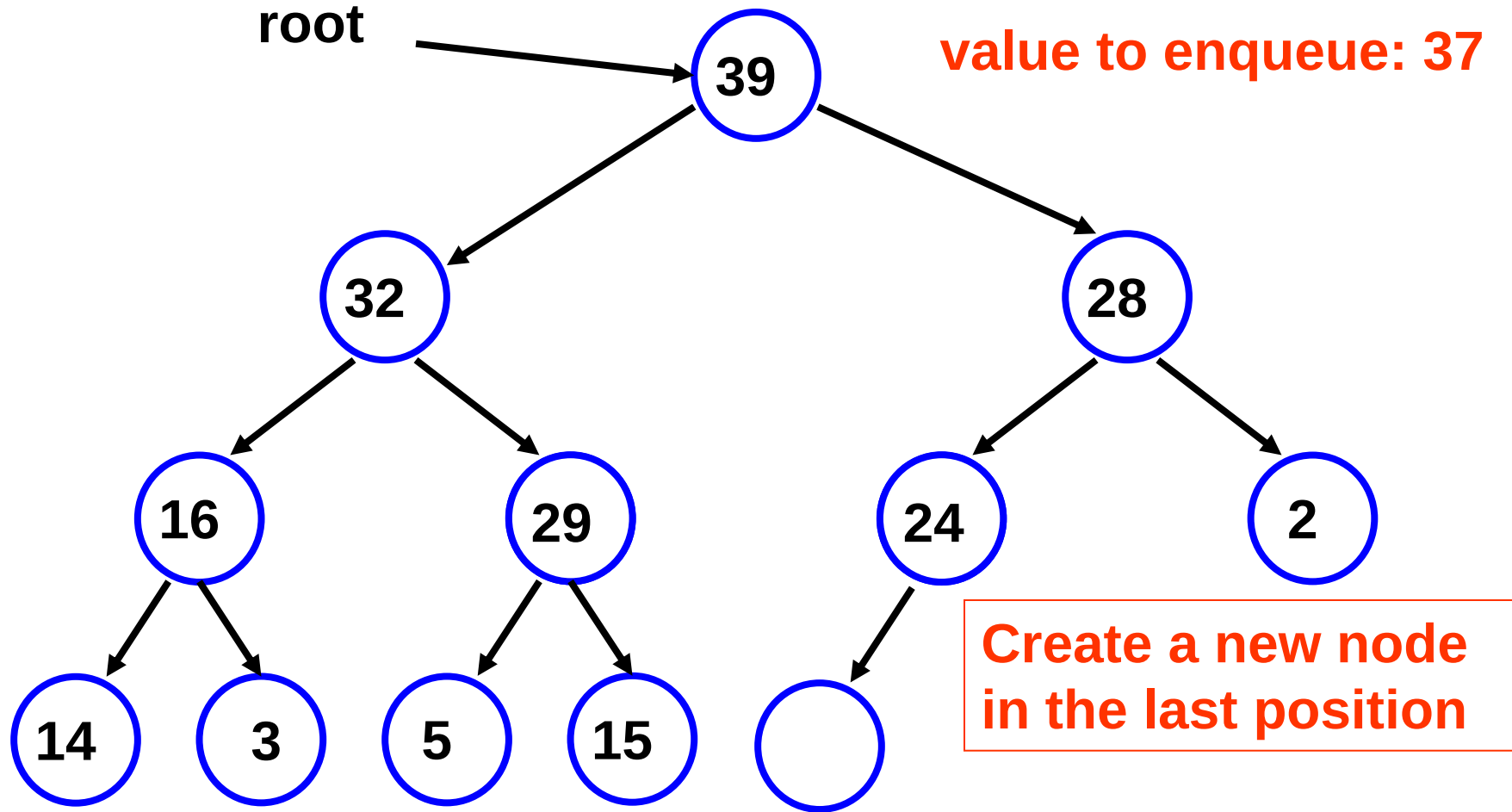
Reducing the Work in a Swap (cont.)

- In an enqueue or a dequeue for a heap, we do not really have to use this three-assignment swap (although it helped to visualize how the enqueue and dequeue process worked).
- We can save the value we are swapping upwards or downwards.
- Let's look at an enqueue...

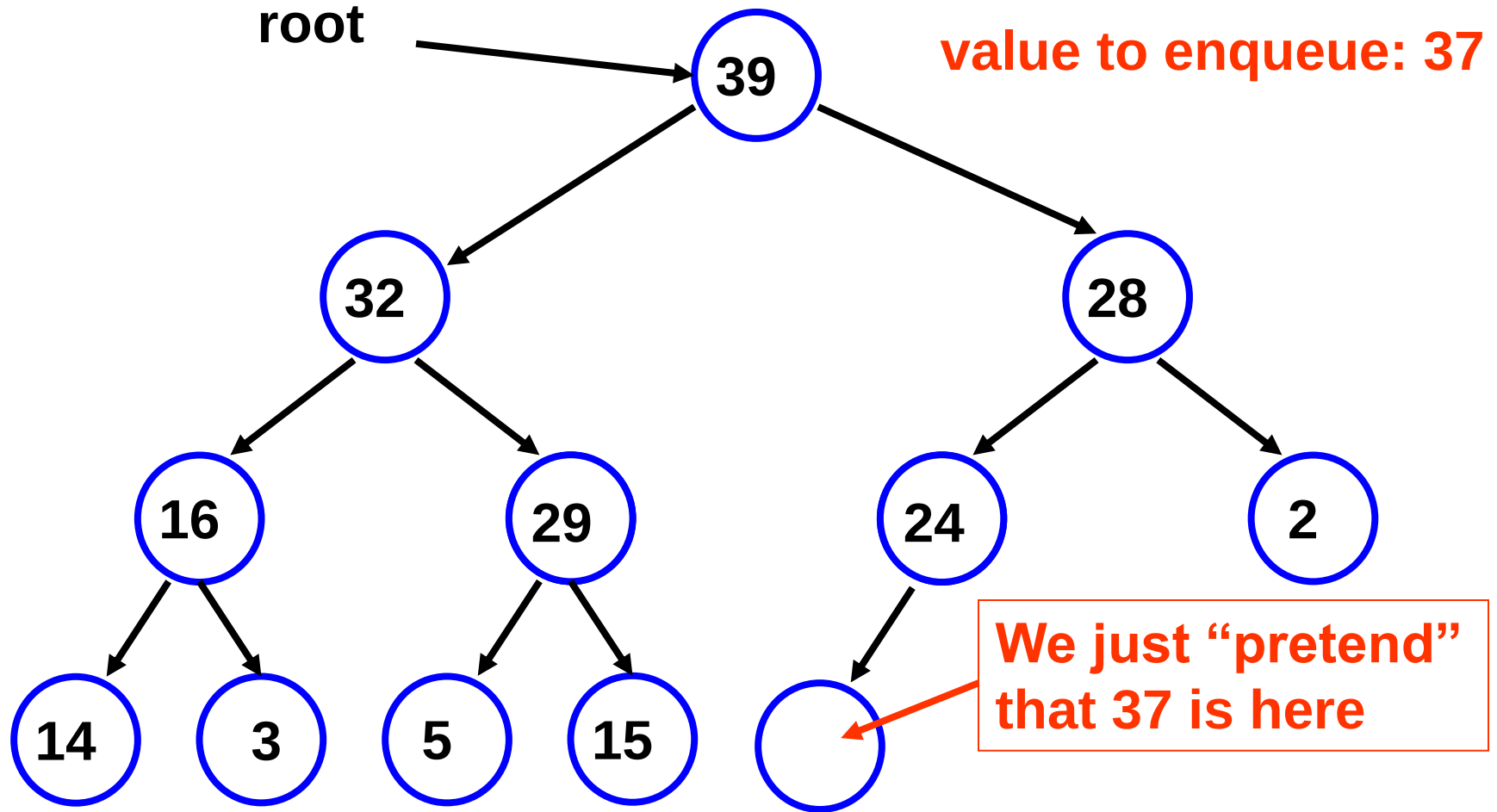
Enqueue



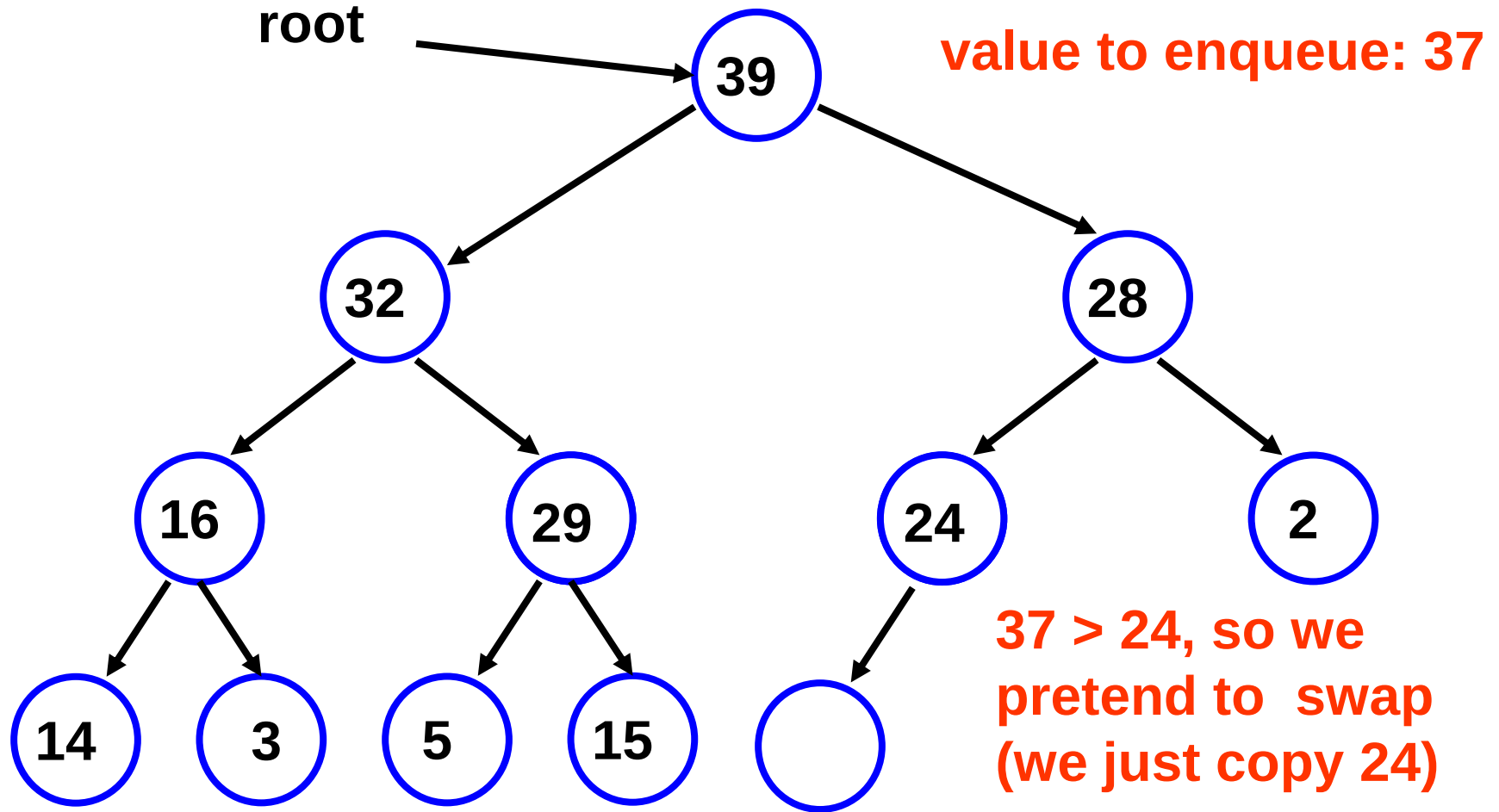
Enqueue (cont.)



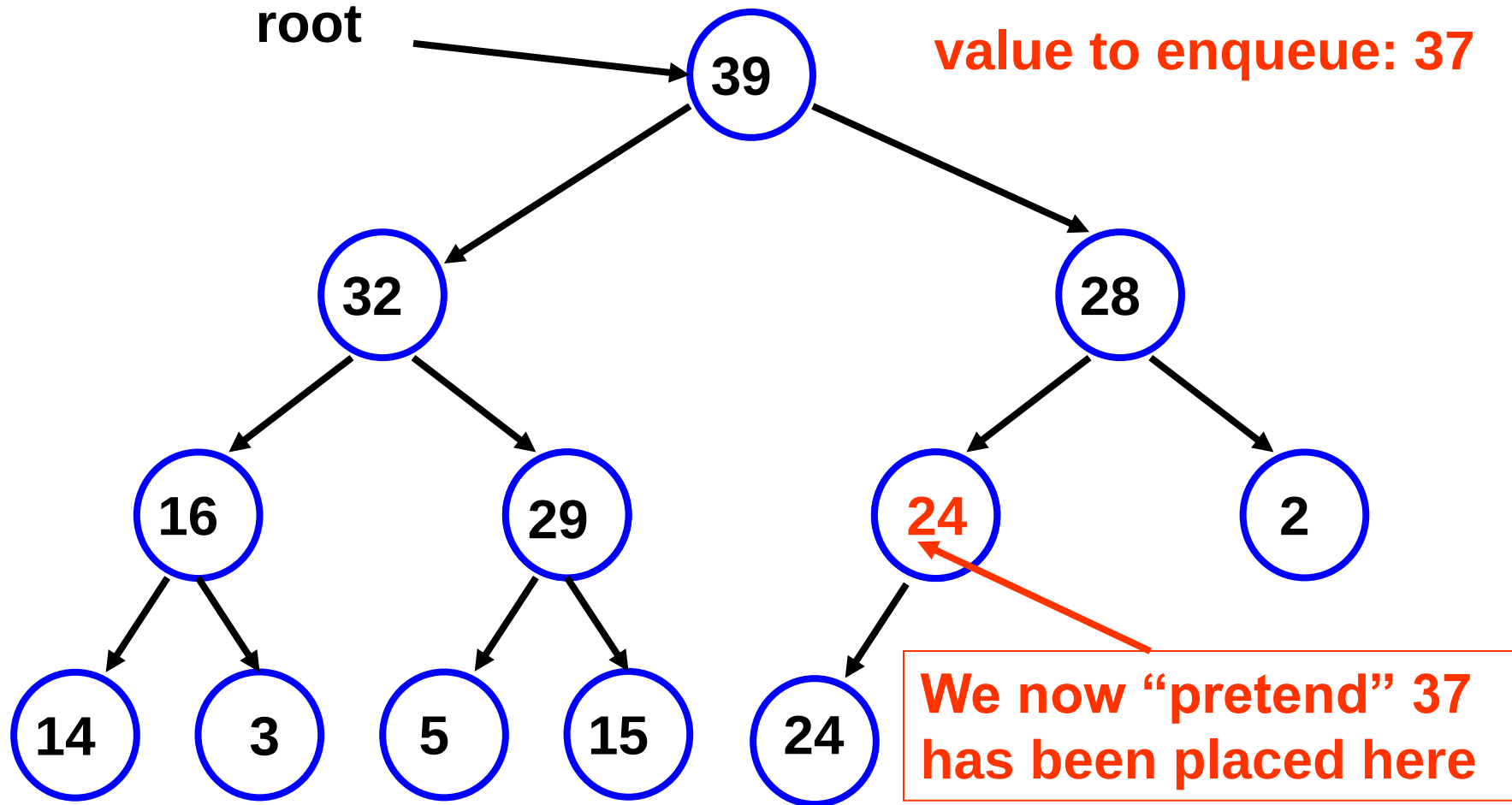
Enqueue (cont.)



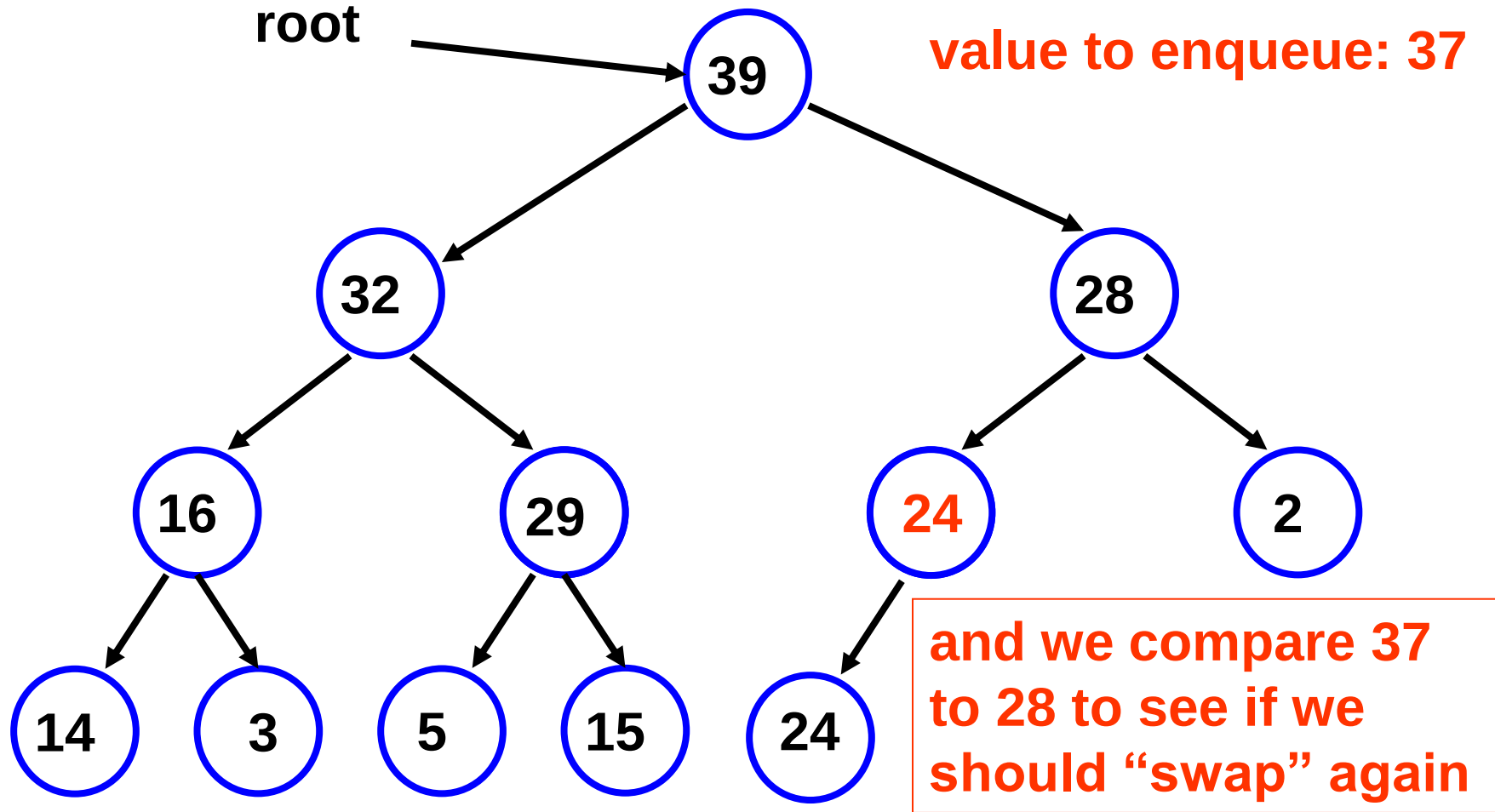
Enqueue (cont.)



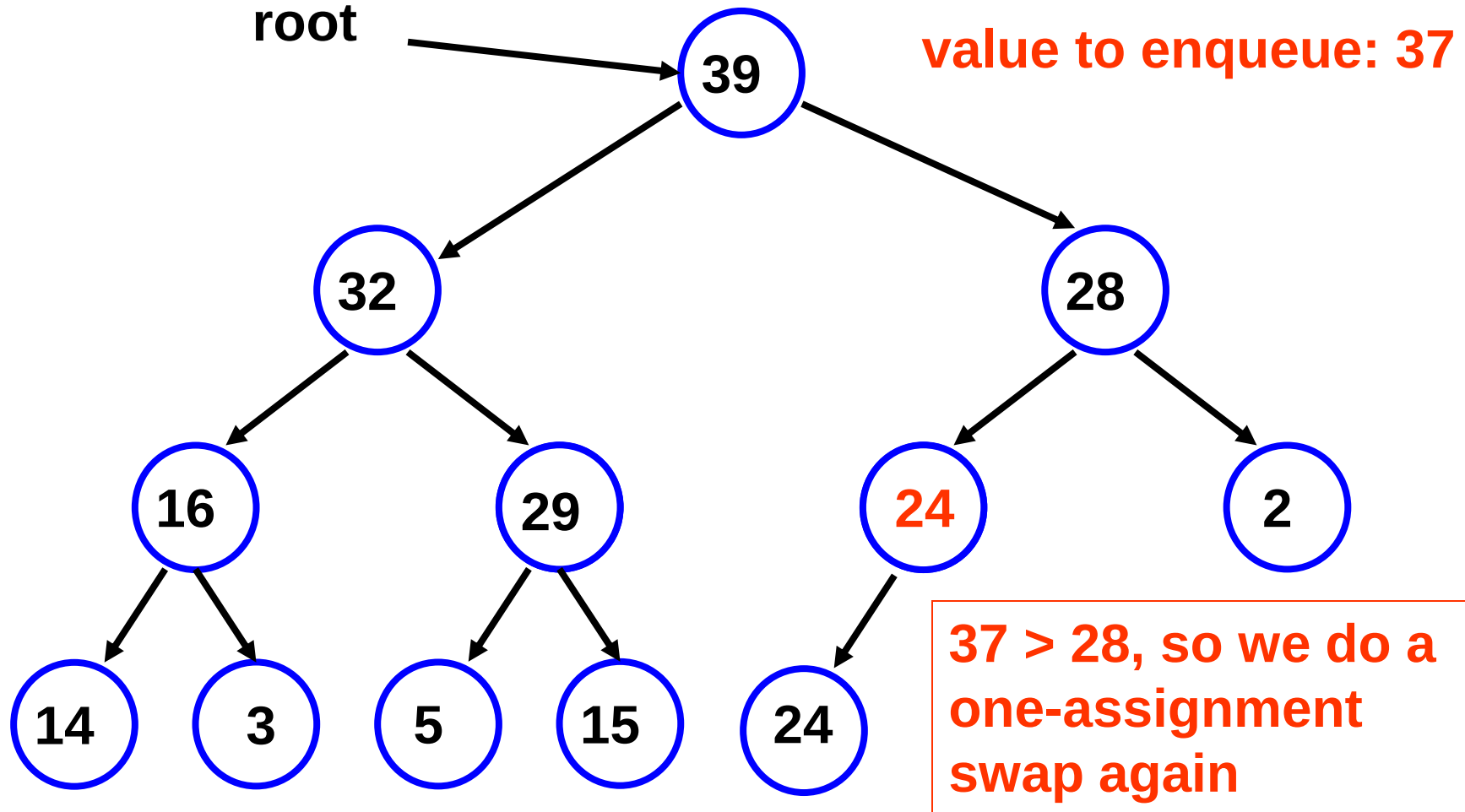
Enqueue (cont.)



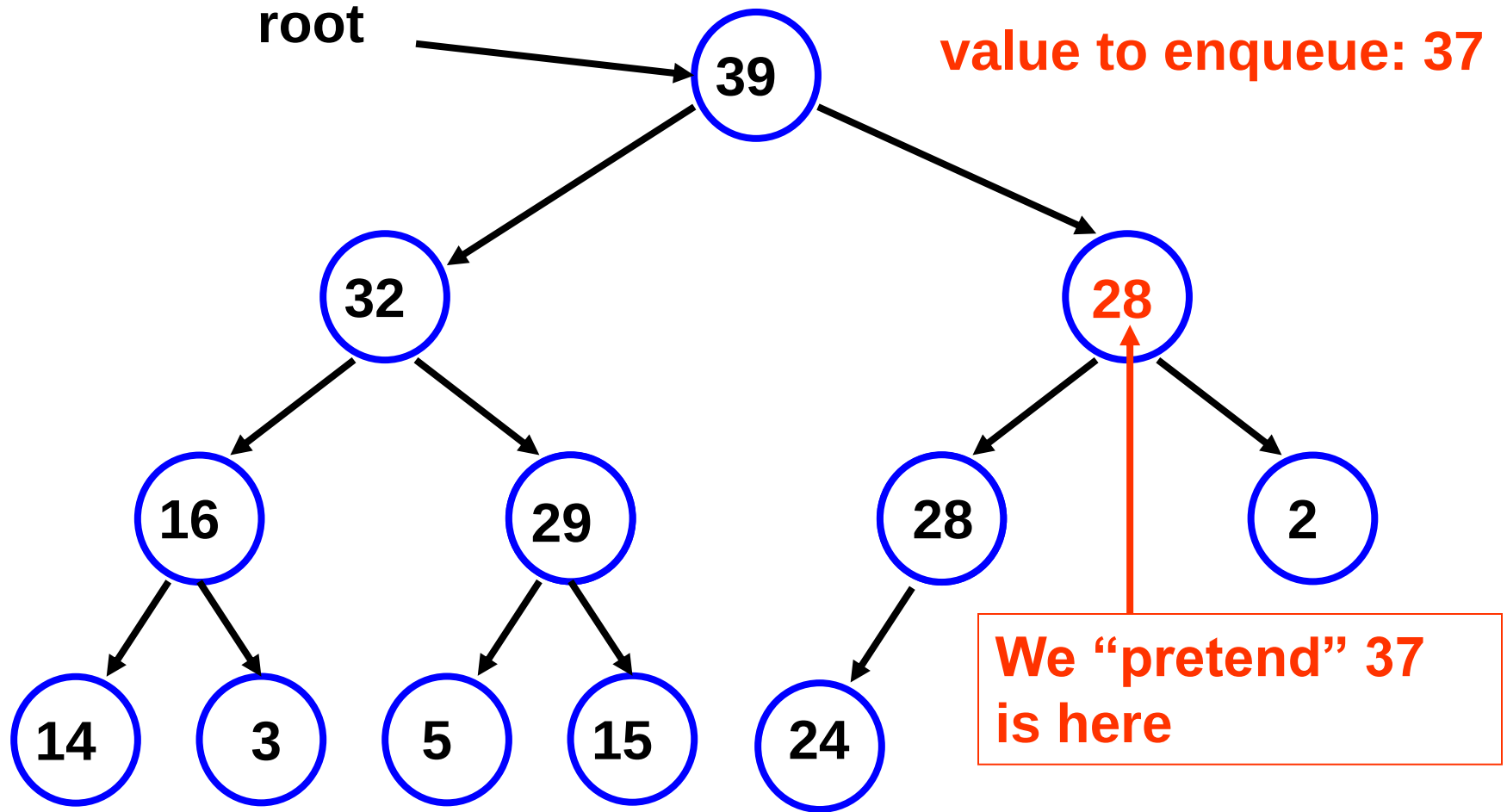
Enqueue (cont.)



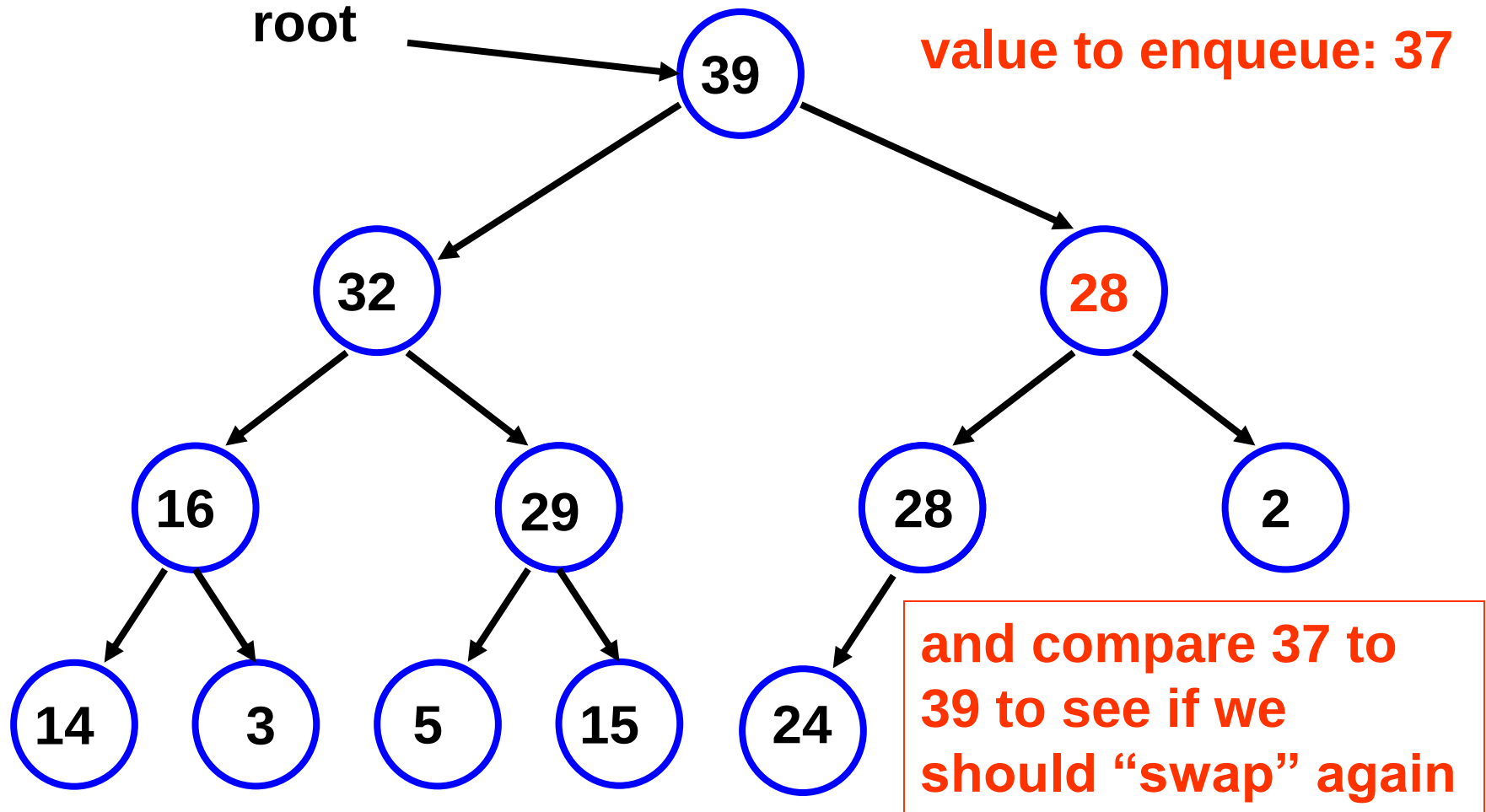
Enqueue (cont.)



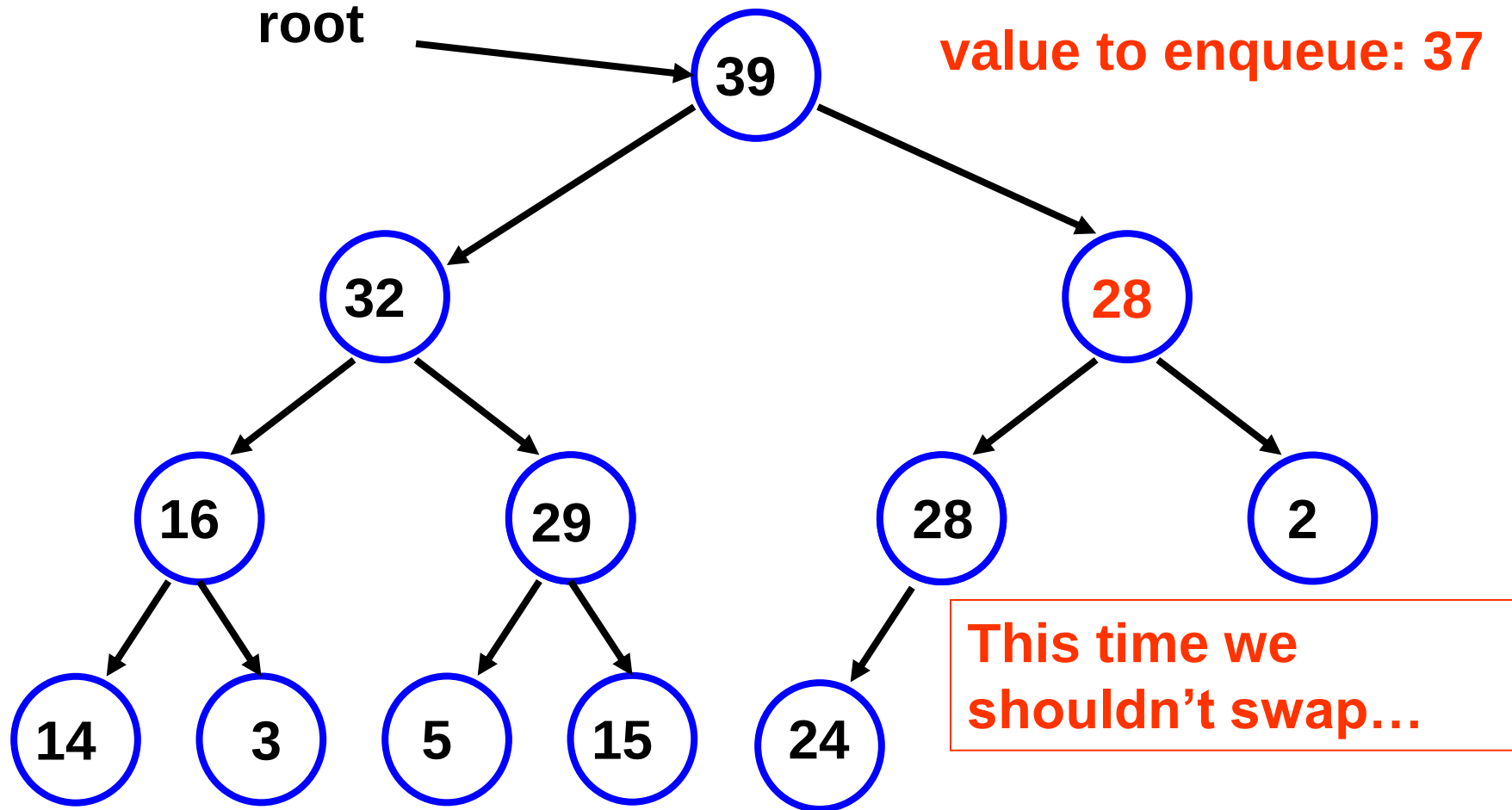
Enqueue (cont.)



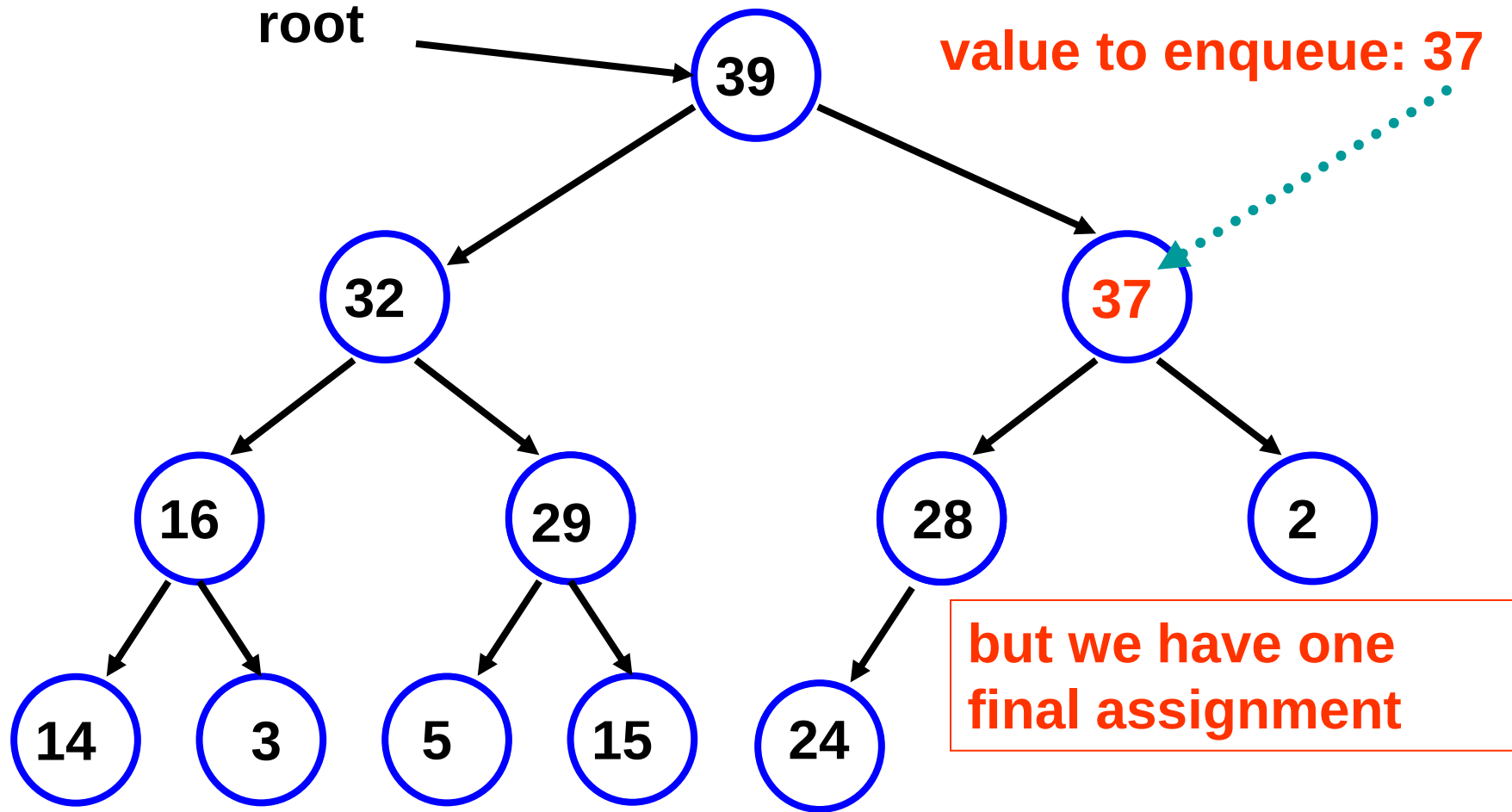
Enqueue (cont.)



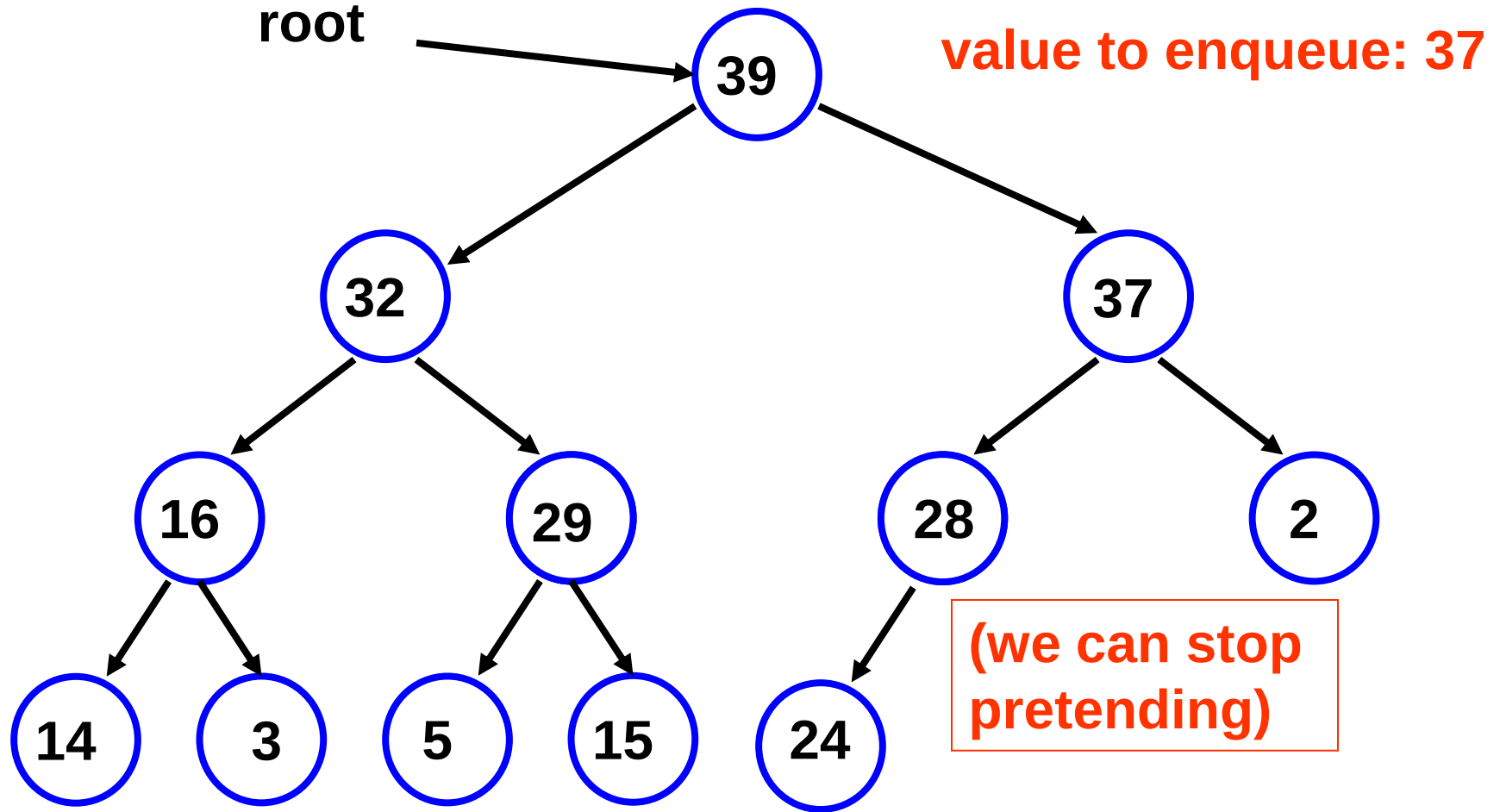
Enqueue (cont.)



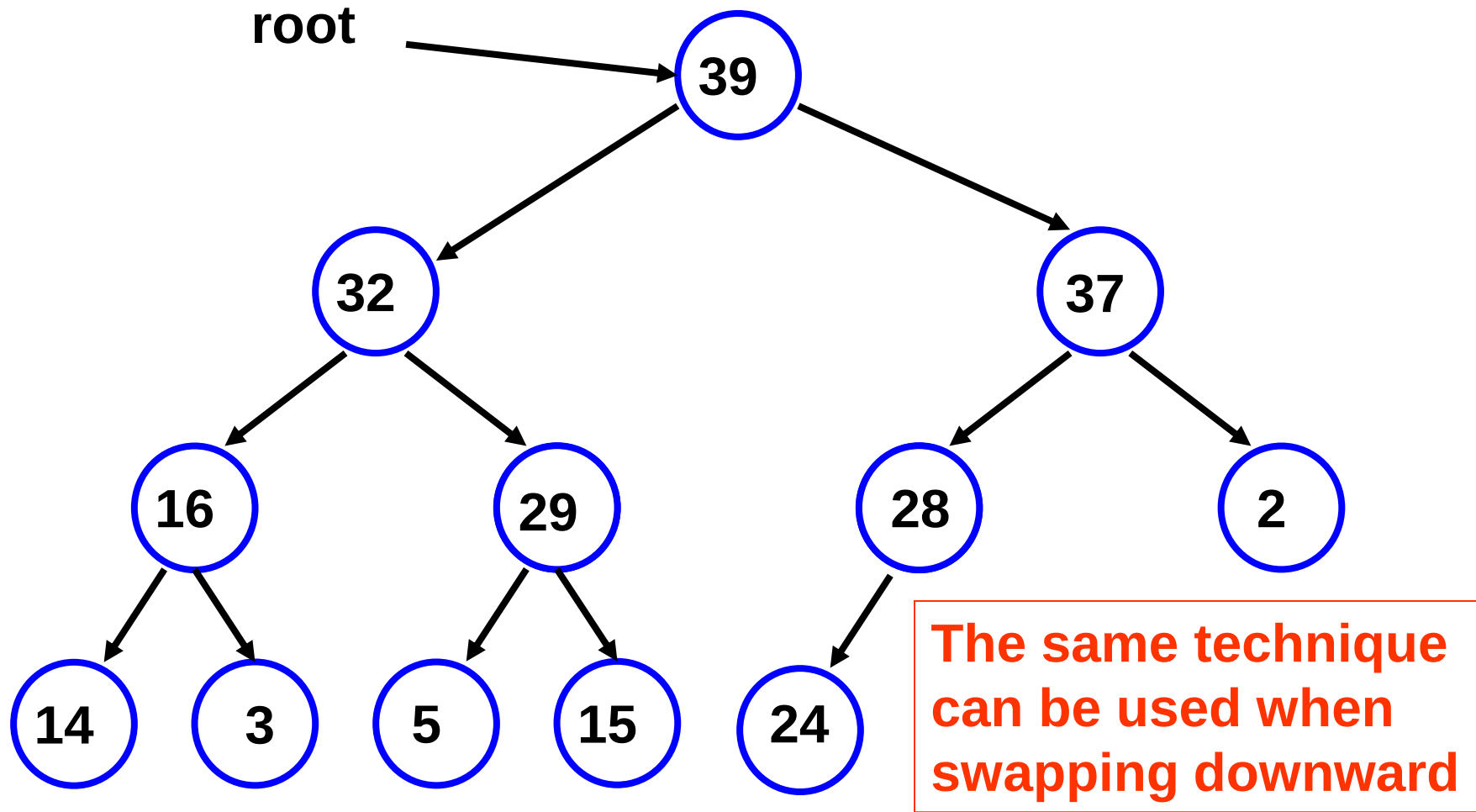
Enqueue (cont.)



Enqueue (cont.)



Dequeue



Parent-Child Formulas

- Parent-child formulas can also be sped up:

$$(\text{left child \#}) = (\text{parent \#}) \ll 1 + 1$$

- the shift $\ll$ operator is the same as integer multiplication by 2.

$$(\text{right child \#}) = (\text{left child \#}) + 1$$

- when finding the greatest child, the left and right children are always found together

$$(\text{parent \#}) = (\text{child \#} - 1) \gg 1$$

- the shift $\gg$ operator is the same as integer division by 2.

Forming an Initial Heap

- A heap of size n can be made by enqueueing n elements one at a time – but it would take $O(n \lg n)$ time.
- A faster method can be used to make a heap out of an initial array in $\Theta(n)$ time.
- The heap will be shown as a linked tree to help visualize how the method works – but in actuality, it is used on the array.

Forming an Initial Heap (cont.)

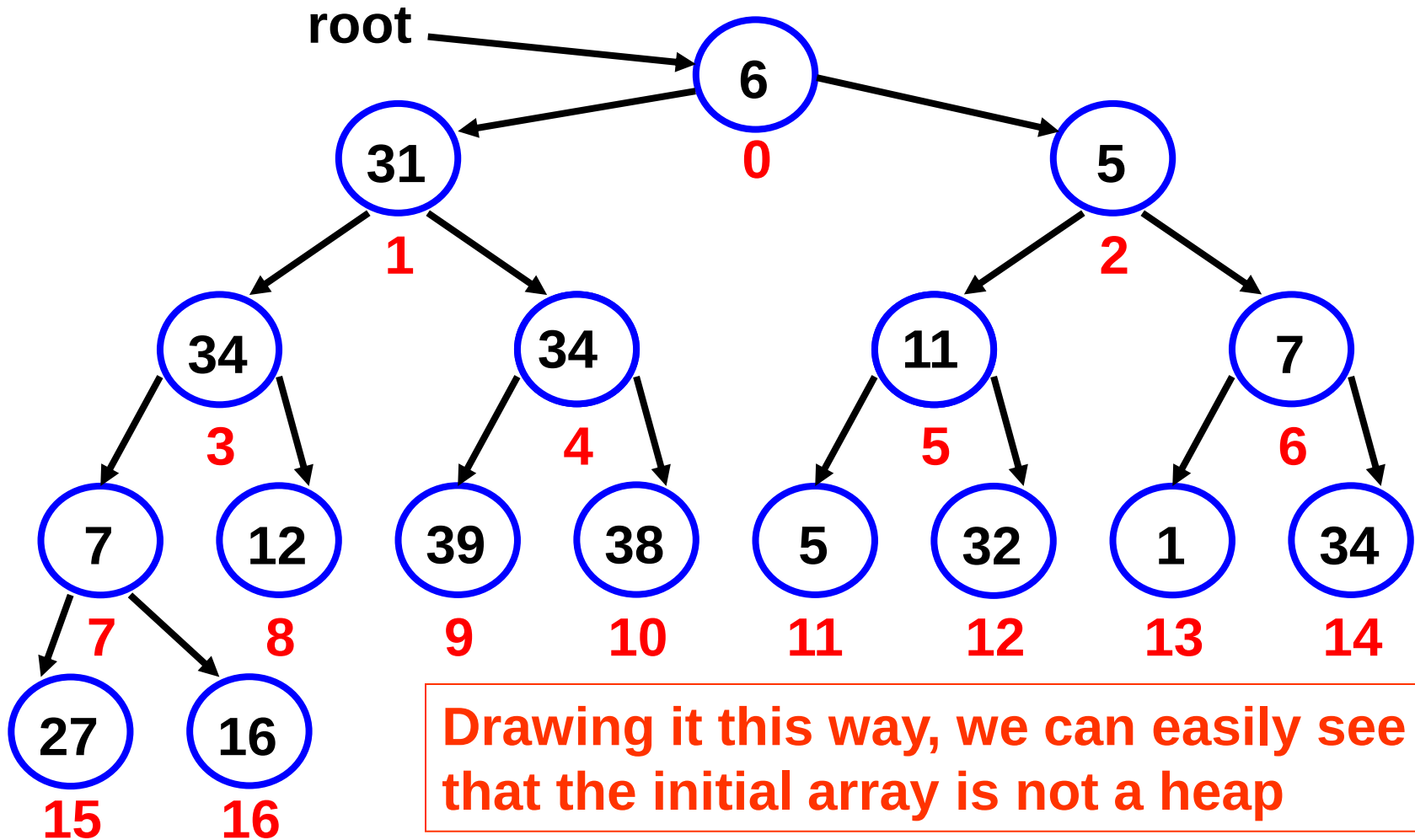
6	31	5	34	34	11	7	7	12	39	38	5	32	1	34	27	16
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Suppose the initial array looks like this.

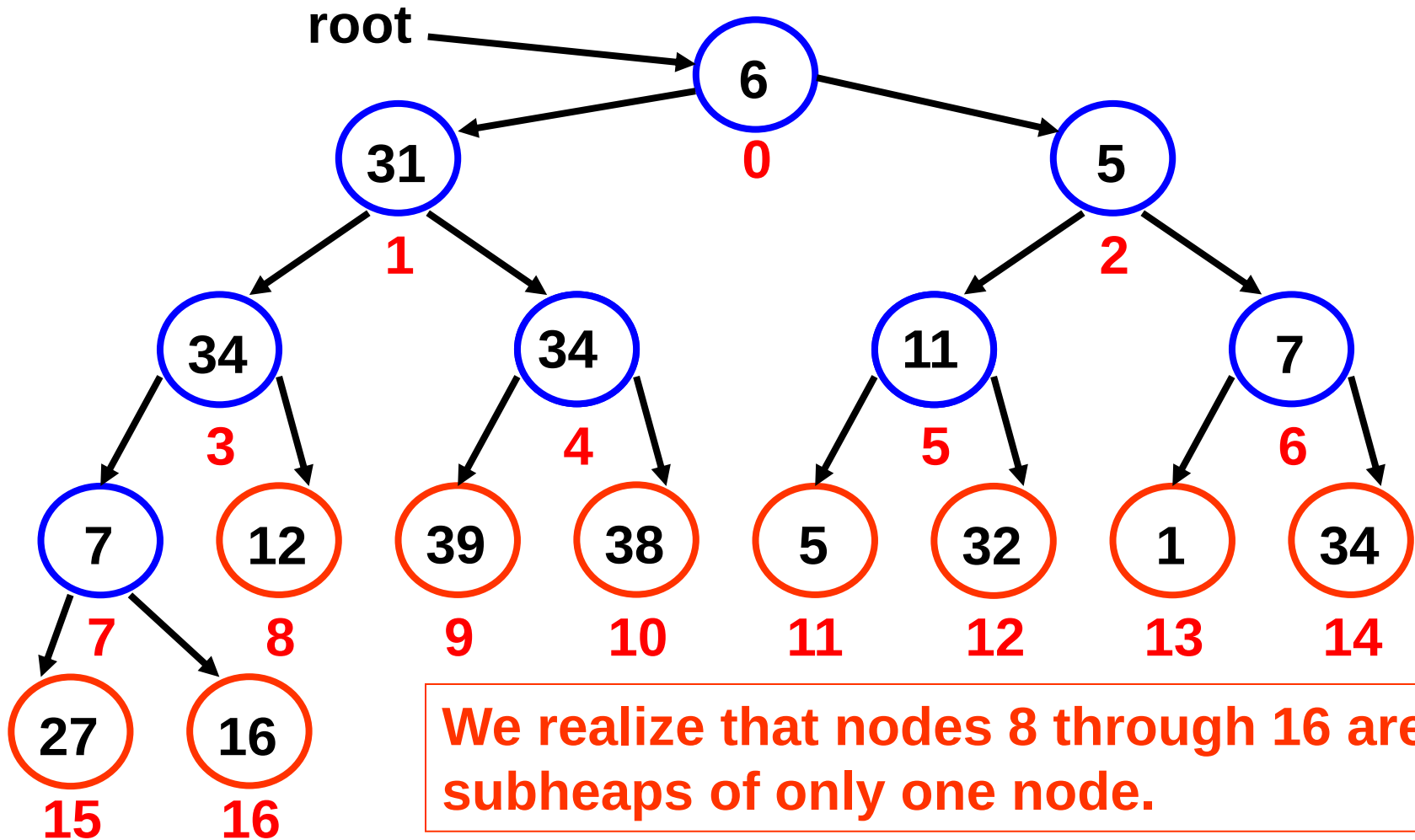
It is not initially a heap, but by rearranging the elements, it will be.

We will look at the tree form of this to see why the method works.

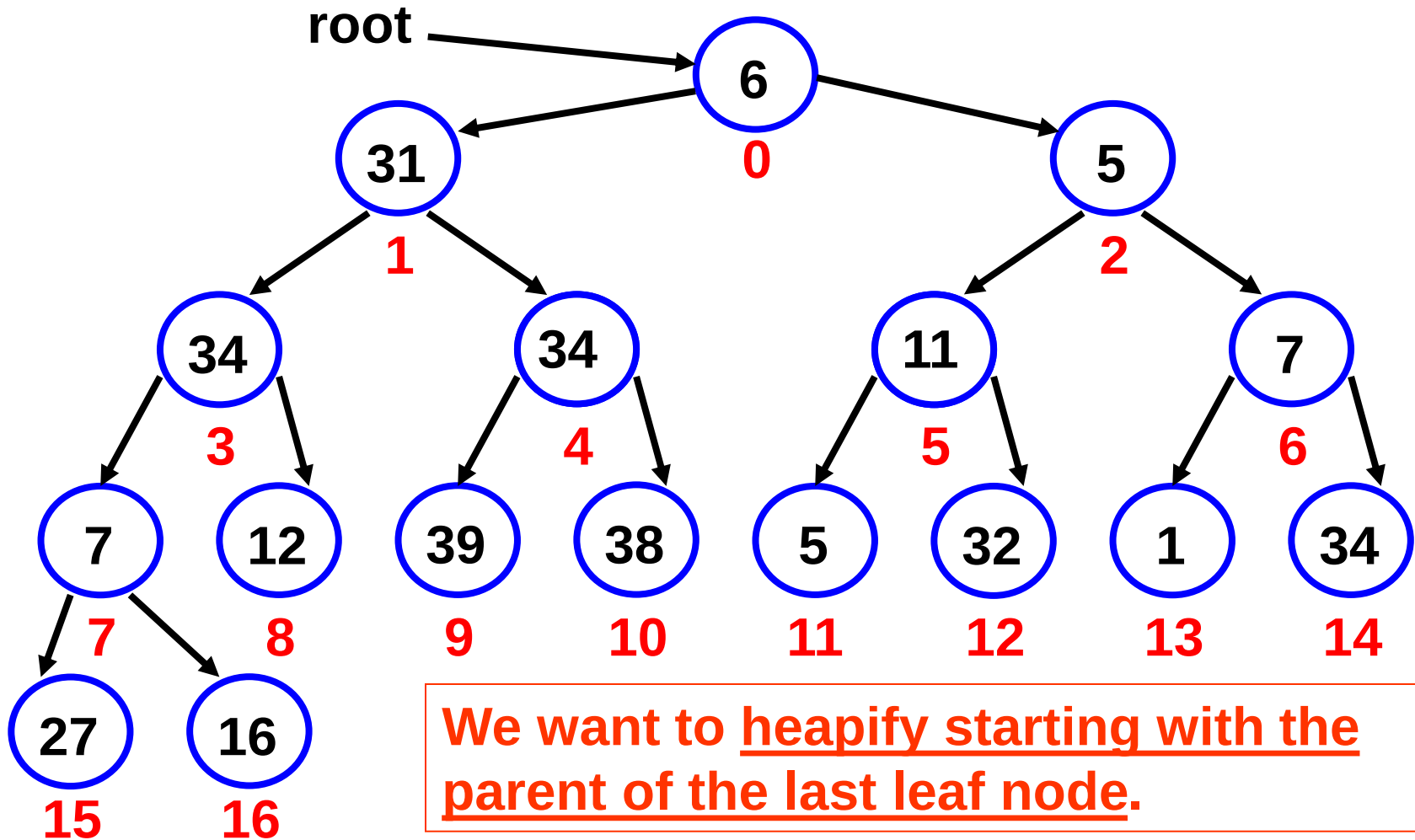
Forming an Initial Heap (cont.)



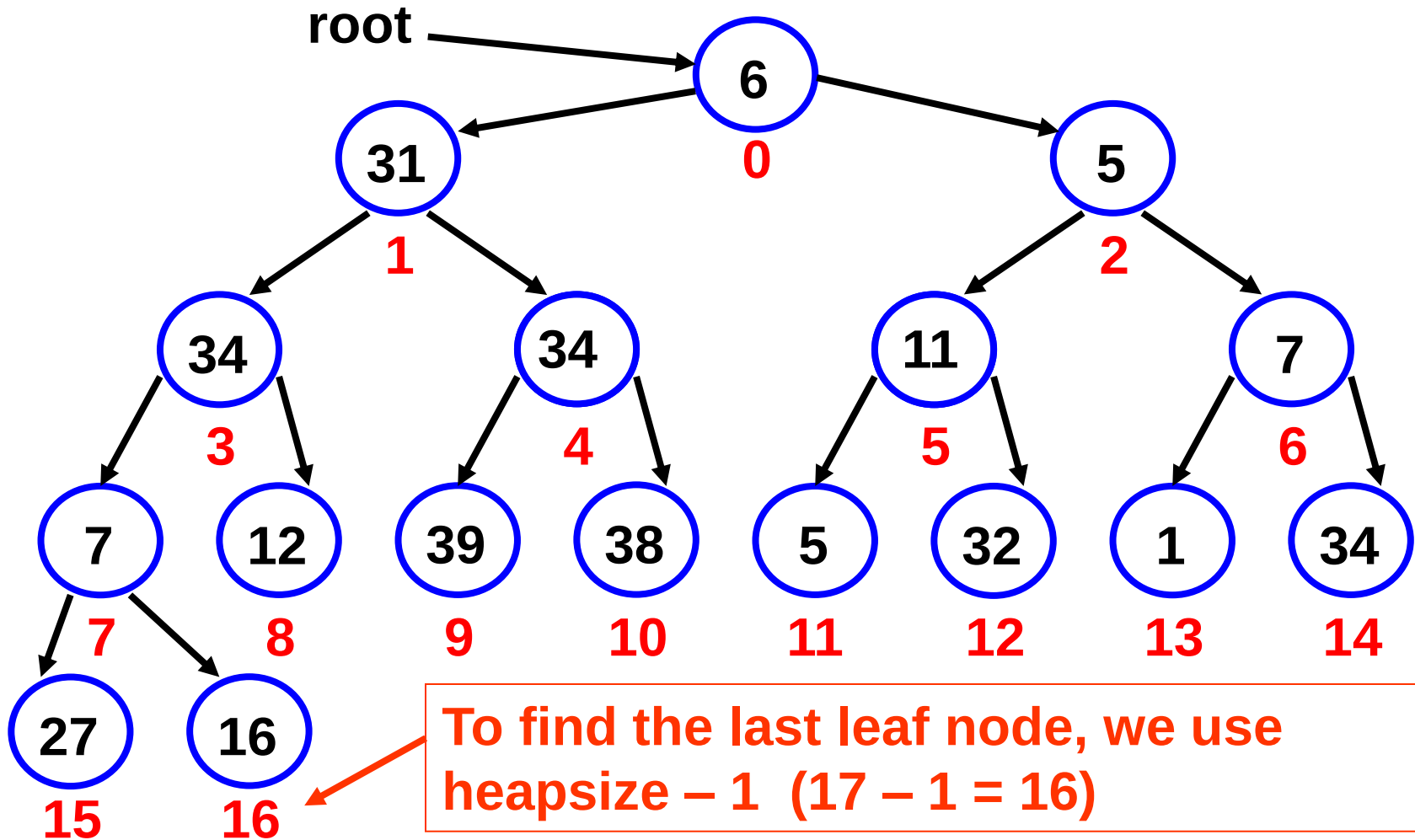
Forming an Initial Heap (cont.)



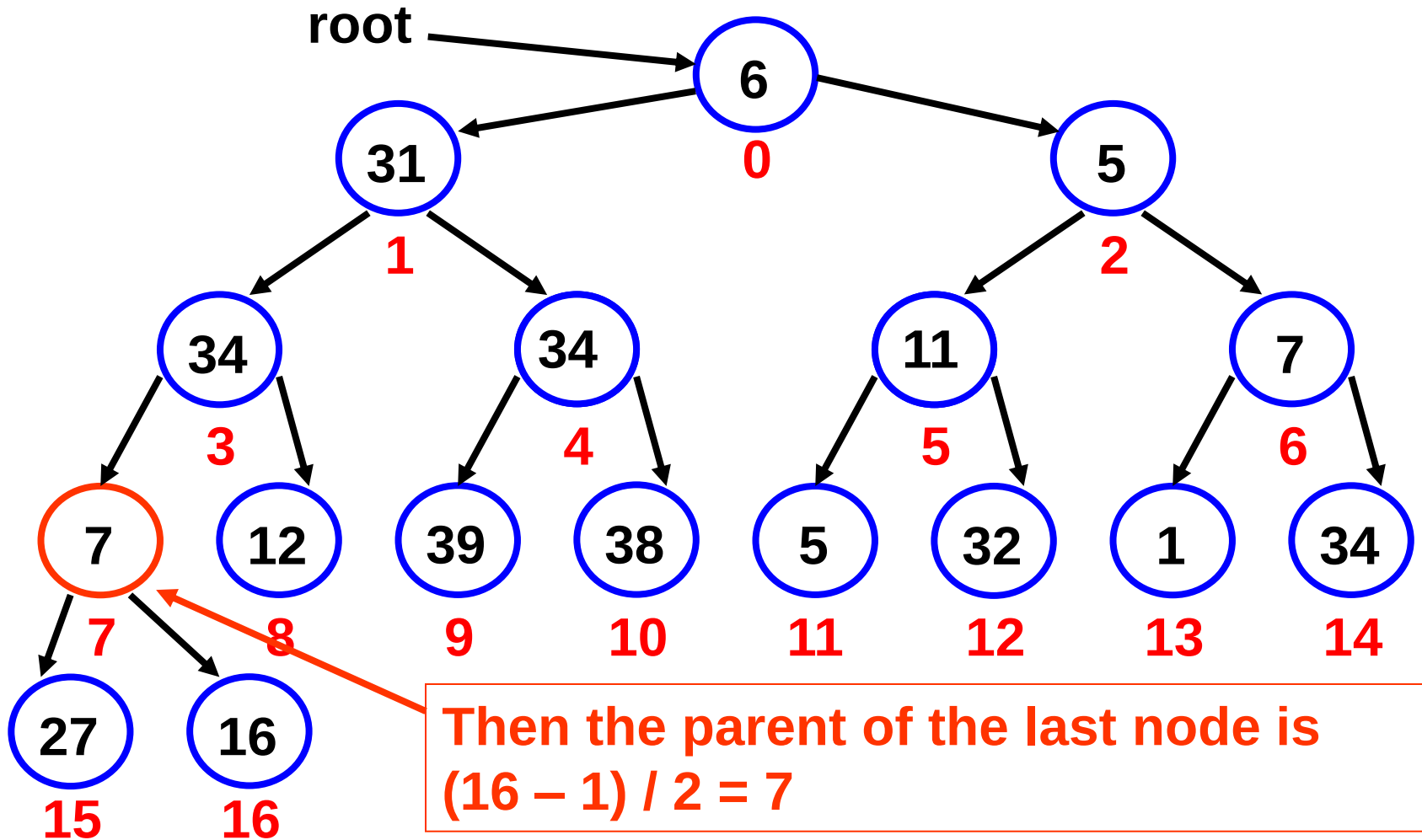
Forming an Initial Heap (cont.)



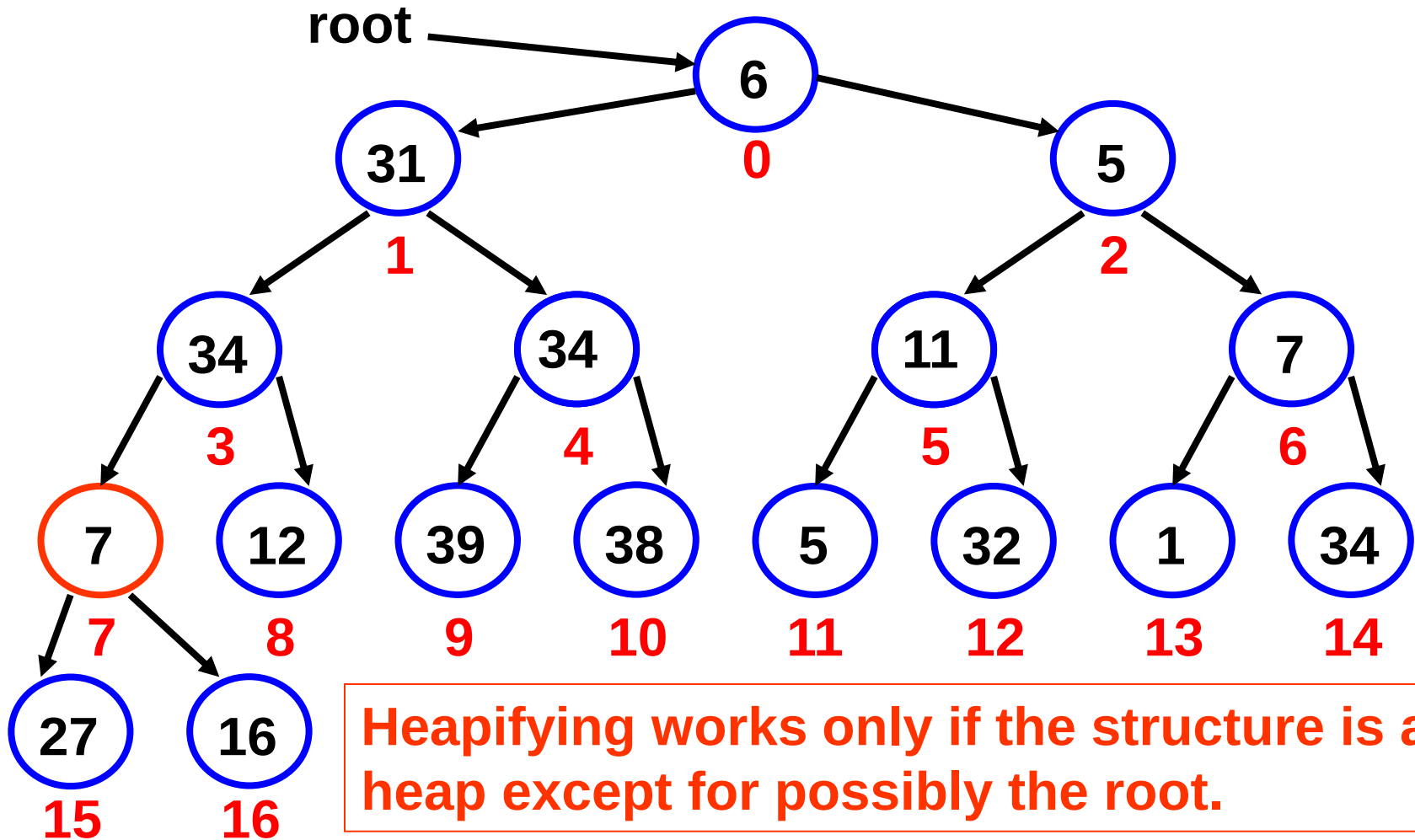
Forming an Initial Heap (cont.)



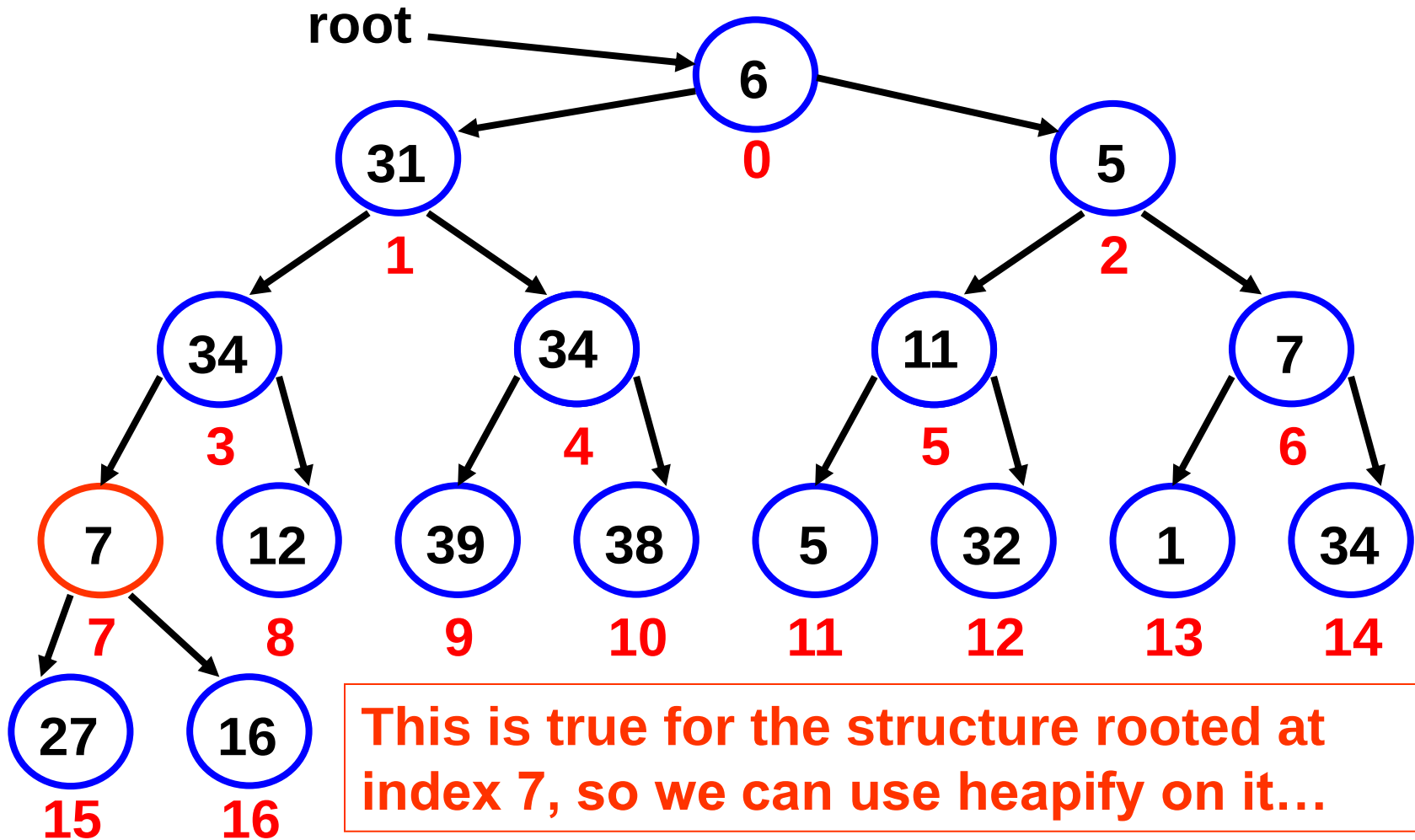
Forming an Initial Heap (cont.)



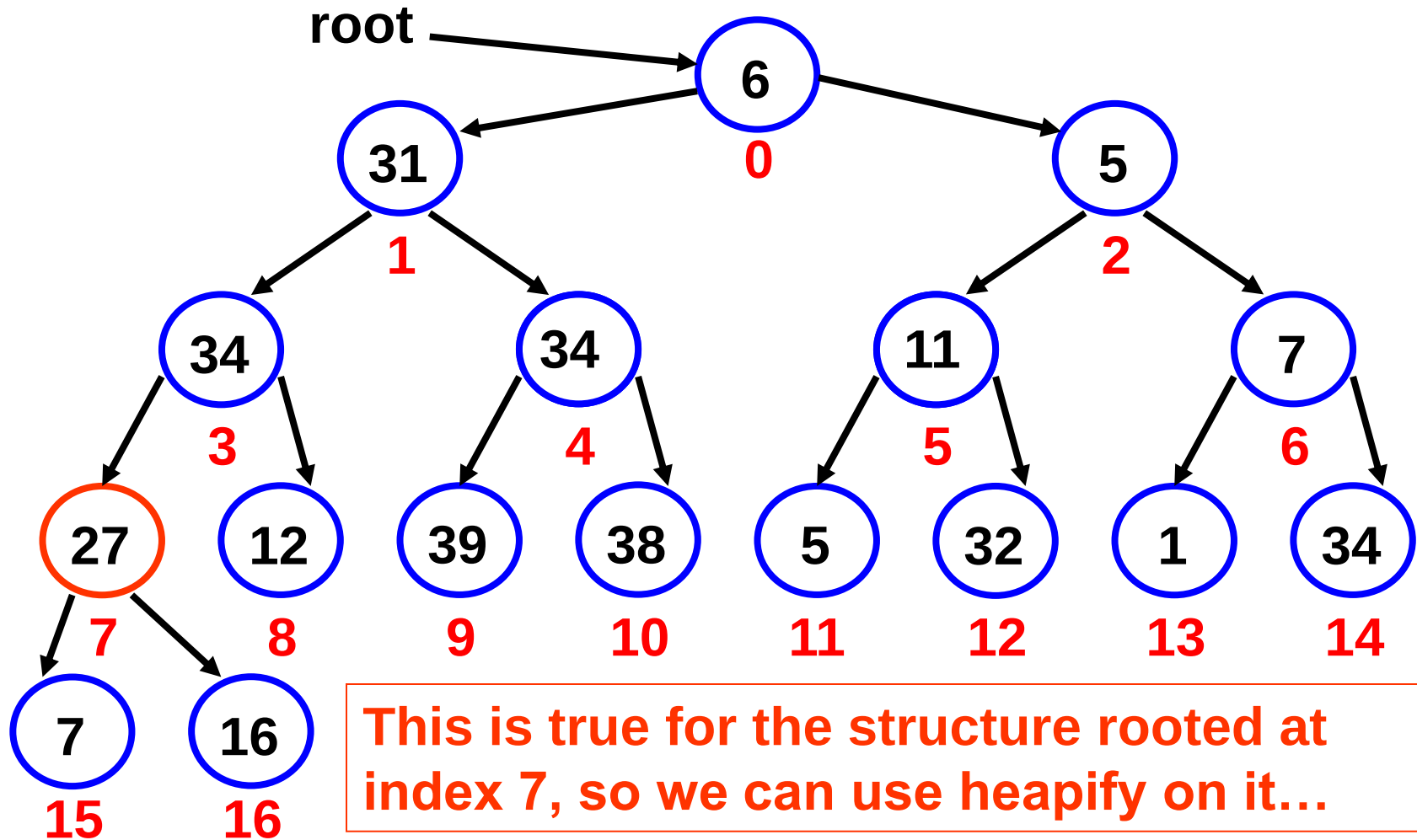
Forming an Initial Heap (cont.)



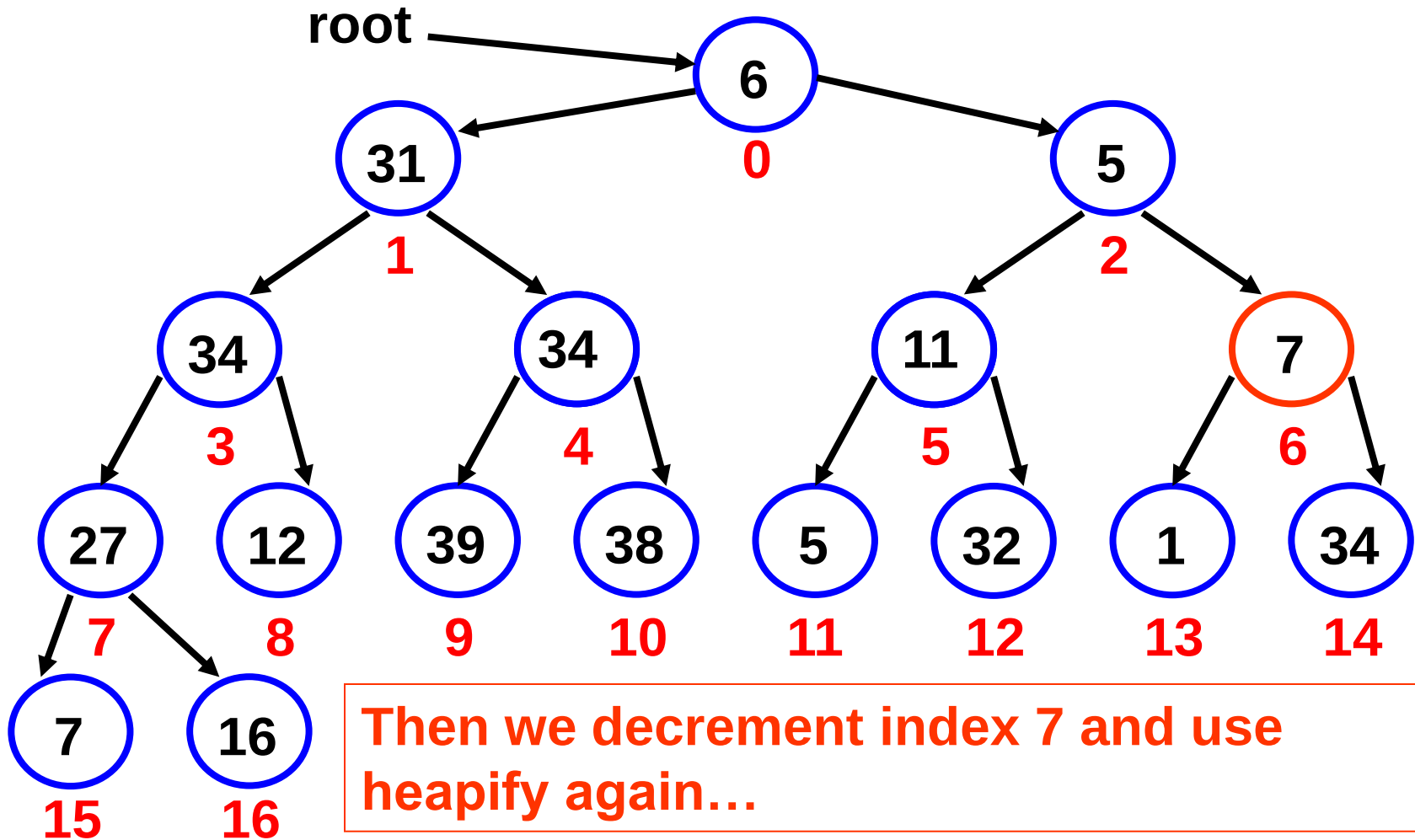
Forming an Initial Heap (cont.)



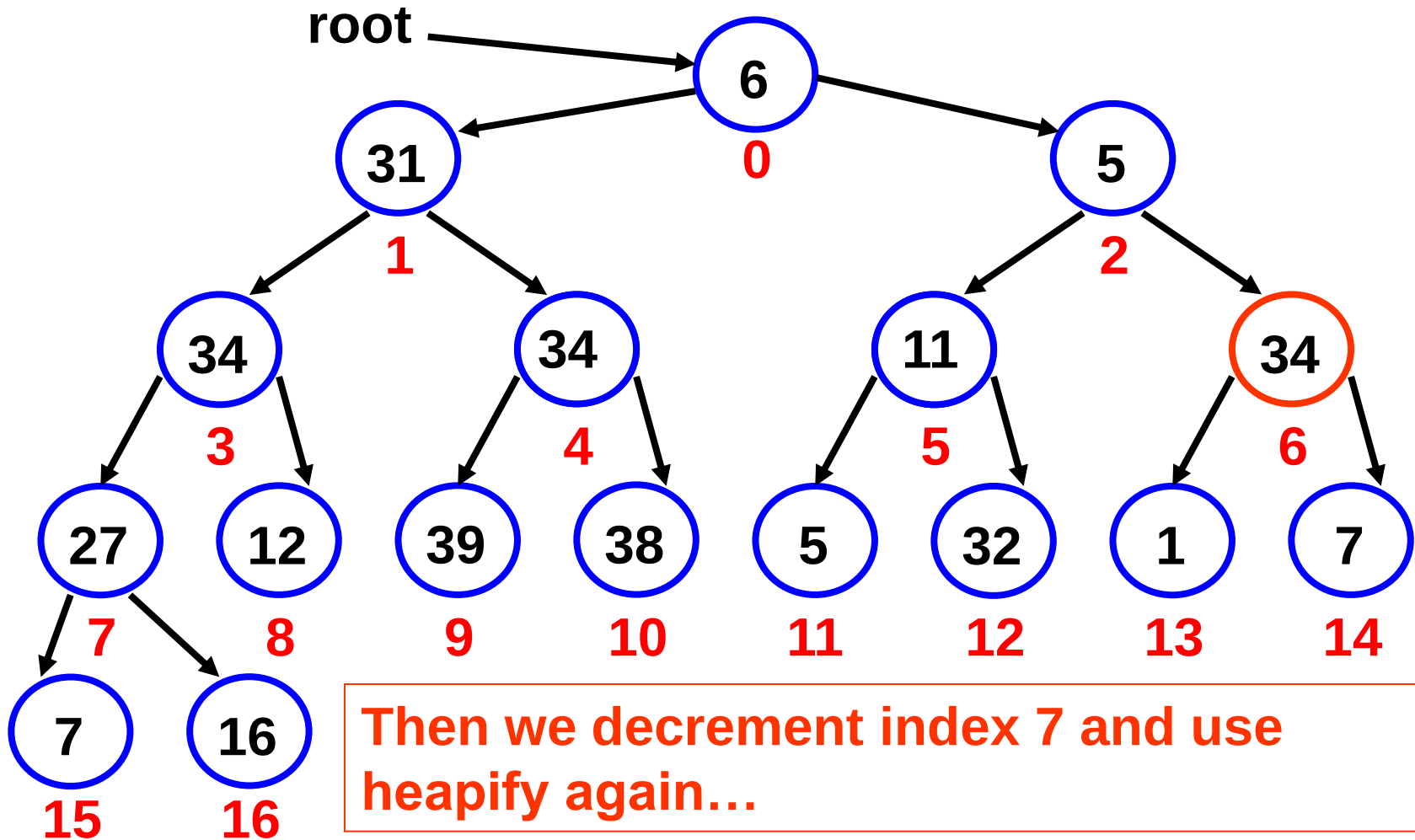
Forming an Initial Heap (cont.)



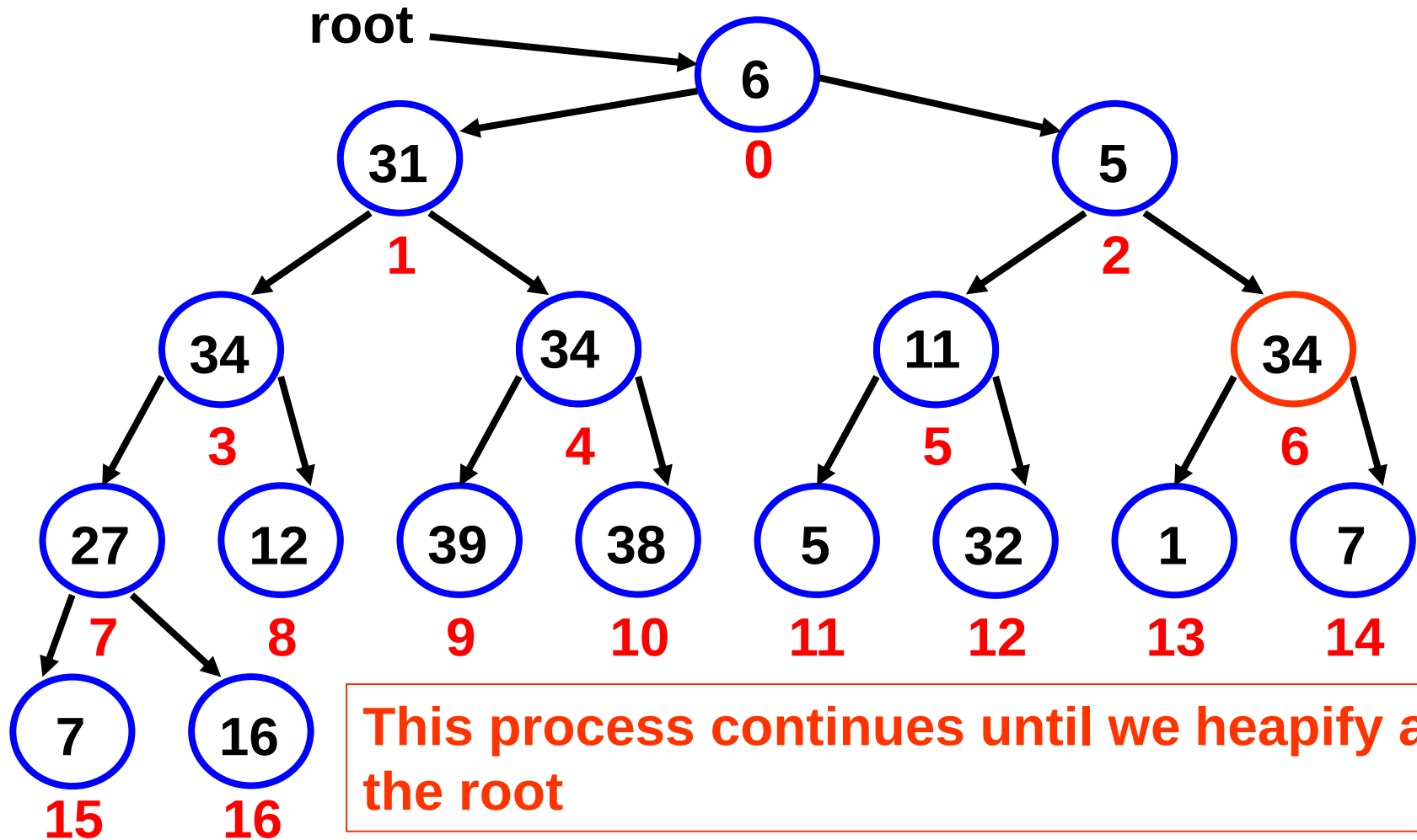
Forming an Initial Heap (cont.)



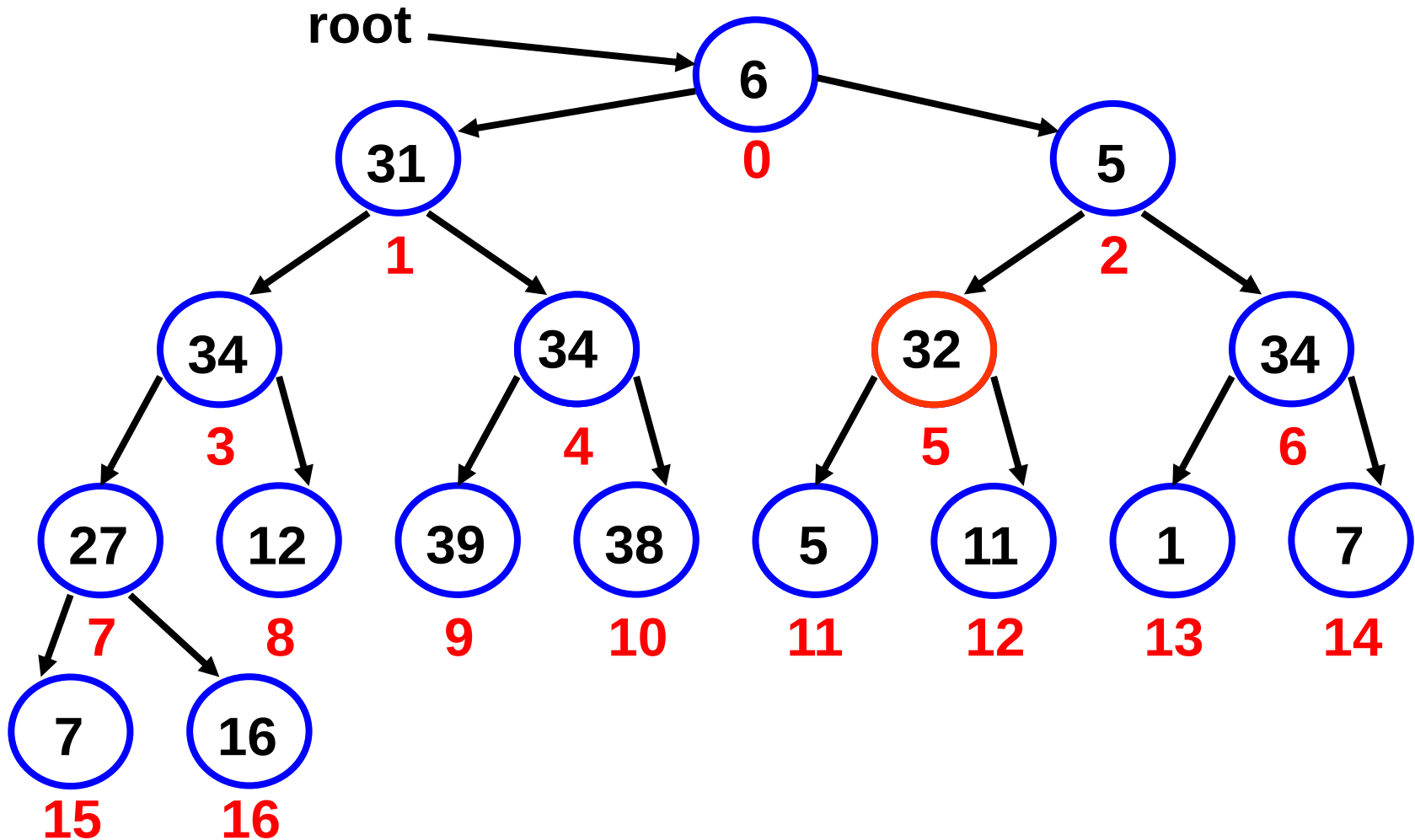
Forming an Initial Heap (cont.)



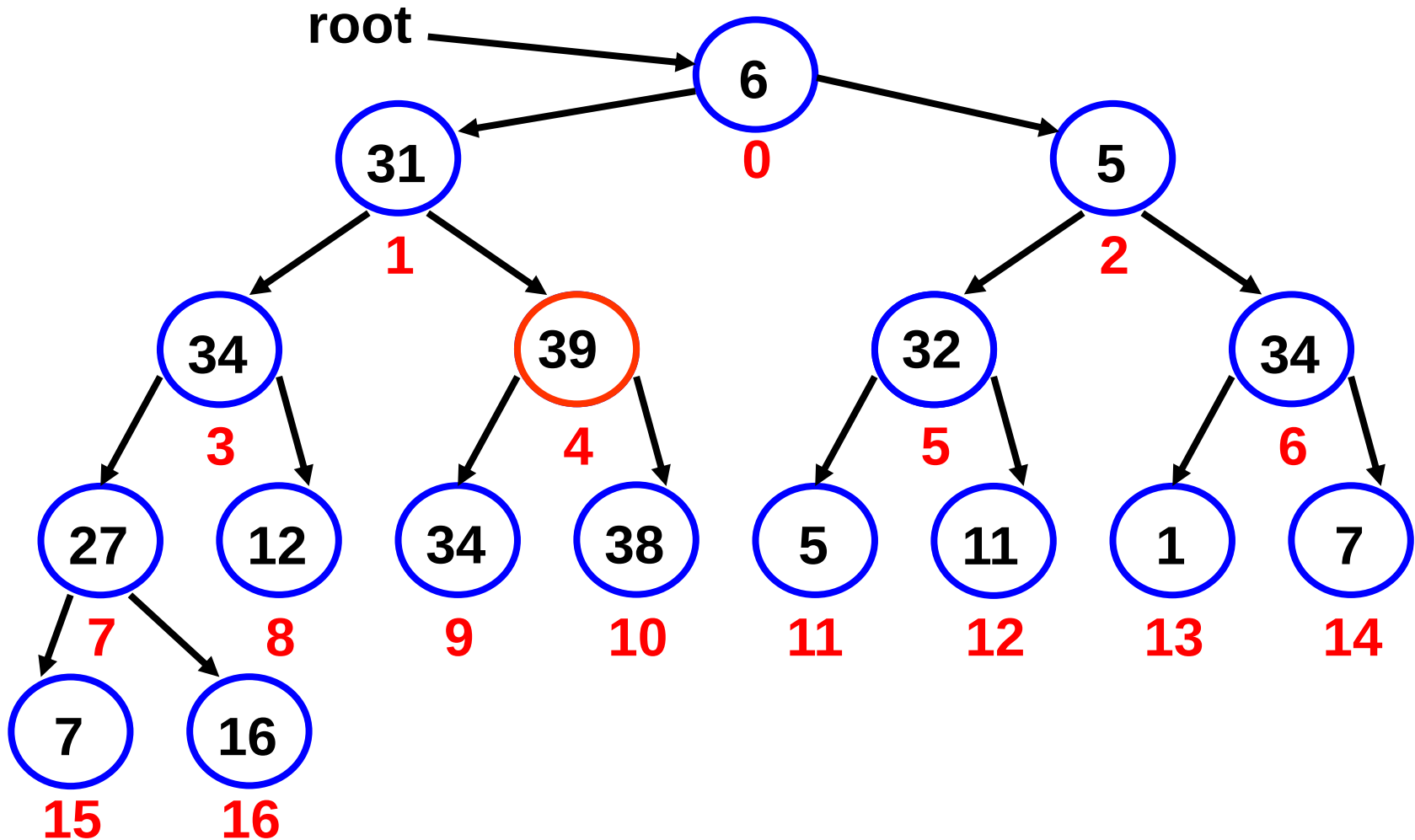
Forming an Initial Heap (cont.)



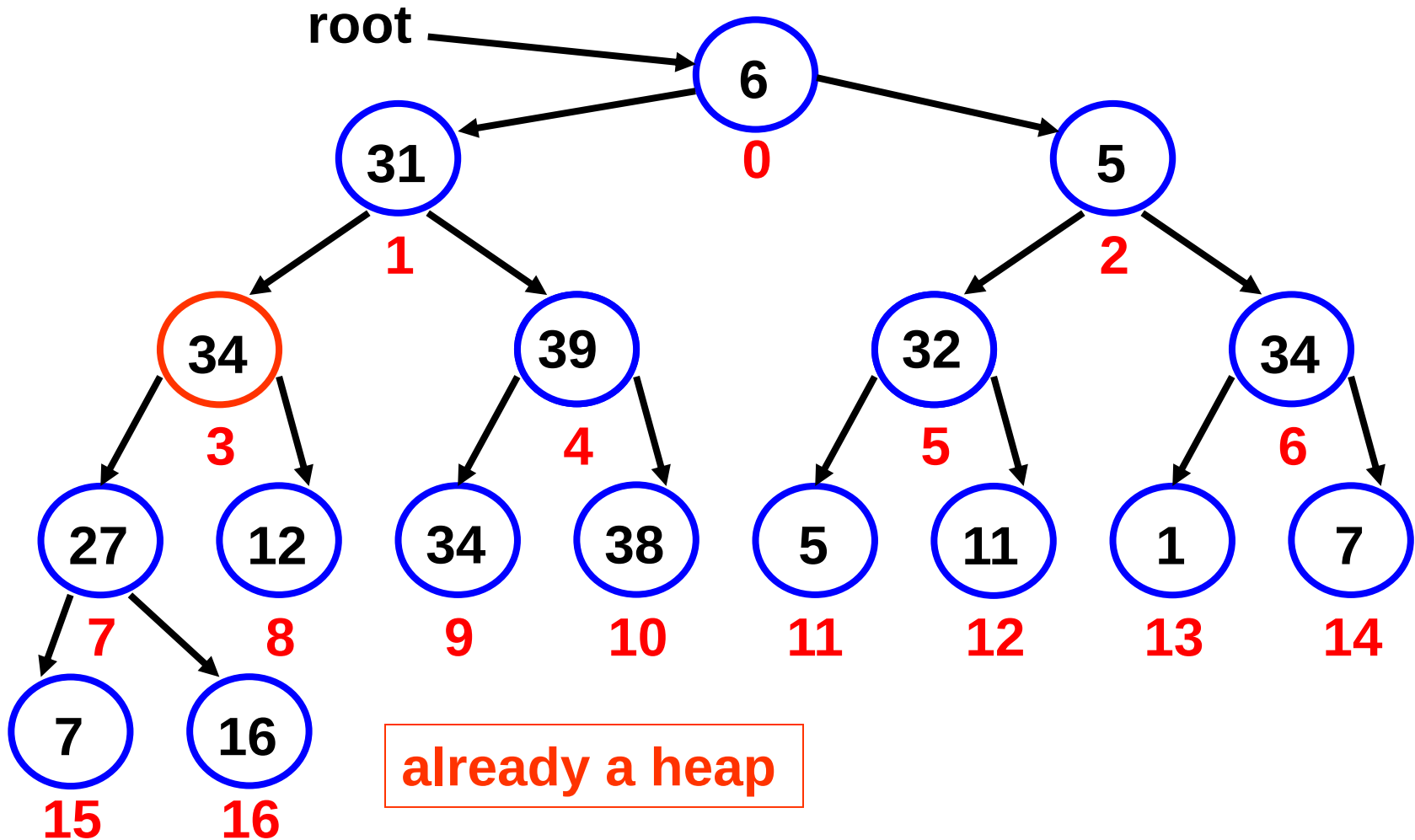
Forming an Initial Heap (cont.)



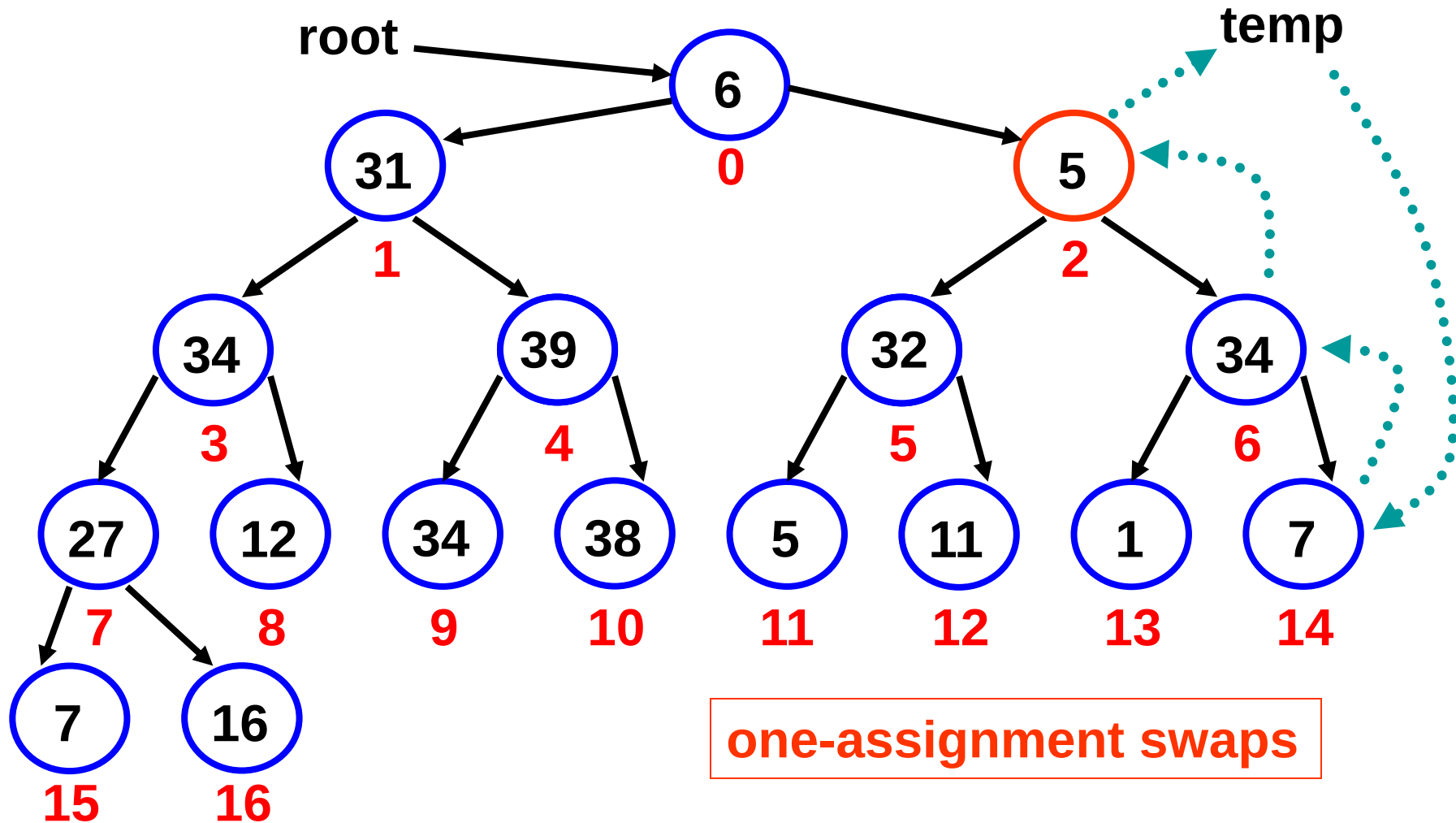
Forming an Initial Heap (cont.)



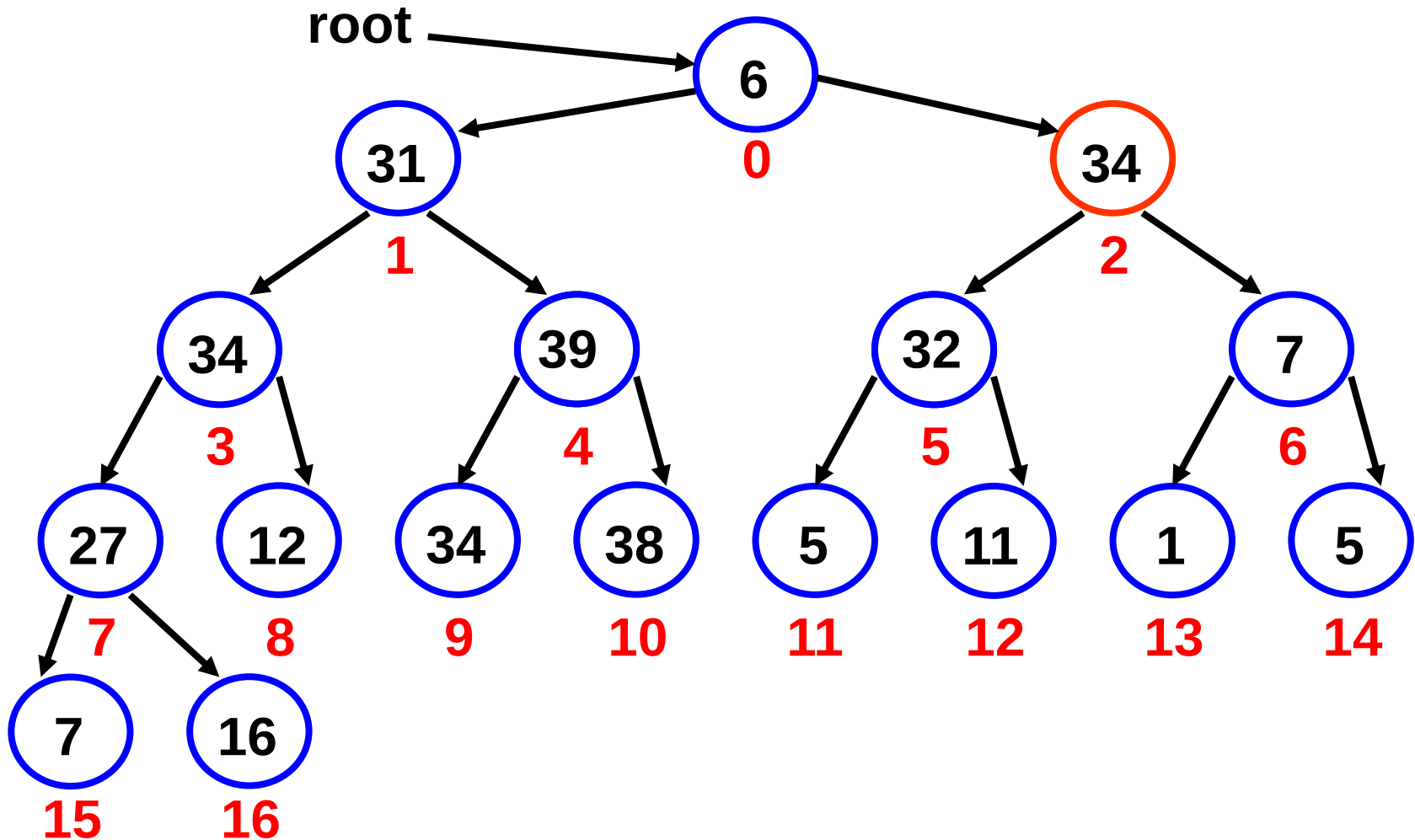
Forming an Initial Heap (cont.)



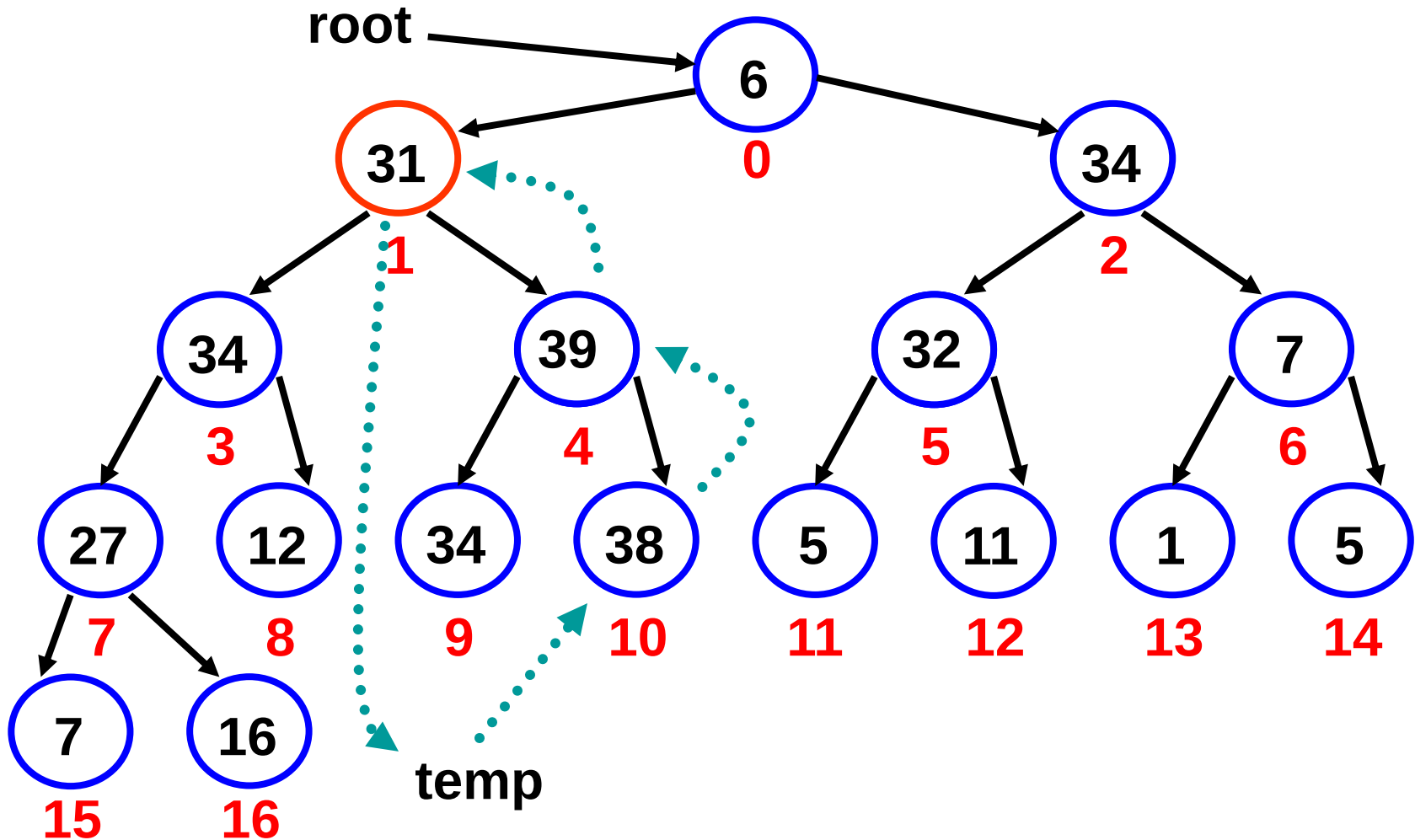
Forming an Initial Heap (cont.)



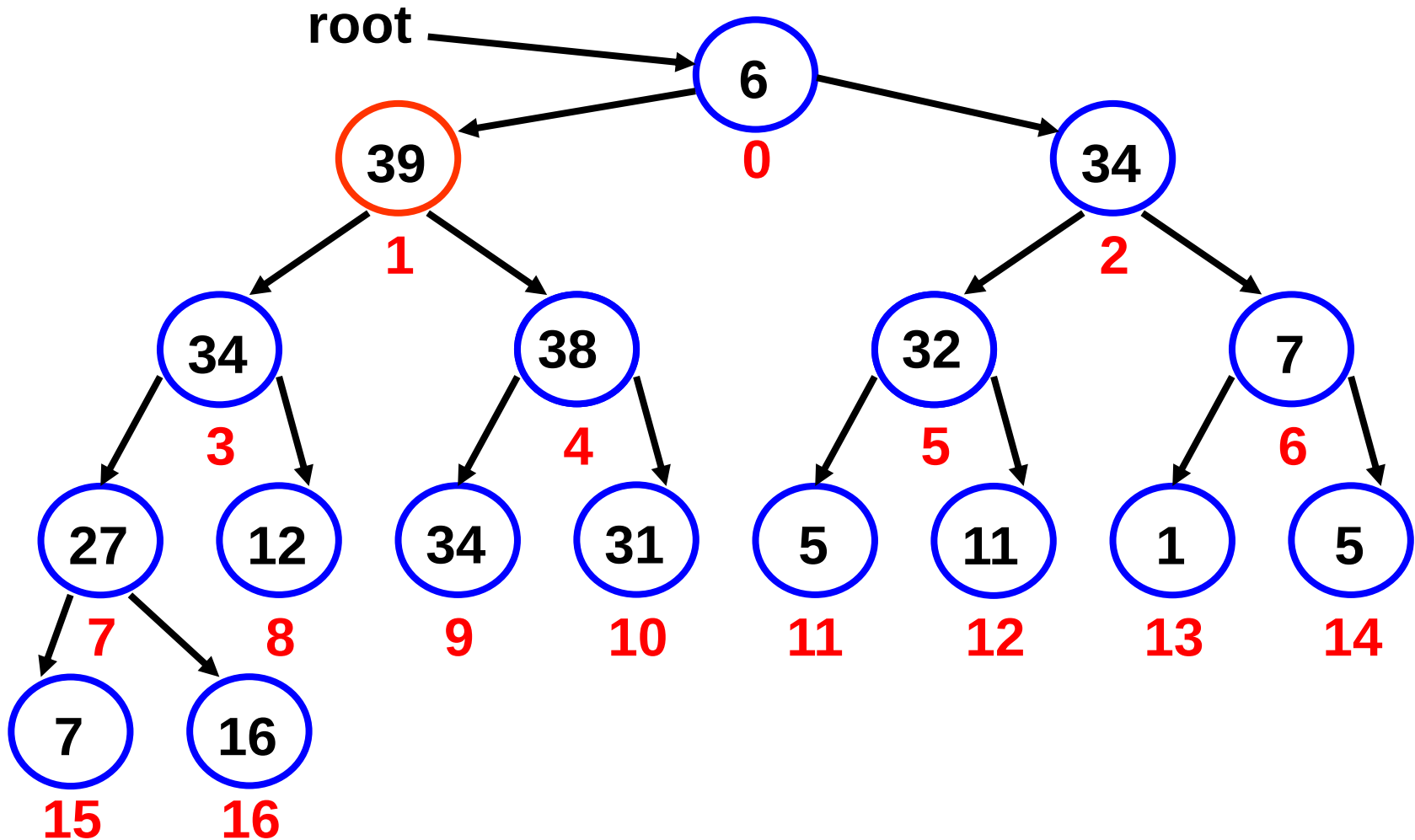
Forming an Initial Heap (cont.)



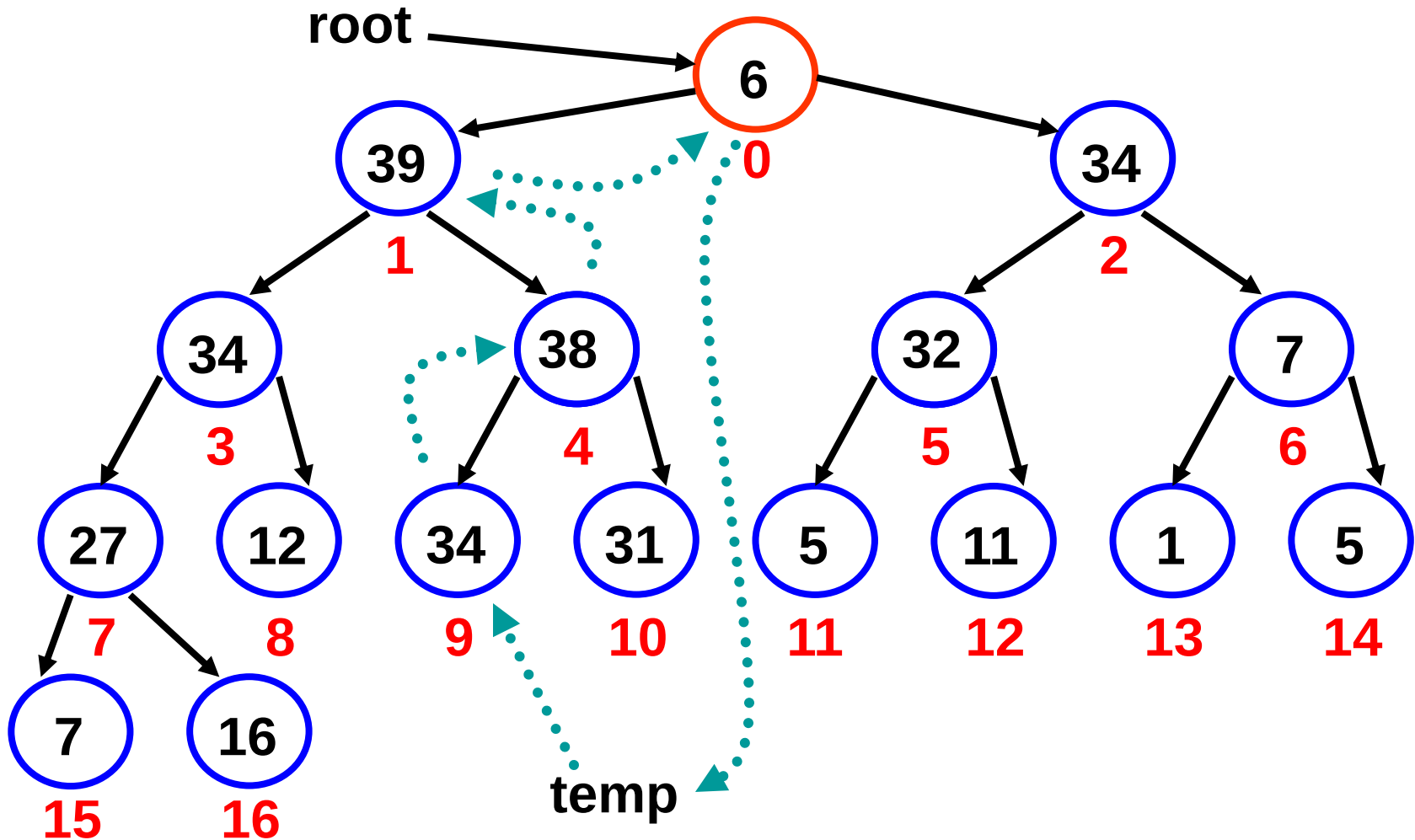
Forming an Initial Heap (cont.)



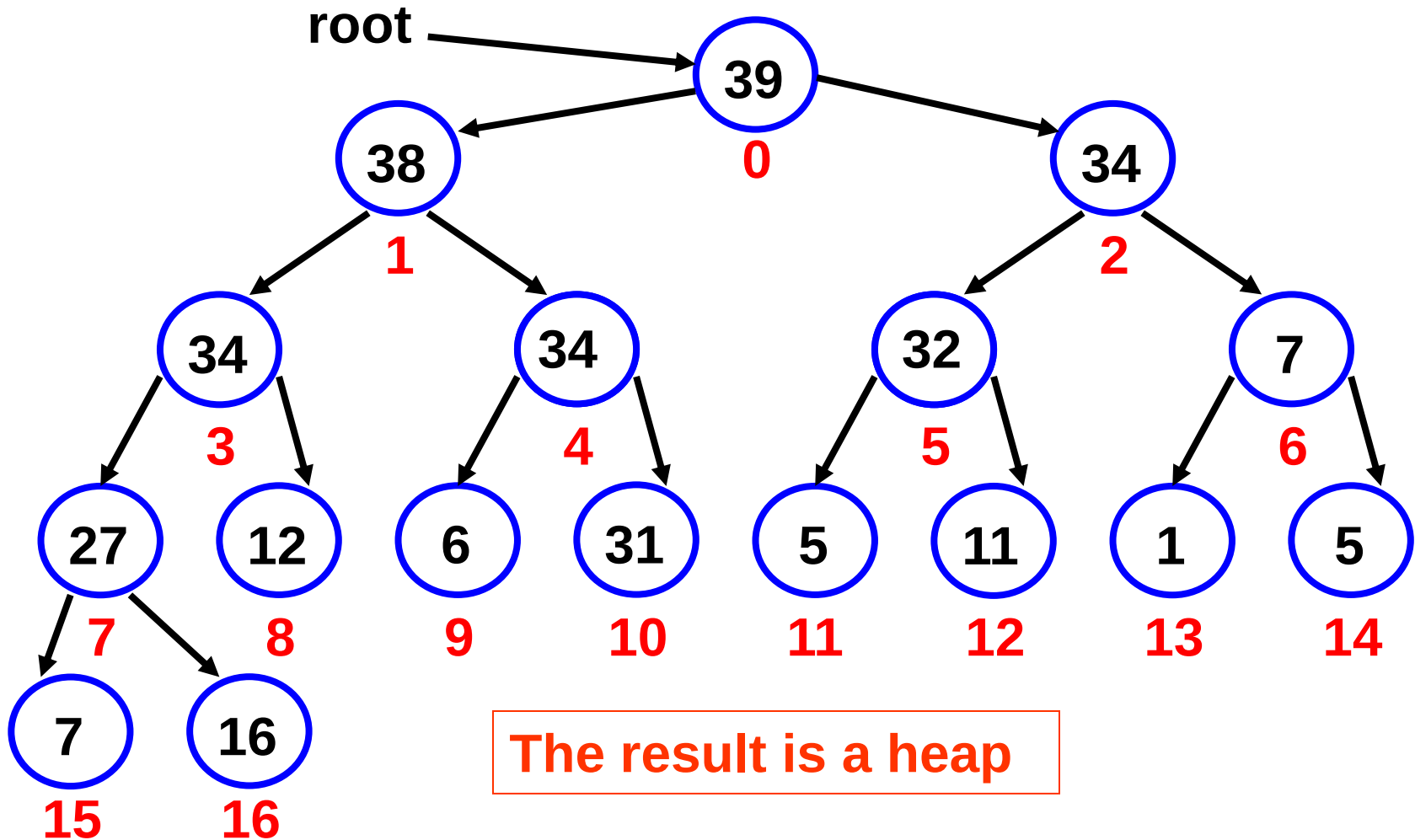
Forming an Initial Heap (cont.)



Forming an Initial Heap (cont.)



Forming an Initial Heap (cont.)



Linked Heap

- If we have a large element size and we don't know the max size, we might want to consider conserving memory and avoiding resizing array with a linked heap.
- Should have the same time complexities as the array-based heap.